Exploratory Data Analysis Lab Project

Name: Dharsha R
Register Number: 23BDS0274
Course Code: BCSE331P
Course Name: Exploratory Data Analysis
Project Title: Telco Customer Churn Analysis Using Python
Platform: Jupyter Notebook

Project Overview

This project focuses on performing Exploratory Data Analysis (EDA) on the Telco Customer Churn dataset. The dataset contains customer information, subscribed services, contract details, payment methods, monthly charges, total charges, and customer churn status.

The project covers data importing, cleaning, transformation, statistical analysis, data visualization, bivariate and multivariate analysis, clustering, and Principal Component Analysis (PCA).

Objectives

- Load and explore the Telco Customer Churn dataset.
- Identify and handle missing values and duplicate records.
- Perform data cleaning and transformation.
- Calculate descriptive statistics and analyze variable distributions.
- Visualize relationships between numerical and categorical variables.
- Perform correlation analysis and multivariate visualization.
- Apply K-Means and Hierarchical Clustering.
- Perform Principal Component Analysis (PCA).
- Analyze the AirPassengers time series and generate forecasts.
- Interpret the results to understand customer behavior and churn patterns.

Dataset Information

Dataset: Telco Customer Churn
Source: GitHub Dataset

The dataset includes customer demographics, account information, subscribed services, billing details, and the target variable `Churn`.

Tools and Libraries

- Python
- Jupyter Notebook
- Pandas and NumPy
- Matplotlib and Seaborn
- SciPy
- Scikit-learn
- Statsmodels

Expected Outcome

The project aims to identify useful patterns in customer data, understand relationships among variables, explore customer groups using clustering, reduce dimensionality using PCA, and develop an understanding of customer churn through statistical analysis and visualization.

Assessment 1: Merging and Data Transformation

Part A: DataFrame Operations

Create three dataframes and perform append, concatenation, merging, and join operations using ProductID.

In [1]:
```
import pandas as pd
import numpy as np

dataframe1 = pd.DataFrame({
    "ProductID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "NameofProd": ["Pen", "Book", "Pencil", "Bag", "Scale",
                   "Eraser", "Marker", "File", "Bottle", "Box"],
    "Price": [10, 50, 5, 300, 20, 5, 25, 40, 100, 60]
})

dataframe2 = pd.DataFrame({
    "ProductID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Company_Location": ["Chennai", "Vellore", "Delhi", "Mumbai",
                         "Pune", "Salem", "Bangalore", "Hyderabad",
                         "Coimbatore", "Trichy"]
})

dataframe3 = pd.DataFrame({
    "ProductID": [11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "NameofProd": ["Laptop", "Mouse", "Keyboard", "Monitor", "Printer",
                   "Speaker", "Cable", "Charger", "Tablet", "Webcam"],
```

```
    "Price": [50000, 500, 1000, 12000, 8000,
              1500, 200, 700, 20000, 2500]
})

print("Dataframe 1")
display(dataframe1)

print("Dataframe 2")
display(dataframe2)

print("Dataframe 3")
display(dataframe3)
```

```
Dataframe 1
```

	ProductID	NameofProd	Price
0	1	Pen	10
1	2	Book	50
2	3	Pencil	5
3	4	Bag	300
4	5	Scale	20
5	6	Eraser	5
6	7	Marker	25
7	8	File	40
8	9	Bottle	100
9	10	Box	60

```
Dataframe 2
```

	ProductID	Company_Location
0	1	Chennai
1	2	Vellore
2	3	Delhi
3	4	Mumbai
4	5	Pune
5	6	Salem
6	7	Bangalore
7	8	Hyderabad
8	9	Coimbatore
9	10	Trichy

```
Dataframe 3
```

	ProductID	NameofProd	Price
0	11	Laptop	50000
1	12	Mouse	500
2	13	Keyboard	1000
3	14	Monitor	12000
4	15	Printer	8000
5	16	Speaker	1500
6	17	Cable	200
7	18	Charger	700
8	19	Tablet	20000
9	20	Webcam	2500

In [2]:
```
appended_data = pd.concat([dataframe1, dataframe3], ignore_index=True)

display(appended_data)
```

	ProductID	NameofProd	Price
0	1	Pen	10
1	2	Book	50
2	3	Pencil	5
3	4	Bag	300
4	5	Scale	20
5	6	Eraser	5
6	7	Marker	25
7	8	File	40
8	9	Bottle	100
9	10	Box	60
10	11	Laptop	50000
11	12	Mouse	500
12	13	Keyboard	1000
13	14	Monitor	12000
14	15	Printer	8000
15	16	Speaker	1500
16	17	Cable	200
17	18	Charger	700
18	19	Tablet	20000
19	20	Webcam	2500

In [3]:
```
concatenated_data = pd.concat([dataframe1, dataframe3], axis=1)

display(concatenated_data)
```

	ProductID	NameofProd	Price	ProductID	NameofProd	Price
0	1	Pen	10	11	Laptop	50000
1	2	Book	50	12	Mouse	500
2	3	Pencil	5	13	Keyboard	1000
3	4	Bag	300	14	Monitor	12000
4	5	Scale	20	15	Printer	8000
5	6	Eraser	5	16	Speaker	1500
6	7	Marker	25	17	Cable	200
7	8	File	40	18	Charger	700
8	9	Bottle	100	19	Tablet	20000
9	10	Box	60	20	Webcam	2500

In [4]:
```
merged_data = pd.merge(dataframe1, dataframe2, on="ProductID")

display(merged_data)
```

	ProductID	NameofProd	Price	Company_Location
0	1	Pen	10	Chennai
1	2	Book	50	Vellore
2	3	Pencil	5	Delhi
3	4	Bag	300	Mumbai
4	5	Scale	20	Pune
5	6	Eraser	5	Salem
6	7	Marker	25	Bangalore
7	8	File	40	Hyderabad
8	9	Bottle	100	Coimbatore
9	10	Box	60	Trichy

In [5]:
```
inner_join = pd.merge(dataframe1, dataframe2, on="ProductID", how="inner")

display(inner_join)
```

	ProductID	NameofProd	Price	Company_Location
0	1	Pen	10	Chennai
1	2	Book	50	Vellore
2	3	Pencil	5	Delhi
3	4	Bag	300	Mumbai
4	5	Scale	20	Pune
5	6	Eraser	5	Salem
6	7	Marker	25	Bangalore
7	8	File	40	Hyderabad
8	9	Bottle	100	Coimbatore
9	10	Box	60	Trichy

In [6]:
```
left_join = pd.merge(dataframe1, dataframe2, on="ProductID", how="left")

display(left_join)
```

	ProductID	NameofProd	Price	Company_Location
0	1	Pen	10	Chennai
1	2	Book	50	Vellore
2	3	Pencil	5	Delhi
3	4	Bag	300	Mumbai
4	5	Scale	20	Pune
5	6	Eraser	5	Salem
6	7	Marker	25	Bangalore
7	8	File	40	Hyderabad
8	9	Bottle	100	Coimbatore
9	10	Box	60	Trichy

In [7]:
```
right_join = pd.merge(dataframe1, dataframe2, on="ProductID", how="right")

display(right_join)
```

	ProductID	NameofProd	Price	Company_Location
0	1	Pen	10	Chennai
1	2	Book	50	Vellore
2	3	Pencil	5	Delhi
3	4	Bag	300	Mumbai
4	5	Scale	20	Pune
5	6	Eraser	5	Salem
6	7	Marker	25	Bangalore
7	8	File	40	Hyderabad
8	9	Bottle	100	Coimbatore
9	10	Box	60	Trichy

In [8]:
```
full_join = pd.merge(dataframe1, dataframe2, on="ProductID", how="outer")

display(full_join)
```

	ProductID	NameofProd	Price	Company_Location
0	1	Pen	10	Chennai
1	2	Book	50	Vellore
2	3	Pencil	5	Delhi
3	4	Bag	300	Mumbai
4	5	Scale	20	Pune
5	6	Eraser	5	Salem
6	7	Marker	25	Bangalore
7	8	File	40	Hyderabad
8	9	Bottle	100	Coimbatore
9	10	Box	60	Trichy

Part B: Data Transformation Techniques

Normalize a numeric vector using different ranges, Z-score transformation, and Decimal Scaling Normalization.

In [9]:
```
Dharsha = np.array([274, 12, 25, 38, 45, 52, 61, 73, 84, 95])
```

```
print("Original vector:")
print(Dharsha)
```

```
Original vector:
[274  12  25  38  45  52  61  73  84  95]
```

In [10]:
```
normalized_3 = -3 + (Dharsha - Dharsha.min()) * 6 / (Dharsha.max() - Dharsha.min())

print(normalized_3)
```

```
[ 3.         -3.         -2.70229008 -2.40458015 -2.24427481 -2.08396947
 -1.8778626  -1.60305344 -1.35114504 -1.09923664]
```

In [11]:
```
normalized_1 = -1 + 2 * (Dharsha - Dharsha.min()) / (Dharsha.max() - Dharsha.min())

print(normalized_1)
```

```
[ 1.         -1.         -0.90076336 -0.80152672 -0.7480916  -0.69465649
 -0.6259542  -0.53435115 -0.45038168 -0.36641221]
```

In [12]:
```
minmax = (Dharsha - Dharsha.min()) / (Dharsha.max() - Dharsha.min())

print(minmax)
```

```
[1.         0.         0.04961832 0.09923664 0.1259542  0.15267176
 0.1870229  0.23282443 0.27480916 0.31679389]
```

In [13]:
```
z_score = (Dharsha - Dharsha.mean()) / Dharsha.std()

print(z_score)
```

```
[ 2.81450827 -0.90786006 -0.7231624  -0.53846473 -0.43901214 -0.33955955
 -0.21169194 -0.04120179  0.11508085  0.27136349]
```

In [14]:
```
max_value = np.max(np.abs(Dharsha))
j = len(str(int(max_value)))

decimal_scaled = Dharsha / (10 ** j)

print(decimal_scaled)
```

```
[0.274 0.012 0.025 0.038 0.045 0.052 0.061 0.073 0.084 0.095]
```

Assessment 2: Data Import, Transformation and Basic Statistical Analysis

Dataset: Telco Customer Churn

In [15]:
```
import pandas as pd
import numpy as np

url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/telco%20custome

data = pd.read_csv(url)
data.columns = data.columns.str.strip()
```

```
data["TotalCharges"] = pd.to_numeric(data["TotalCharges"], errors="coerce")

print("Dataset loaded successfully")
```

```
Dataset loaded successfully
```

In [16]:

```
# Q1: Import dataset
display(data.head())

# Q2: First 10 records
display(data.head(10))

# Q3: Last 10 records
display(data.tail(10))

# Q4: Number of rows and columns
print("Rows and columns:", data.shape)

# Q5: Column names
print(data.columns.tolist())

# Q6: Data types
print(data.dtypes)

# Q7: Summary statistics
display(data.describe(include="all").T)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	Female	0.0	Yes	No	1	No	No pl se
1	5575-GNVDE	Male	0.0	No	No	34	Yes	
2	3668-QPYBK	Male	0.0	No	No	2	Yes	
3	7795-CFOCW	Male	0.0	No	No	45	No	No pl se
4	9237-HQITU	Female	0.0	No	No	2	Yes	

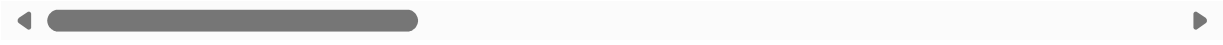
5 rows × 21 columns

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	Female	0.0	Yes	No	1	No	No pl se
1	5575-GNVDE	Male	0.0	No	No	34	Yes	
2	3668-QPYBK	Male	0.0	No	No	2	Yes	
3	7795-CFOCW	Male	0.0	No	No	45	No	No pl se
4	9237-HQITU	Female	0.0	No	No	2	Yes	
5	9305-CDSKC	Female	0.0	No	No	8	Yes	
6	1452-KIOVK	Male	0.0	No	Yes	22	Yes	
7	6713-OKOMC	Female	0.0	No	No	10	No	No pl se
8	7892-POOKP	Female	0.0	Yes	No	28	Yes	
9	6388-TABGU	Male	0.0	No	Yes	62	Yes	

10 rows × 21 columns

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multi
7033	9767-FFLEM	Male	0.0	No	No	38	Yes	
7034	0639-TSIQW	Female	0.0	No	No	67	Yes	
7035	8456-QDAVC	Male	0.0	No	No	19	Yes	
7036	7750-EYXWZ	Female	0.0	No	No	12	No	N
7037	2569-WGERO	Female	0.0	No	No	72	Yes	
7038	6840-RESVB	Male	0.0	Yes	Yes	24	Yes	
7039	2234-XADUH	Female	0.0	Yes	Yes	72	Yes	
7040	4801-JZAZL	Female	0.0	Yes	Yes	11	No	N
7041	8361-LTMKD	Male	1.0	Yes	No	4	Yes	
7042	3186-AJIEK	Male	0.0	No	No	66	Yes	

10 rows × 21 columns



```
Rows and columns: (7043, 21)
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneS
ervice', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'Devi
ceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract', 'Paperle
ssBilling', 'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn']
customerID          object
gender              object
SeniorCitizen      float64
Partner             object
Dependents          object
tenure              object
PhoneService        object
MultipleLines       object
InternetService     object
OnlineSecurity      object
OnlineBackup        object
DeviceProtection    object
TechSupport         object
StreamingTV         object
StreamingMovies     object
Contract            object
PaperlessBilling    object
PaymentMethod       object
MonthlyCharges     float64
TotalCharges       float64
Churn               object
dtype: object
```

	count	unique	top	freq	mean	std	min	25%
customerID	7043	7043	7590-VHVEG	1	NaN	NaN	NaN	NaN
gender	7043	2	Male	3555	NaN	NaN	NaN	NaN
SeniorCitizen	7035.0	NaN	NaN	NaN	0.162331	0.368781	0.0	0.0
Partner	7038	2	No	3640	NaN	NaN	NaN	NaN
Dependents	7043	3	No	4931	NaN	NaN	NaN	NaN
tenure	7042	74	1	613	NaN	NaN	NaN	NaN
PhoneService	7043	2	Yes	6361	NaN	NaN	NaN	NaN
MultipleLines	7043	3	No	3390	NaN	NaN	NaN	NaN
InternetService	7043	3	Fiber optic	3096	NaN	NaN	NaN	NaN
OnlineSecurity	7043	3	No	3498	NaN	NaN	NaN	NaN
OnlineBackup	7043	3	No	3088	NaN	NaN	NaN	NaN
DeviceProtection	7043	3	No	3095	NaN	NaN	NaN	NaN
TechSupport	7043	4	No	3472	NaN	NaN	NaN	NaN
StreamingTV	7043	3	No	2810	NaN	NaN	NaN	NaN
StreamingMovies	7043	3	No	2785	NaN	NaN	NaN	NaN
Contract	7043	3	Month-to-month	3875	NaN	NaN	NaN	NaN
PaperlessBilling	7043	3	Yes	4170	NaN	NaN	NaN	NaN
PaymentMethod	7043	4	Electronic check	2365	NaN	NaN	NaN	NaN
MonthlyCharges	7040.0	NaN	NaN	NaN	64.761889	30.087384	18.25	35.5
TotalCharges	7032.0	NaN	NaN	NaN	2283.300441	2266.771362	18.8	401.45
Churn	7042	3	No	5172	NaN	NaN	NaN	NaN

In [17]:
```
# Q8 and Q10: Missing values in each column
print(data.isnull().sum())

# Q9: Total missing values
print("Total missing values:", data.isnull().sum().sum())

# Q11: Rows containing missing values
display(data[data.isnull().any(axis=1)].head(10))

# Q12: Remove rows containing missing values
```

```python
clean_data = data.dropna().copy()

print("Shape before cleaning:", data.shape)
print("Shape after cleaning:", clean_data.shape)
```

```
customerID           0
gender               0
SeniorCitizen        8
Partner              5
Dependents           0
tenure               1
PhoneService         0
MultipleLines        0
InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       3
TotalCharges        11
Churn                1
dtype: int64
Total missing values: 29
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multip
16	8191-XWSZG	Female	NaN	No	No	52	Yes	
369	3520-FJGCV	Male	NaN	Yes	Yes	72	Yes	
370	6563-VRERX	Male	NaN	Yes	Yes	33	Yes	
371	0259-GBZSH	Male	NaN	No	No	2	Yes	
372	6122-EFVKN	Male	NaN	No	Yes	24	No	No
373	2805-EDJPQ	Female	NaN	Yes	Yes	56	Yes	
374	6862-CQUMB	Male	NaN	No	No	37	Yes	
375	7156-MXBJE	Female	NaN	No	No	43	Yes	
472	1304-BCCFO	Male	0.0	NaN	No	9	Yes	
473	4104-PVRPS	Male	0.0	NaN	No	24	Yes	

10 rows × 21 columns

```
Shape before cleaning: (7043, 21)
Shape after cleaning: (7014, 21)
```

In [18]:
```python
# Q13: Replace missing numerical values with mean
mean_data = data.copy()
mean_data["MonthlyCharges"] = mean_data["MonthlyCharges"].fillna(
    mean_data["MonthlyCharges"].mean()
)

# Q14: Replace missing numerical values with median
median_data = data.copy()
median_data["TotalCharges"] = median_data["TotalCharges"].fillna(
    median_data["TotalCharges"].median()
)

# Q15: Replace missing categorical values with mode
mode_data = data.copy()
mode_value = mode_data["PaymentMethod"].mode()[0]
mode_data["PaymentMethod"] = mode_data["PaymentMethod"].fillna(mode_value)
```

```
# Q16: Forward filling
forward_data = data.ffill()

# Q17: Backward filling
backward_data = data.bfill()

# Q18: Replace remaining missing values with constants
constant_data = data.copy()

for col in constant_data.columns:
    if constant_data[col].dtype == "object":
        constant_data[col] = constant_data[col].fillna("Unknown")
    else:
        constant_data[col] = constant_data[col].fillna(0)

print("Missing-value handling completed")
```

```
Missing-value handling completed
```

In [19]:

```
# Q19: Remove duplicate records
clean_data = data.drop_duplicates().copy()
print("Duplicate records remaining:", clean_data.duplicated().sum())

# Q20: Convert appropriate variables into categorical types
for col in ["gender", "Partner", "Dependents", "PhoneService",
            "InternetService", "Contract", "PaymentMethod", "Churn"]:
    if col in clean_data.columns:
        clean_data[col] = clean_data[col].astype("category")

print(clean_data.dtypes)

# Q21: Rename a column
clean_data = clean_data.rename(
    columns={"MonthlyCharges": "Monthly_Charges"}
)

# Q22: Create a derived column
clean_data["TotalServices"] = (
    clean_data[["PhoneService", "OnlineSecurity", "OnlineBackup",
                "DeviceProtection", "TechSupport",
                "StreamingTV", "StreamingMovies"]]
    .apply(lambda col: col.eq("Yes").sum(), axis=1)
)

display(clean_data.head())
```

```
Duplicate records remaining: 0
customerID           object
gender             category
SeniorCitizen       float64
Partner            category
Dependents         category
tenure               object
PhoneService       category
MultipleLines        object
InternetService    category
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract           category
PaperlessBilling     object
PaymentMethod      category
MonthlyCharges      float64
TotalCharges        float64
Churn              category
dtype: object
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	Female	0.0	Yes	No	1	No	No pl se
1	5575-GNVDE	Male	0.0	No	No	34	Yes	
2	3668-QPYBK	Male	0.0	No	No	2	Yes	
3	7795-CFOCW	Male	0.0	No	No	45	No	No pl se
4	9237-HQITU	Female	0.0	No	No	2	Yes	

5 rows × 22 columns

In [21]:
```
# Q23: Summary statistics
display(data.describe().T)

# Convert numerical columns to numeric format
data["tenure"] = pd.to_numeric(data["tenure"], errors="coerce")
data["MonthlyCharges"] = pd.to_numeric(data["MonthlyCharges"], errors="coerce")
data["TotalCharges"] = pd.to_numeric(data["TotalCharges"], errors="coerce")

# Q24: Minimum and maximum
```

```
print("Minimum tenure:", data["tenure"].min())
print("Maximum tenure:", data["tenure"].max())

# Q25: Range
print("Tenure range:", data["tenure"].max() - data["tenure"].min())

# Q26: Variance
print("MonthlyCharges variance:", data["MonthlyCharges"].var())

# Q27: Standard deviation
print("MonthlyCharges standard deviation:", data["MonthlyCharges"].std())

# Q28: Interquartile range
iqr = data["MonthlyCharges"].quantile(0.75) - data["MonthlyCharges"].quantile(0.25)
print("MonthlyCharges IQR:", iqr)
```

	count	mean	std	min	25%	50%	75%	ma
SeniorCitizen	7035.0	0.162331	0.368781	0.00	0.00	0.000	0.0000	1.0
MonthlyCharges	7040.0	64.761889	30.087384	18.25	35.50	70.350	89.8500	118.7
TotalCharges	7032.0	2283.300441	2266.771362	18.80	401.45	1397.475	3794.7375	8684.8

```
Minimum tenure: 0.0
Maximum tenure: 72.0
Tenure range: 72.0
MonthlyCharges variance: 905.2506826074163
MonthlyCharges standard deviation: 30.087384110411065
MonthlyCharges IQR: 54.349999999999994
```

Assessment 3: Data Cleaning and Univariate Analysis

This assessment focuses on identifying and handling missing values, removing duplicates, converting data types, detecting outliers, and performing univariate analysis using statistical summaries and visualizations.

In [22]:
```
data = pd.read_csv(url)
data.columns = data.columns.str.strip()

data["TotalCharges"] = pd.to_numeric(data["TotalCharges"], errors="coerce")

print("First 6 records")
display(data.head(6))

print("Last 6 records")
display(data.tail(6))

print("Total missing values:", data.isnull().sum().sum())
```

```
print("Records containing missing values")
display(data[data.isnull().any(axis=1)].head(10))
```

First 6 records

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	Female	0.0	Yes	No	1	No	No pl se
1	5575-GNVDE	Male	0.0	No	No	34	Yes	
2	3668-QPYBK	Male	0.0	No	No	2	Yes	
3	7795-CFOCW	Male	0.0	No	No	45	No	No pl se
4	9237-HQITU	Female	0.0	No	No	2	Yes	
5	9305-CDSKC	Female	0.0	No	No	8	Yes	

6 rows × 21 columns

Last 6 records

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multi
7037	2569-WGERO	Female	0.0	No	No	72	Yes	
7038	6840-RESVB	Male	0.0	Yes	Yes	24	Yes	
7039	2234-XADUH	Female	0.0	Yes	Yes	72	Yes	
7040	4801-JZAZL	Female	0.0	Yes	Yes	11	No	N
7041	8361-LTMKD	Male	1.0	Yes	No	4	Yes	
7042	3186-AJIEK	Male	0.0	No	No	66	Yes	

6 rows × 21 columns

```
Total missing values: 29
Records containing missing values
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Multip
16	8191-XWSZG	Female	NaN	No	No	52	Yes	
369	3520-FJGCV	Male	NaN	Yes	Yes	72	Yes	
370	6563-VRERX	Male	NaN	Yes	Yes	33	Yes	
371	0259-GBZSH	Male	NaN	No	No	2	Yes	
372	6122-EFVKN	Male	NaN	No	Yes	24	No	No
373	2805-EDJPQ	Female	NaN	Yes	Yes	56	Yes	
374	6862-CQUMB	Male	NaN	No	No	37	Yes	
375	7156-MXBJE	Female	NaN	No	No	43	Yes	
472	1304-BCCFO	Male	0.0	NaN	No	9	Yes	
473	4104-PVRPS	Male	0.0	NaN	No	24	Yes	

10 rows × 21 columns

In [23]:
```
missing_percent = data.isnull().mean() * 100

data = data.loc[:, missing_percent <= 70].copy()

for col in data.select_dtypes(include="number").columns:
    data[col] = data[col].fillna(data[col].mean())

for col in data.select_dtypes(include="object").columns:
    if data[col].isnull().any():
        mode_value = data[col].mode()
        if not mode_value.empty:
            data[col] = data[col].fillna(mode_value[0])

print("Missing values after cleaning:")
print(data.isnull().sum().sum())
```

```
Missing values after cleaning:
0
```

In [24]:

```python
print("Duplicate rows:", data.duplicated().sum())

data = data.drop_duplicates().copy()

print("Data types:")
print(data.dtypes)

for col in data.select_dtypes(include="object").columns:
    if col != "customerID":
        data[col] = data[col].astype("category")

print("Updated data types:")
print(data.dtypes)

if "gender" in data.columns:
    data["gender"] = data["gender"].astype(str).str.lower()

display(data.head())
```

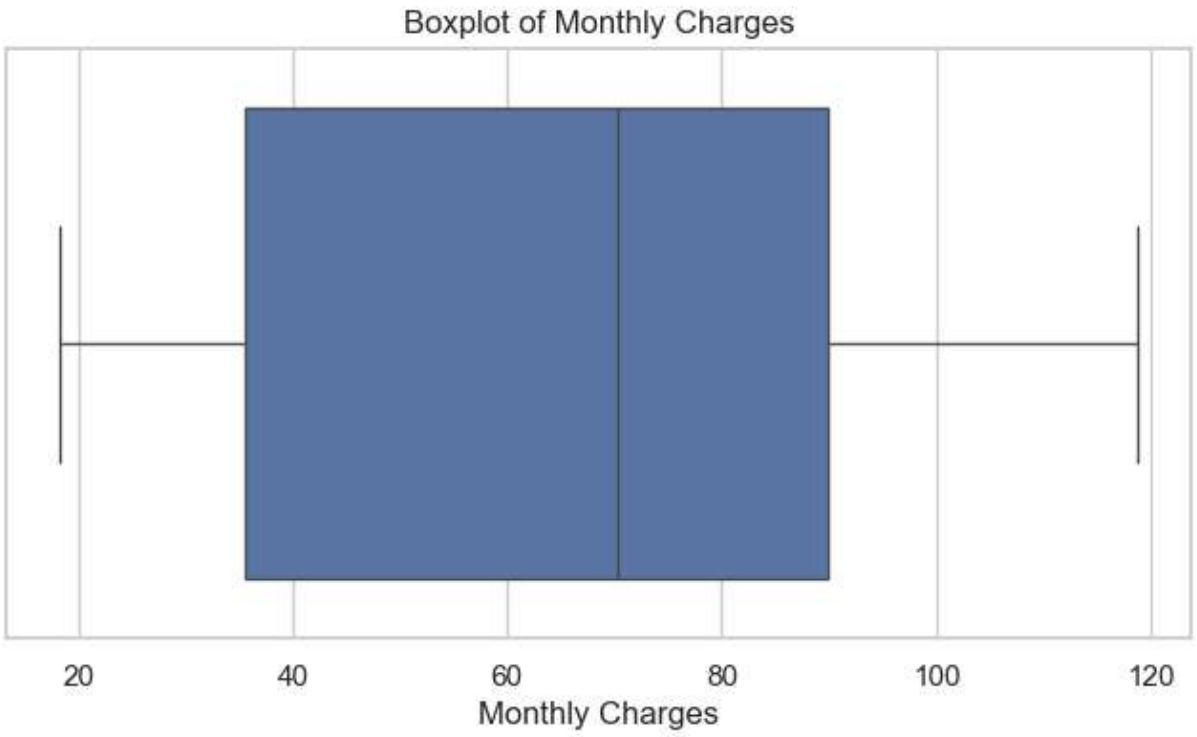
```
Duplicate rows: 0
Data types:
customerID          object
gender              object
SeniorCitizen      float64
Partner             object
Dependents          object
tenure              object
PhoneService        object
MultipleLines       object
InternetService     object
OnlineSecurity      object
OnlineBackup        object
DeviceProtection    object
TechSupport         object
StreamingTV         object
StreamingMovies     object
Contract            object
PaperlessBilling    object
PaymentMethod       object
MonthlyCharges     float64
TotalCharges       float64
Churn               object
dtype: object
Updated data types:
customerID           object
gender             category
SeniorCitizen       float64
Partner            category
Dependents         category
tenure             category
PhoneService       category
MultipleLines      category
InternetService    category
OnlineSecurity     category
OnlineBackup       category
DeviceProtection   category
TechSupport        category
StreamingTV        category
StreamingMovies    category
Contract           category
PaperlessBilling   category
PaymentMethod      category
MonthlyCharges      float64
TotalCharges        float64
Churn              category
dtype: object
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	female	0.0	Yes	No	1	No	No pl se
1	5575-GNVDE	male	0.0	No	No	34	Yes	
2	3668-QPYBK	male	0.0	No	No	2	Yes	
3	7795-CFOCW	male	0.0	No	No	45	No	No pl se
4	9237-HQITU	female	0.0	No	No	2	Yes	

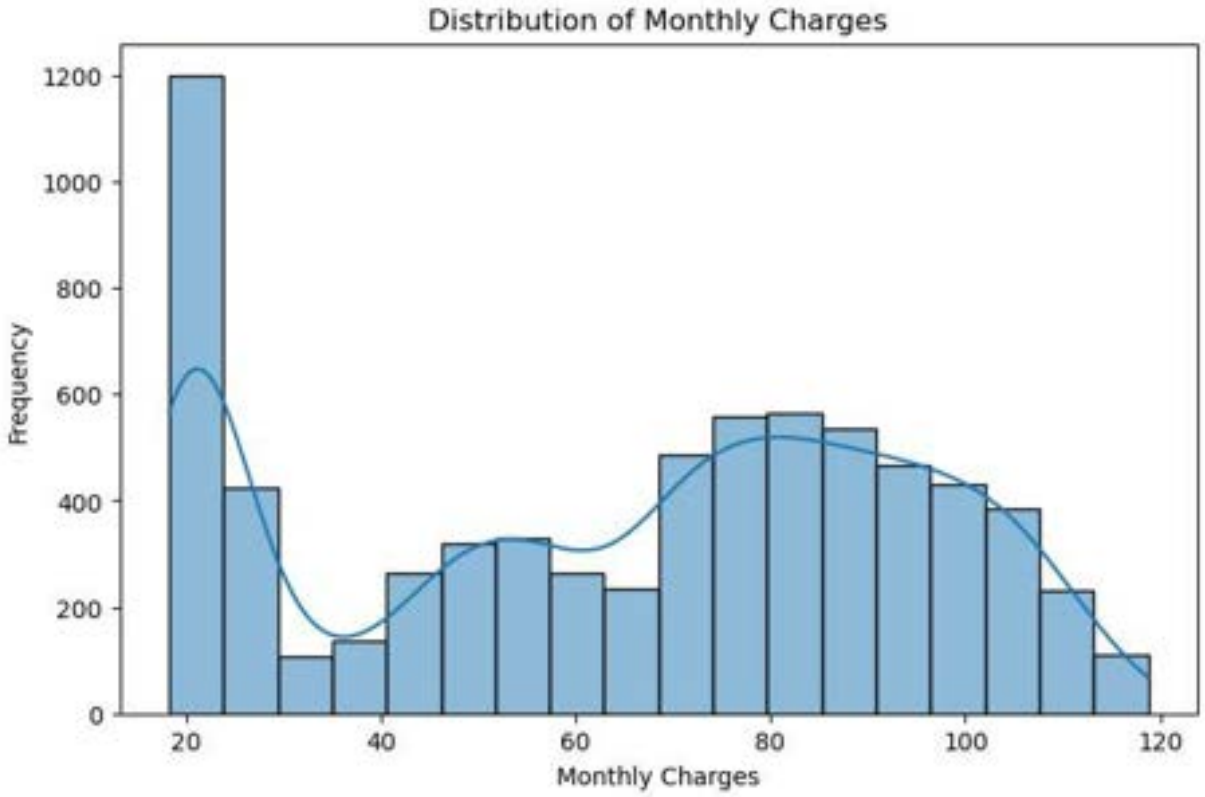
5 rows × 21 columns

In [26]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [116…
```
plt.figure(figsize=(8, 4))
sns.boxplot(x=data["MonthlyCharges"])
plt.title("Boxplot of Monthly Charges")
plt.xlabel("Monthly Charges")
plt.show()
```



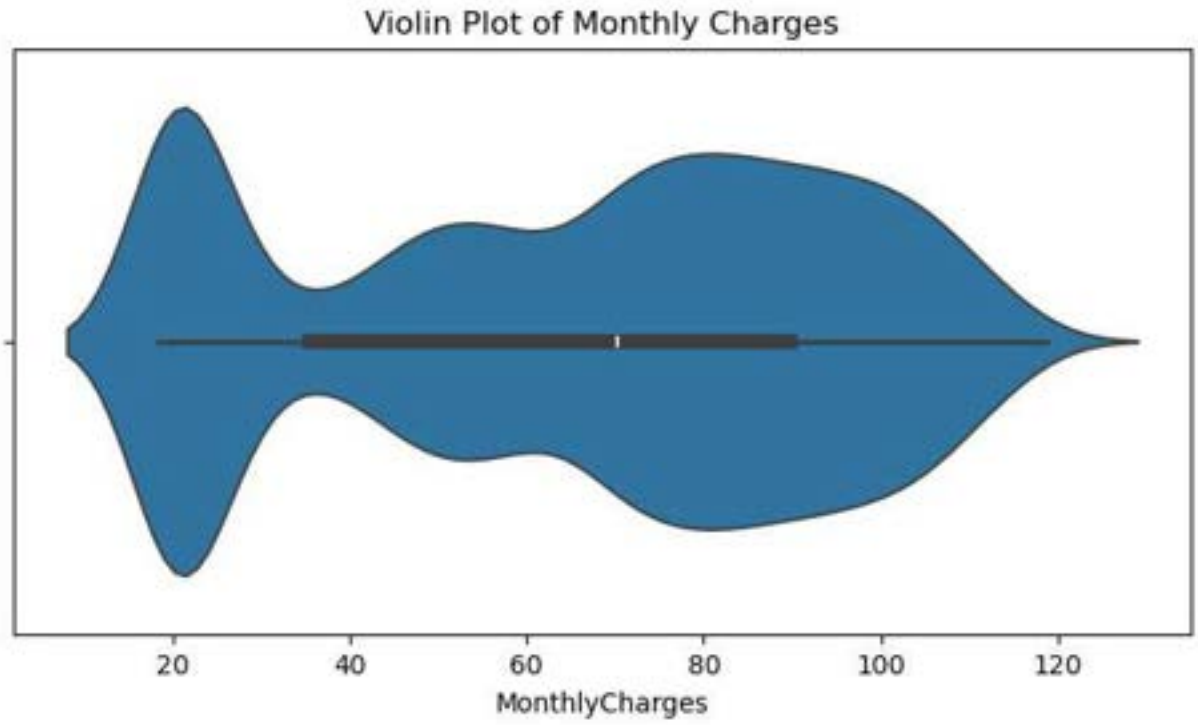
In [29]:
```
plt.figure(figsize=(8, 5))
sns.histplot(data=data, x="MonthlyCharges", kde=True)
plt.title("Distribution of Monthly Charges")
plt.xlabel("Monthly Charges")
plt.ylabel("Frequency")
plt.show()
```

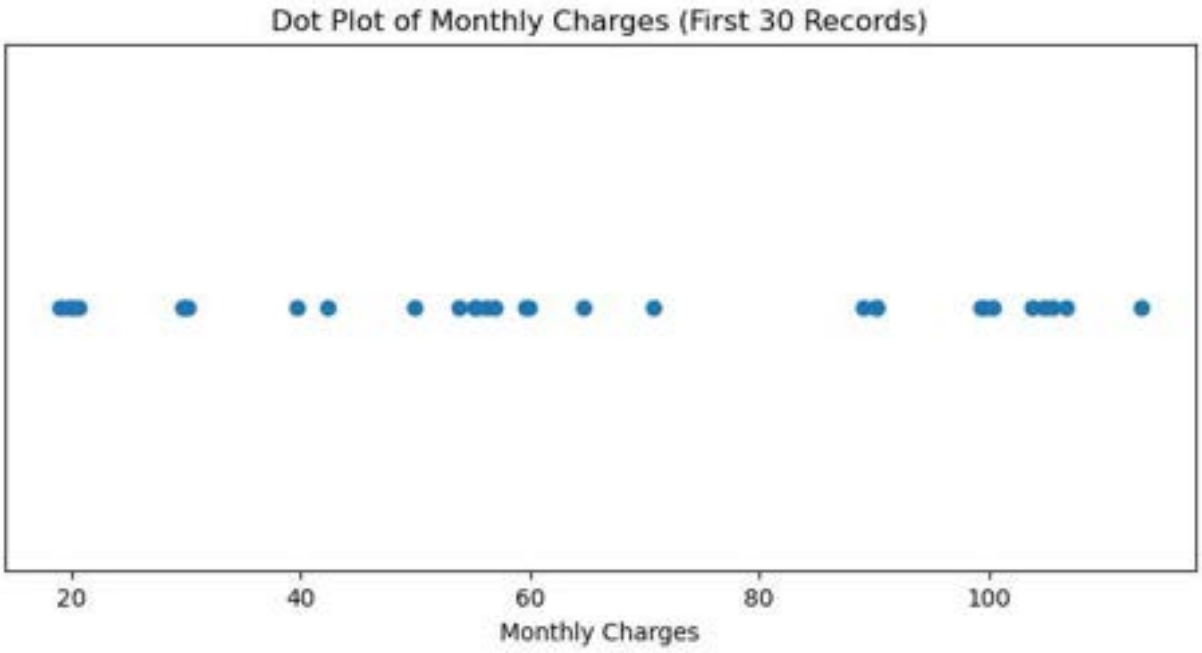


In [30]:
```
plt.figure(figsize=(8, 4))
sns.violinplot(x=data["MonthlyCharges"])
plt.title("Violin Plot of Monthly Charges")
plt.show()

sample_values = data["MonthlyCharges"].dropna().head(30).reset_index(drop=True)

plt.figure(figsize=(9, 4))
plt.plot(sample_values, np.zeros(len(sample_values)), "o")
plt.title("Dot Plot of Monthly Charges (First 30 Records)")
plt.xlabel("Monthly Charges")
plt.yticks([])
plt.show()
```





In [31]:
```
frequency = data["Contract"].value_counts()
proportion = data["Contract"].value_counts(normalize=True)
percentage = proportion * 100

frequency_table = pd.DataFrame({
    "Frequency": frequency,
    "Proportion": proportion,
    "Percentage": percentage
})

display(frequency_table)

frequency.plot(kind="bar", figsize=(7, 4))
```

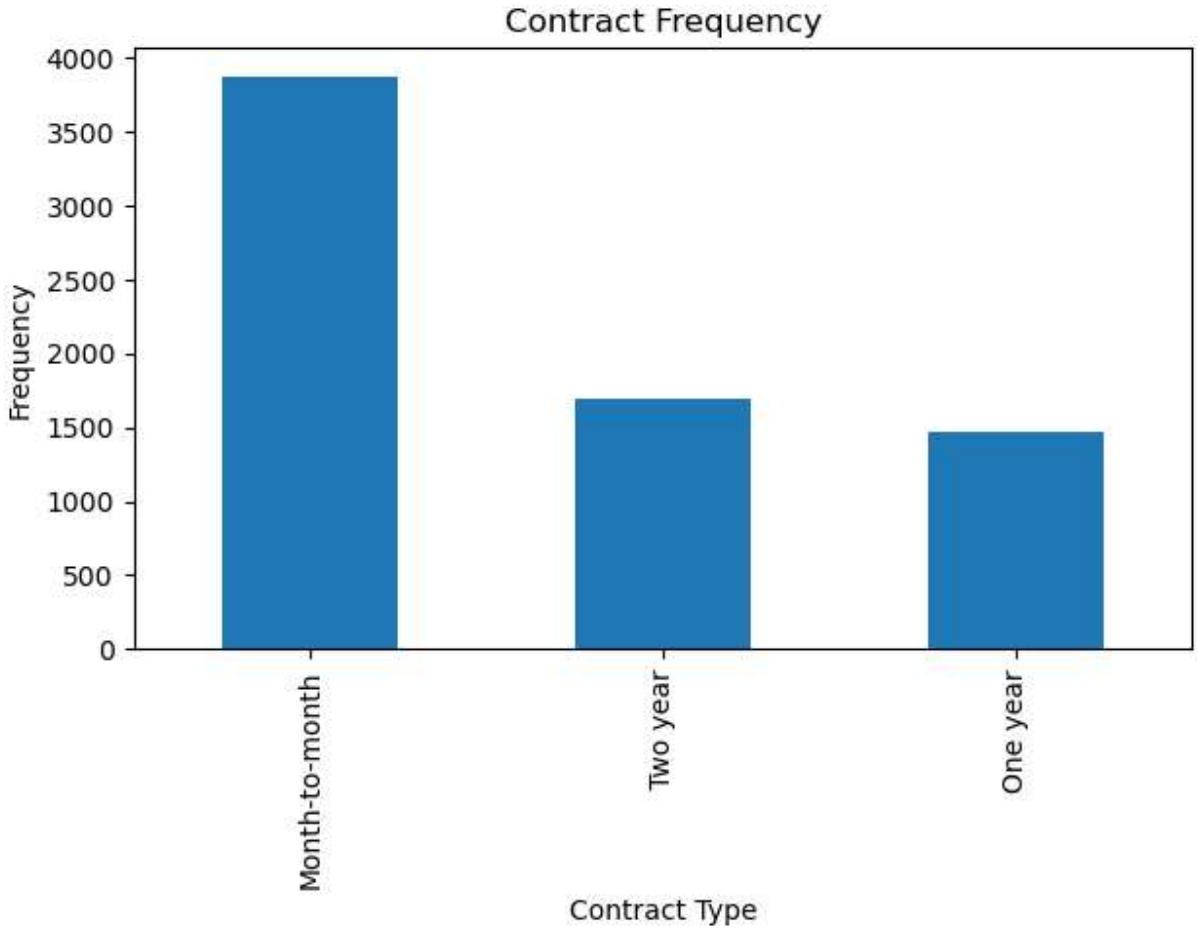
```
plt.title("Contract Frequency")
plt.xlabel("Contract Type")
plt.ylabel("Frequency")
plt.show()

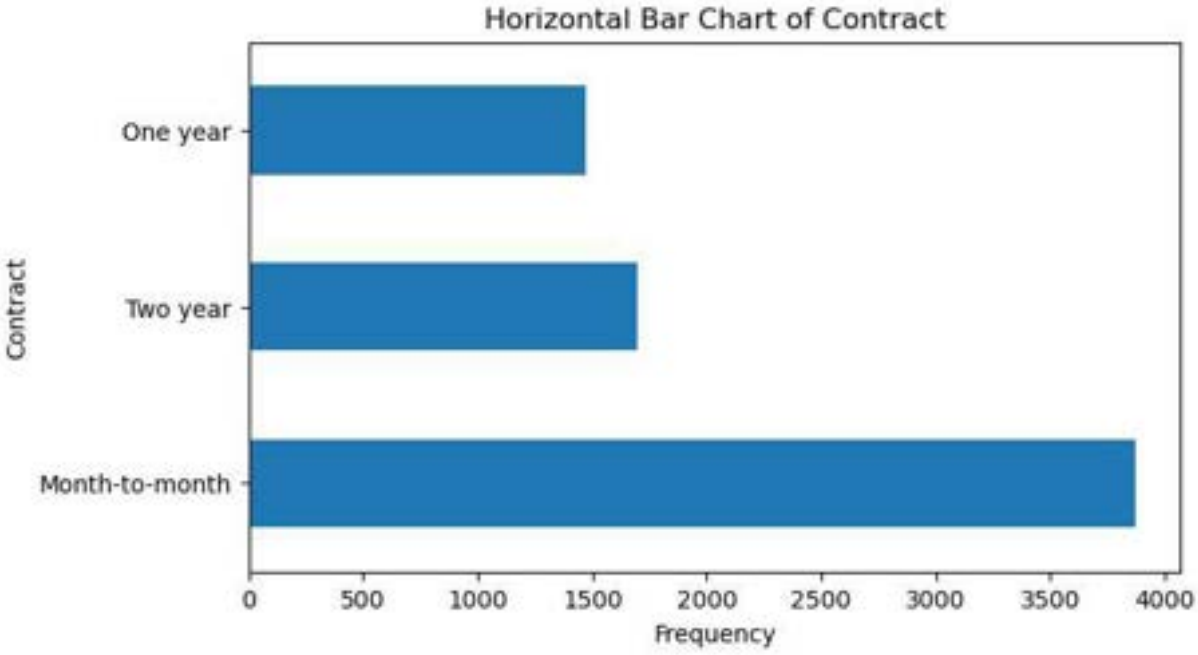
frequency.plot(kind="barh", figsize=(7, 4))
plt.title("Horizontal Bar Chart of Contract")
plt.xlabel("Frequency")
plt.show()

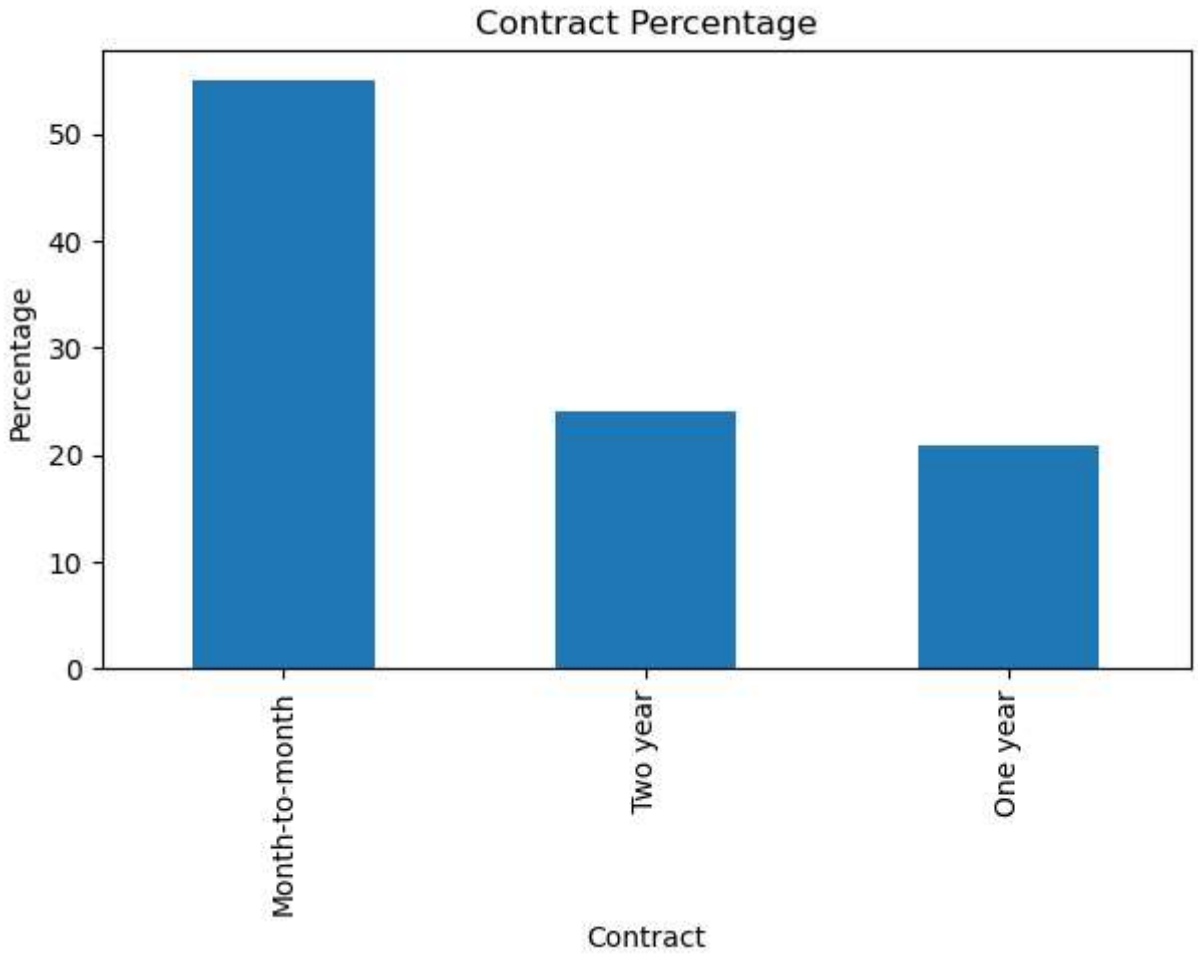
percentage.plot(kind="bar", figsize=(7, 4))
plt.title("Contract Percentage")
plt.ylabel("Percentage")
plt.show()

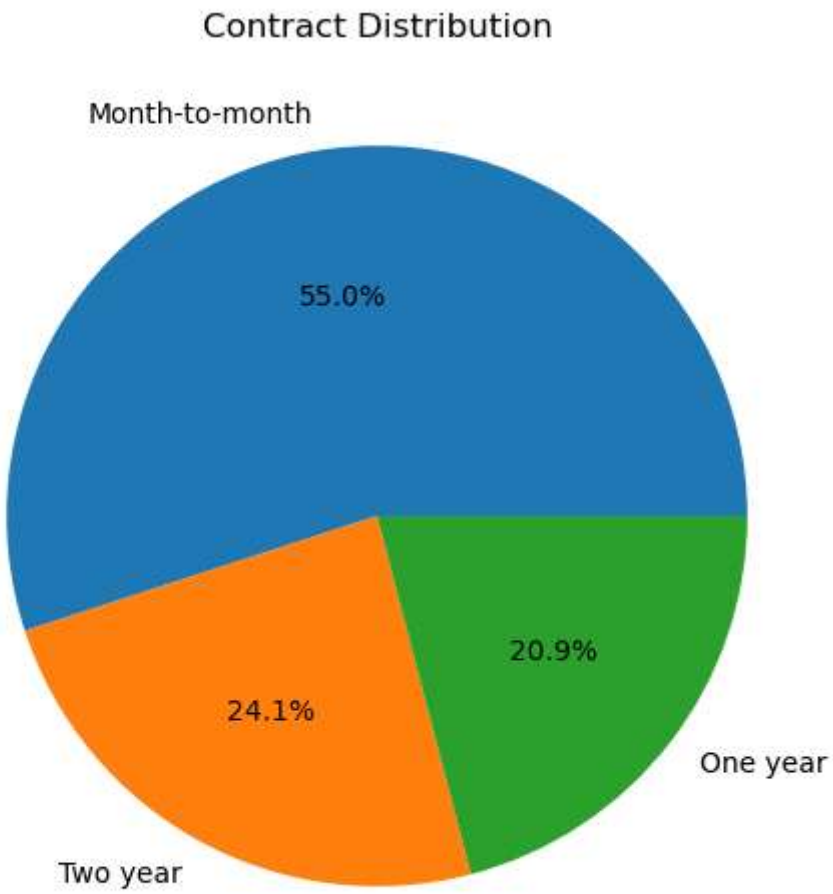
frequency.plot(kind="pie", autopct="%1.1f%%", figsize=(6, 6))
plt.title("Contract Distribution")
plt.ylabel("")
plt.show()
```

Contract	Frequency	Proportion	Percentage
Month-to-month	3875	0.550192	55.019168
Two year	1695	0.240664	24.066449
One year	1473	0.209144	20.914383


Contract Frequency
Frequency
4000
3500
3000
2500
2000
1500
1000
500
0
Month-to-month
Two year
One year
Contract Type


Horizontal Bar Chart of Contract
Contract
One year
Two year
Month-to-month
0
500
1000
1500
2000
2500
3000
3500
4000
Frequency


Contract Percentage
Percentage
0
10
20
30
40
50
Month-to-month
Two year
One year
Contract



In [32]:
```
contract_counts = data["Contract"].value_counts()

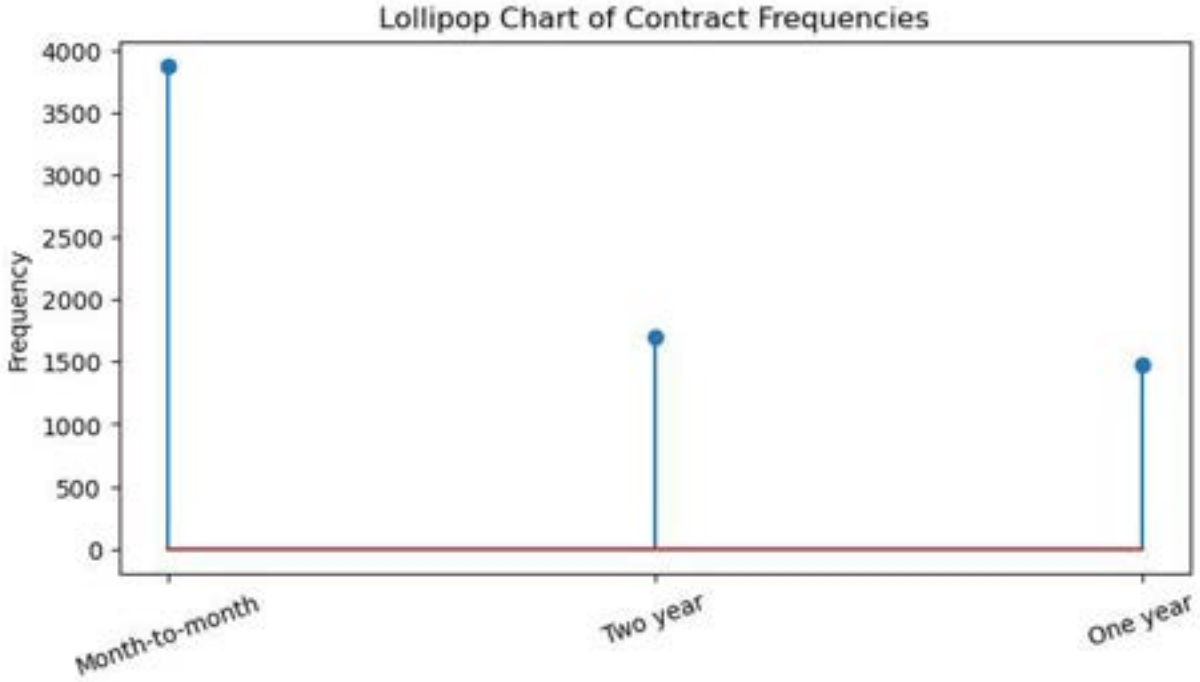
plt.figure(figsize=(8, 4))
plt.stem(range(len(contract_counts)), contract_counts.values)
plt.xticks(range(len(contract_counts)), contract_counts.index, rotation=20)
plt.title("Lollipop Chart of Contract Frequencies")
plt.ylabel("Frequency")
plt.show()

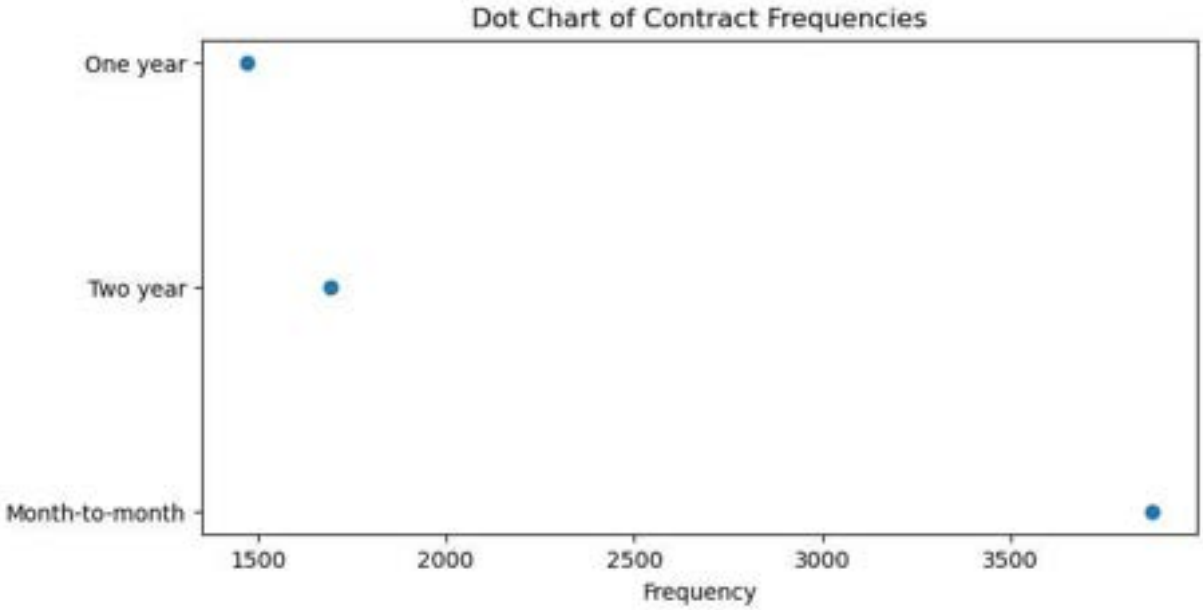
plt.figure(figsize=(8, 4))
plt.plot(contract_counts.values, contract_counts.index, "o")
plt.title("Dot Chart of Contract Frequencies")
plt.xlabel("Frequency")
plt.show()

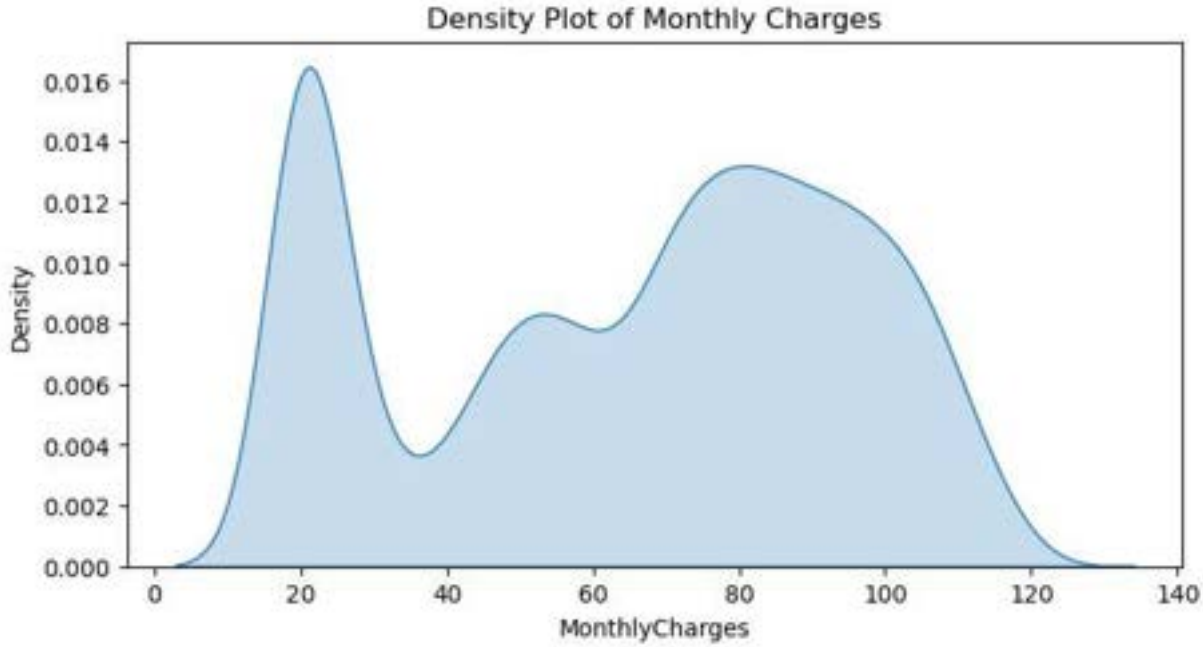
plt.figure(figsize=(8, 4))
sns.kdeplot(data=data, x="MonthlyCharges", fill=True)
plt.title("Density Plot of Monthly Charges")
plt.show()

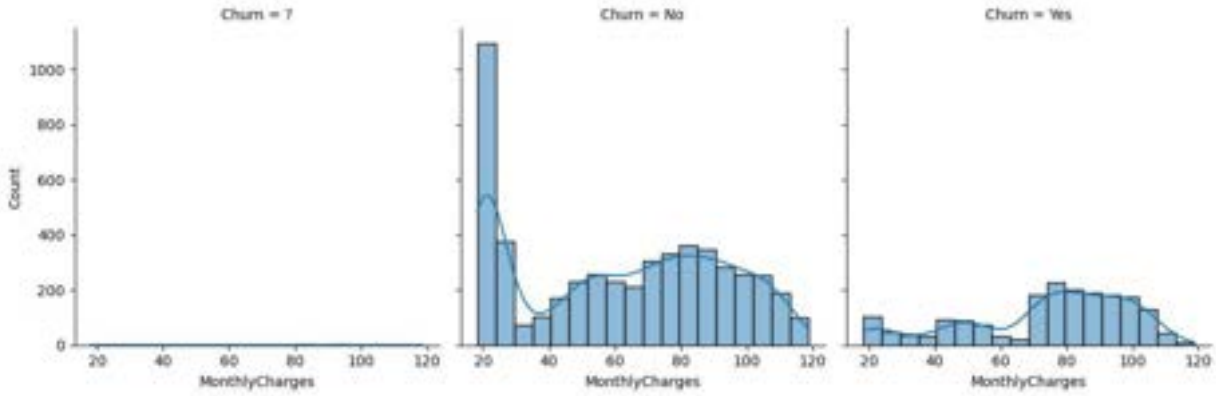
sns.displot(
    data=data,
    x="MonthlyCharges",
    col="Churn",
    kde=True,
```

```
    height=4,
    aspect=1
)
plt.show()
```









Assessment 4: Bivariate Analysis

Name: Dharsha R

Register Number: 23BDS0274

Objective

To perform bivariate analysis using the Telco Customer Churn dataset through contingency tables, bar charts, scatter plots, line plots, heatmaps, and density plots.

In [33]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Contingency Table: Contract vs Churn")
```

```
contingency_table = pd.crosstab(data["Contract"], data["Churn"])

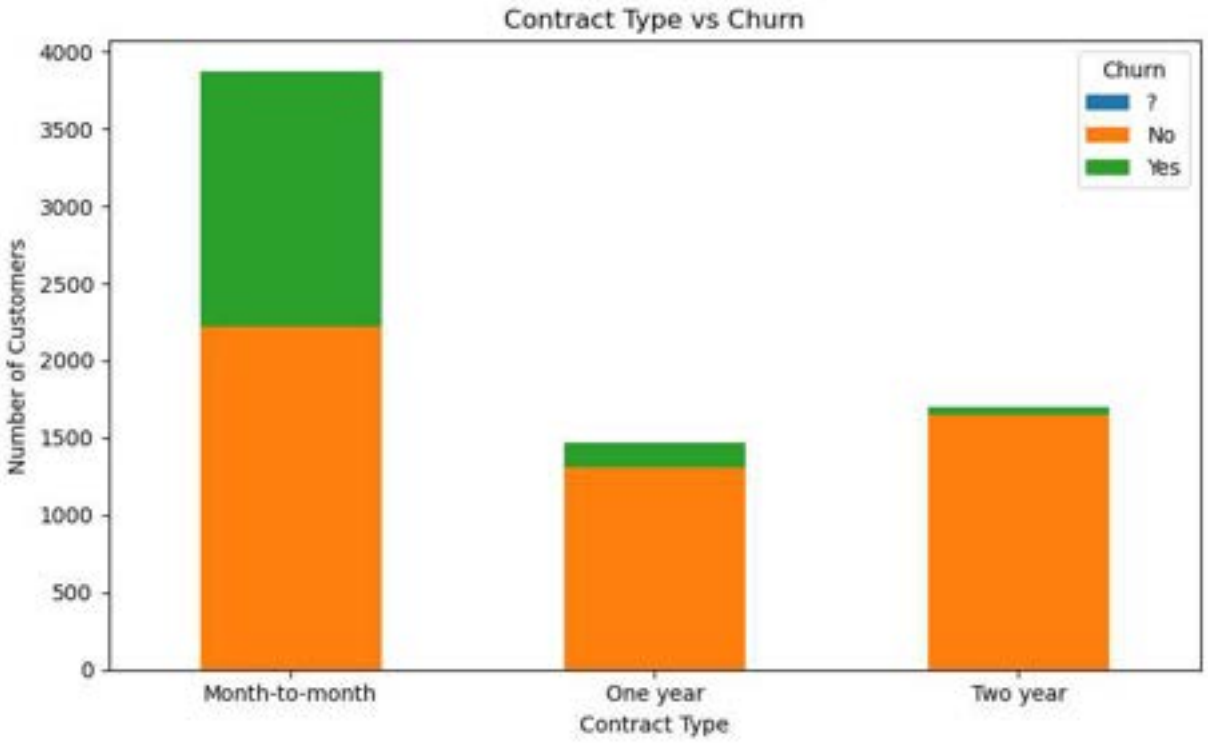
display(contingency_table)
```

Contingency Table: Contract vs Churn

Contract	Churn ?	No	Yes
Month-to-month	0	2221	1654
One year	1	1306	166
Two year	1	1646	48

In [34]:
```
contingency_table.plot(
    kind="bar",
    stacked=True,
    figsize=(8, 5)
)

plt.title("Contract Type vs Churn")
plt.xlabel("Contract Type")
plt.ylabel("Number of Customers")
plt.legend(title="Churn")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

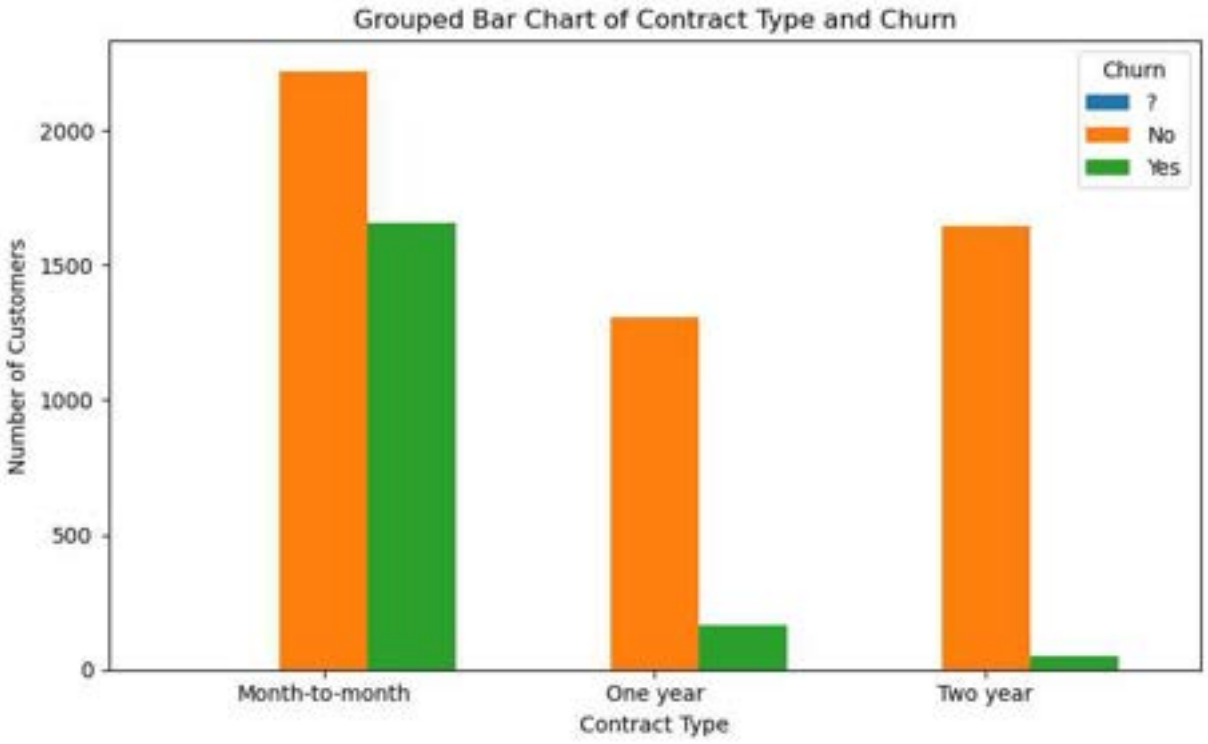


In [35]:
```
grouped_table = pd.crosstab(data["Contract"], data["Churn"])

grouped_table.plot(
    kind="bar",
```

```
    stacked=False,
    figsize=(8, 5),
    width=0.8
)

plt.title("Grouped Bar Chart of Contract Type and Churn")
plt.xlabel("Contract Type")
plt.ylabel("Number of Customers")
plt.legend(title="Churn")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

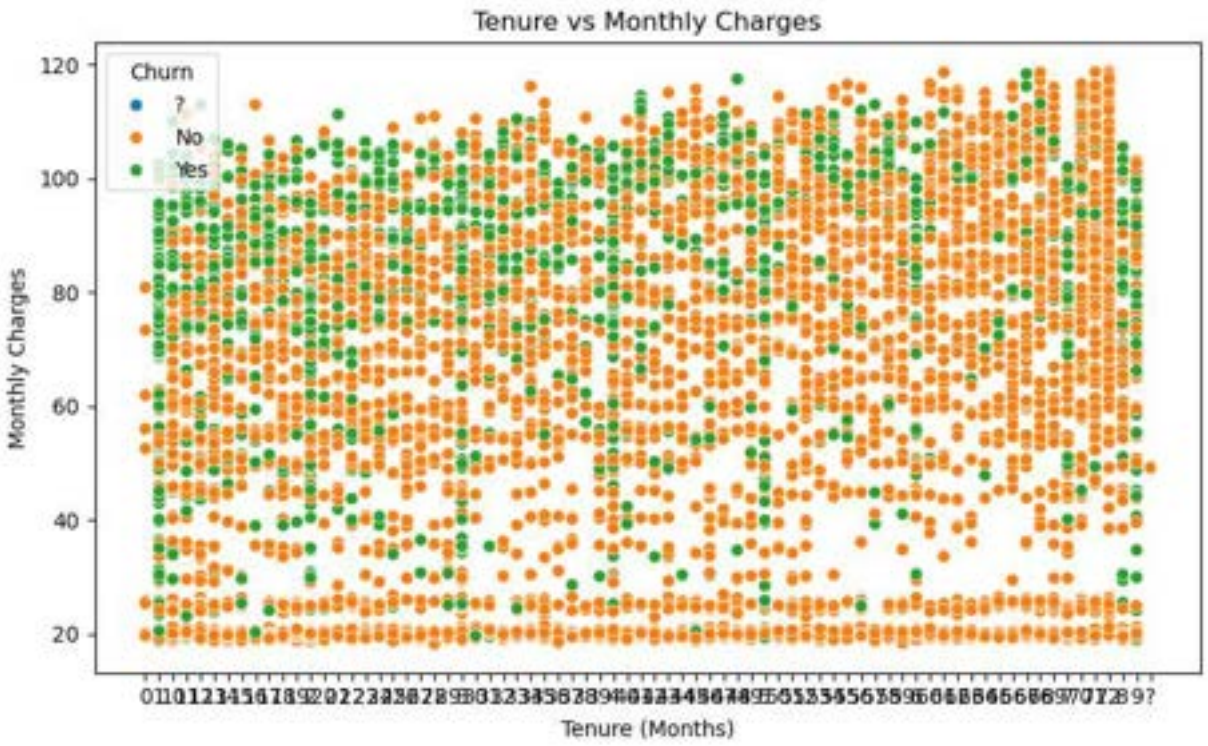


In [36]:

```
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=data,
    x="tenure",
    y="MonthlyCharges",
    hue="Churn"
)

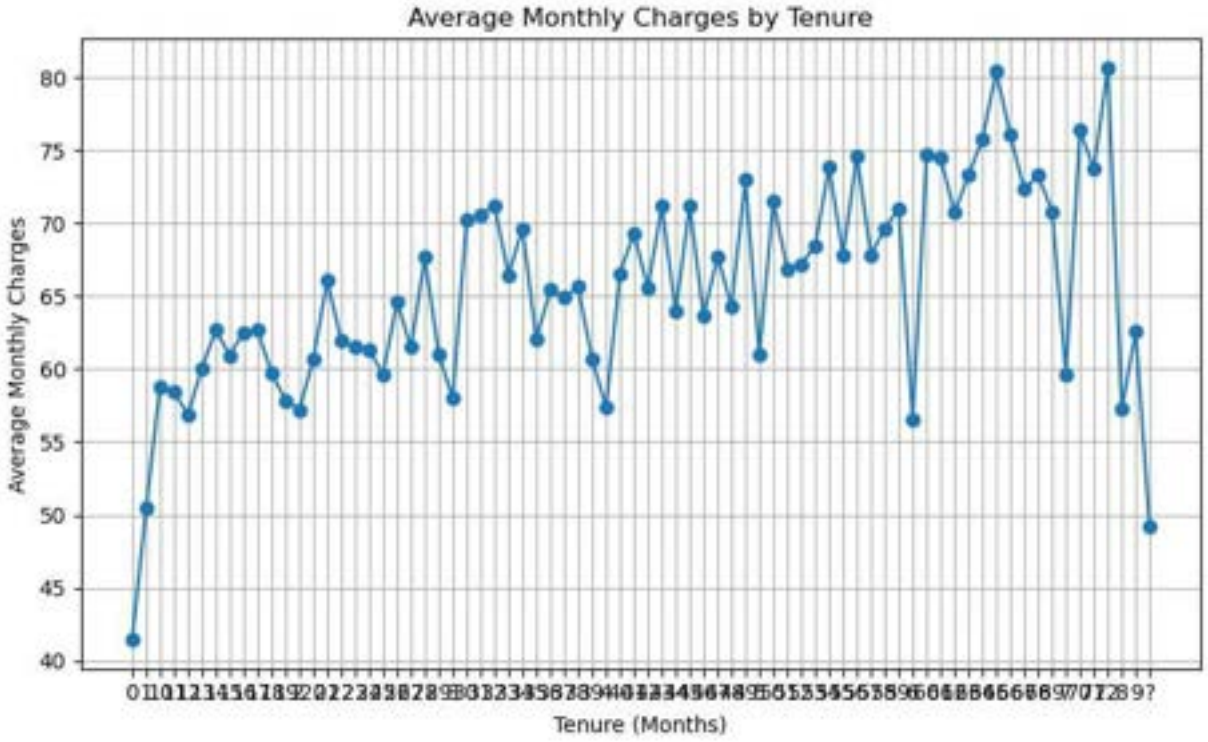
plt.title("Tenure vs Monthly Charges")
plt.xlabel("Tenure (Months)")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```



In [37]:
```
line_data = data.groupby("tenure", observed=True)["MonthlyCharges"].mean().reset_in

plt.figure(figsize=(8, 5))
plt.plot(
    line_data["tenure"],
    line_data["MonthlyCharges"],
    marker="o"
)

plt.title("Average Monthly Charges by Tenure")
plt.xlabel("Tenure (Months)")
plt.ylabel("Average Monthly Charges")
plt.grid(True)
plt.tight_layout()
plt.show()
```



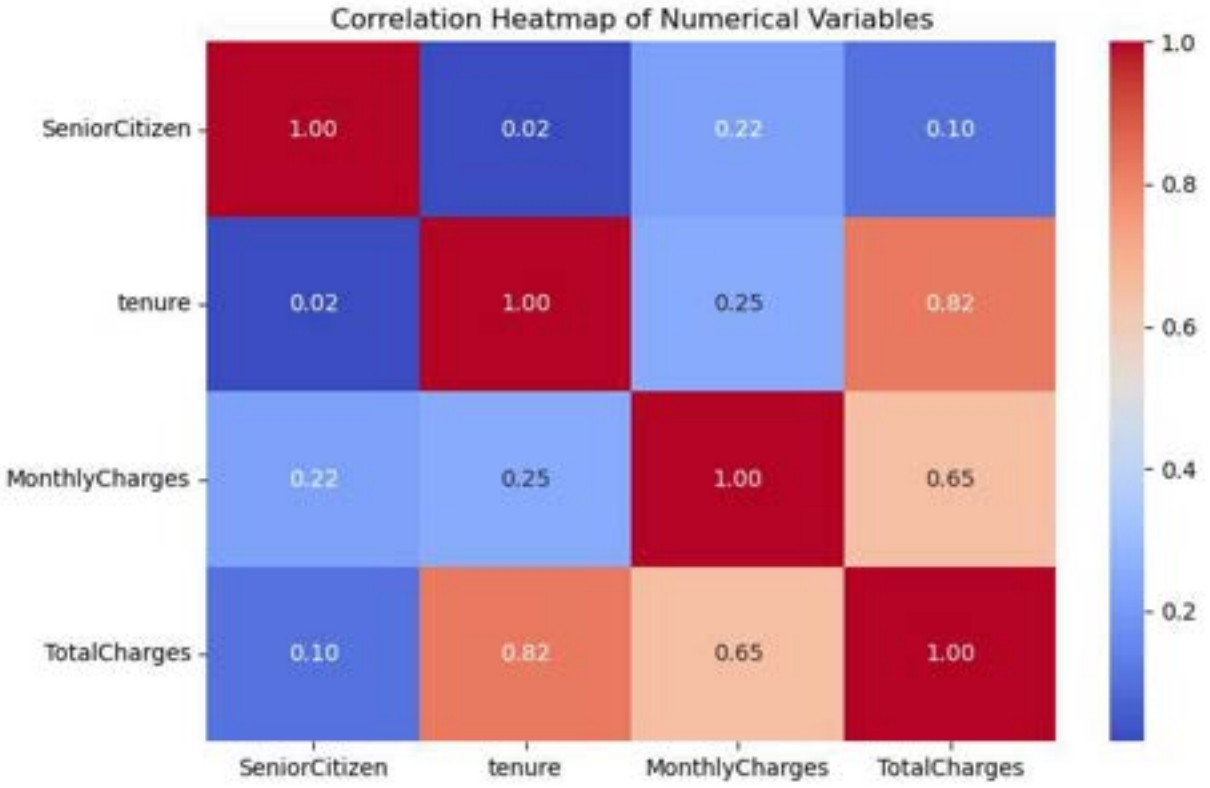
In [38]:
```
numeric_data = data[
    ["SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges"]
].copy()

numeric_data = numeric_data.apply(pd.to_numeric, errors="coerce")

correlation_matrix = numeric_data.corr()

plt.figure(figsize=(8, 5))
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

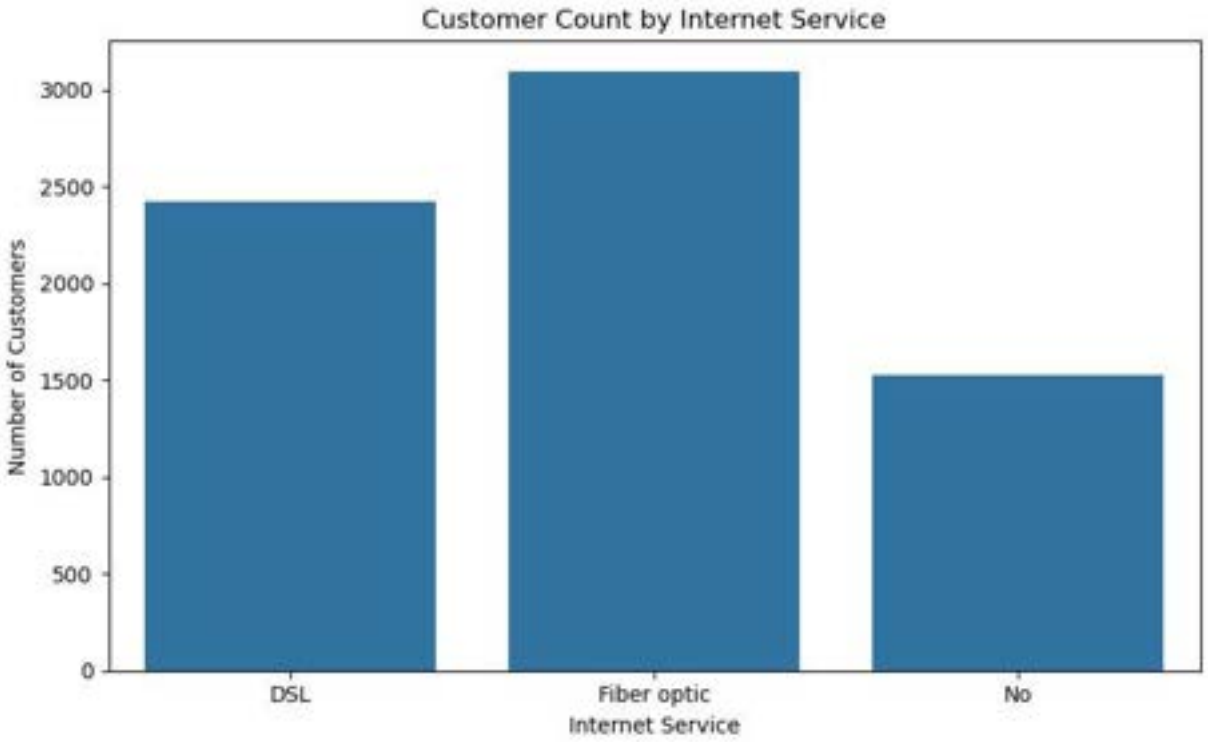
plt.title("Correlation Heatmap of Numerical Variables")
plt.tight_layout()
plt.show()
```



In [39]:
```
internet_counts = data["InternetService"].value_counts()

plt.figure(figsize=(8, 5))
sns.barplot(
    x=internet_counts.index,
    y=internet_counts.values
)

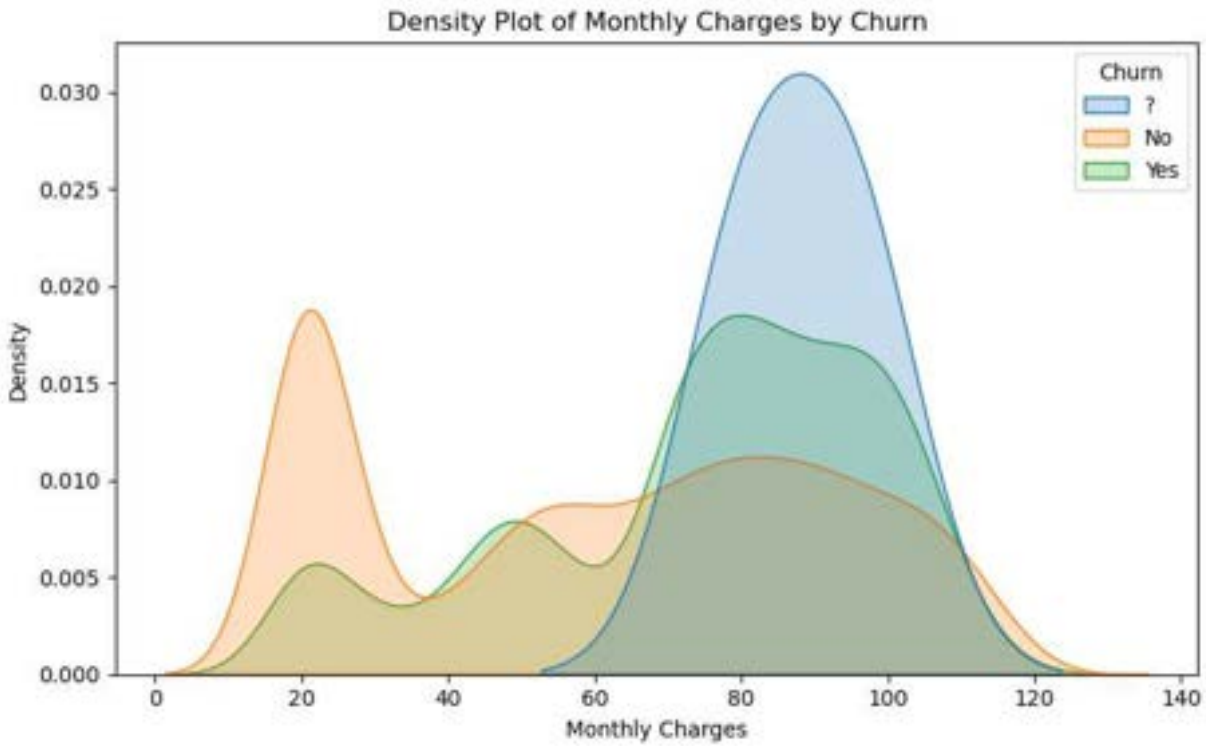
plt.title("Customer Count by Internet Service")
plt.xlabel("Internet Service")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```



In [40]:
```
plt.figure(figsize=(8, 5))

sns.kdeplot(
    data=data,
    x="MonthlyCharges",
    hue="Churn",
    fill=True,
    common_norm=False
)

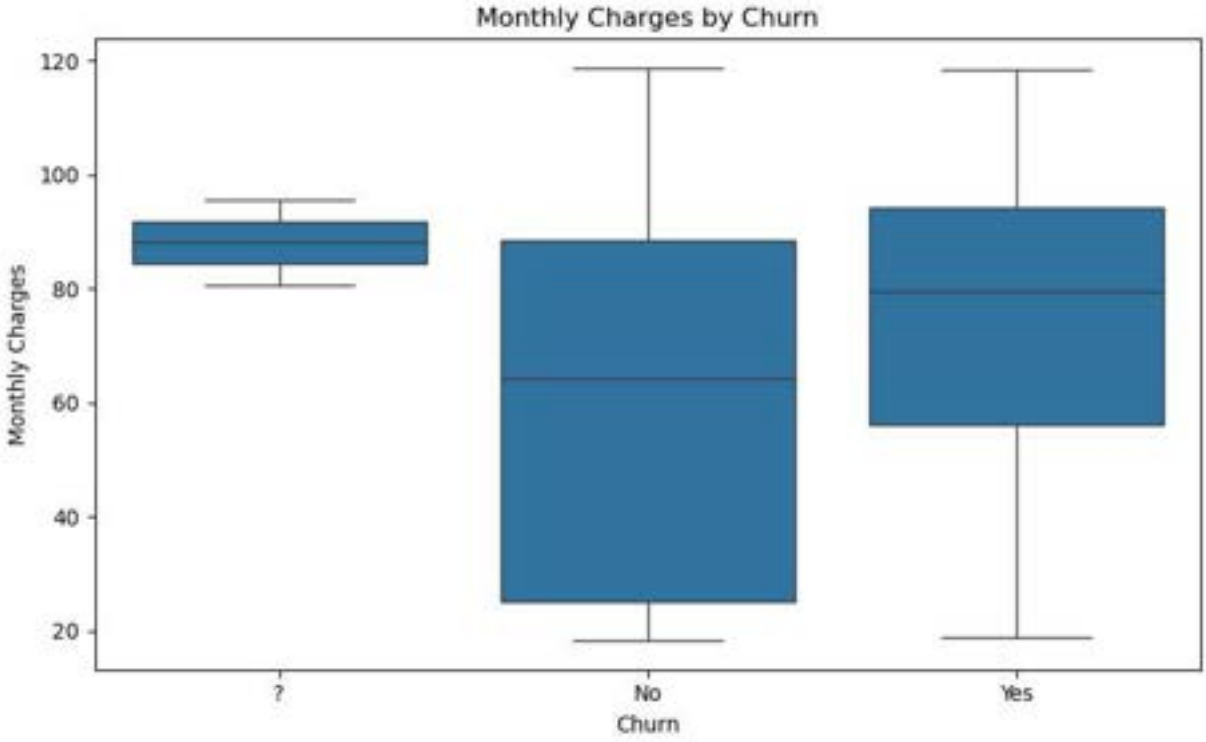
plt.title("Density Plot of Monthly Charges by Churn")
plt.xlabel("Monthly Charges")
plt.ylabel("Density")
plt.tight_layout()
plt.show()
```



In [41]:
```
plt.figure(figsize=(8, 5))

sns.boxplot(data=data, x="Churn", y="MonthlyCharges")

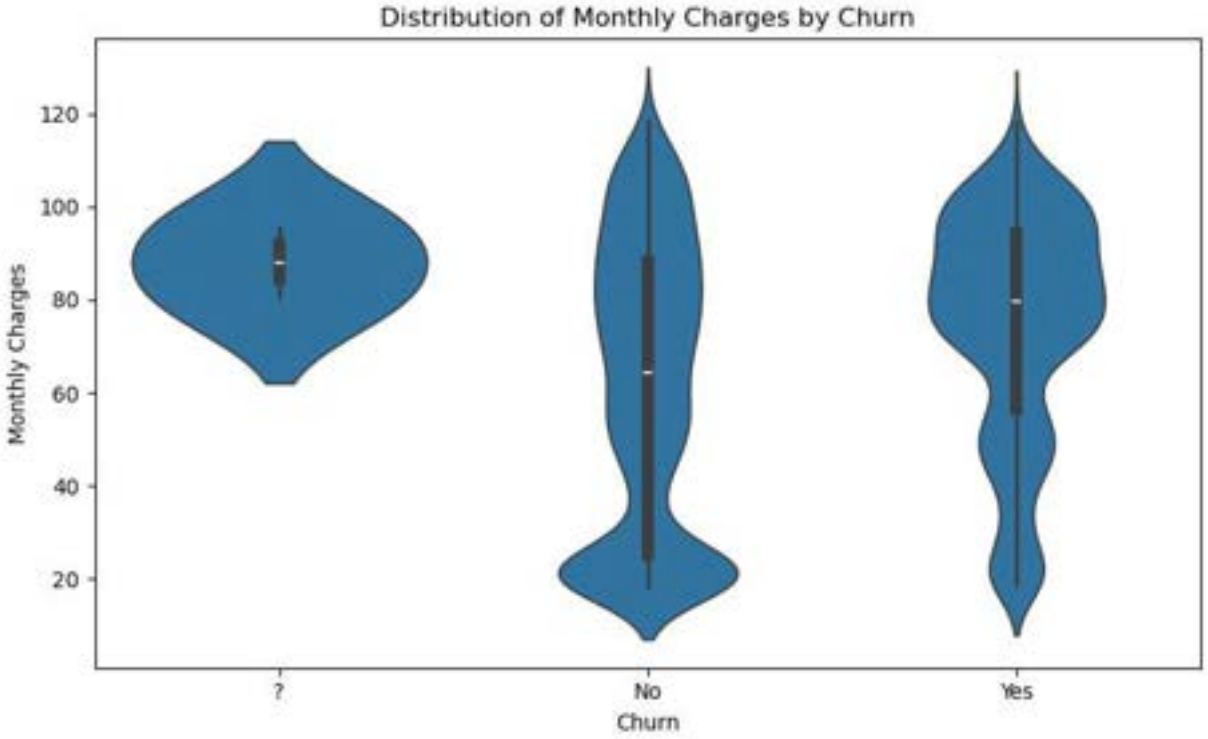
plt.title("Monthly Charges by Churn")
plt.xlabel("Churn")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```



In [42]:
```
plt.figure(figsize=(8, 5))

sns.violinplot(data=data, x="Churn", y="MonthlyCharges")

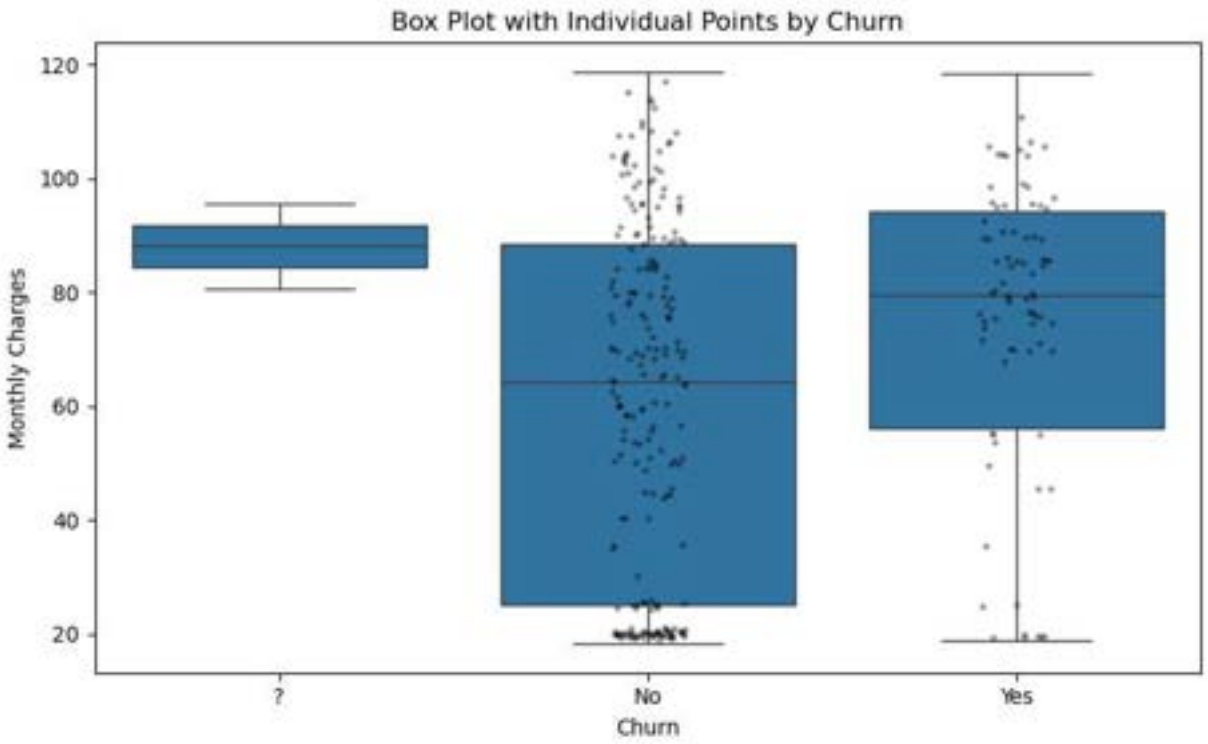
plt.title("Distribution of Monthly Charges by Churn")
plt.xlabel("Churn")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```



In [43]:
```
plt.figure(figsize=(8, 5))

sns.boxplot(data=data, x="Churn", y="MonthlyCharges")
sns.stripplot(
    data=data.sample(min(300, len(data)), random_state=42),
    x="Churn",
    y="MonthlyCharges",
    color="black",
    alpha=0.4,
    size=3
)

plt.title("Box Plot with Individual Points by Churn")
plt.xlabel("Churn")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```



In [44]:
```
numeric_data = data[
    ["SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges"]
].apply(pd.to_numeric, errors="coerce")

correlation_matrix = numeric_data.corr()

print("Correlation Matrix")
display(correlation_matrix.round(2))
```

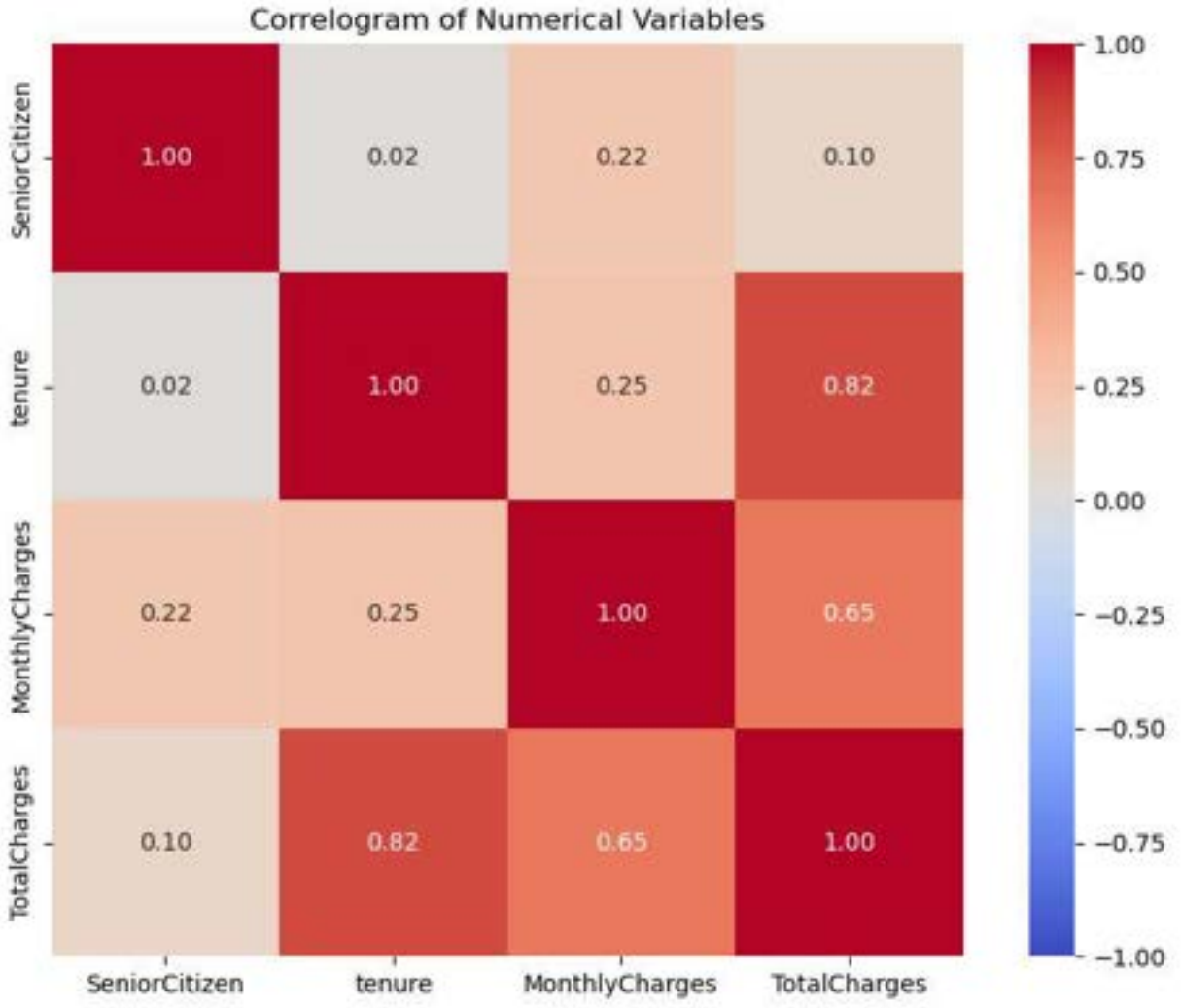
Correlation Matrix

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
SeniorCitizen	1.00	0.02	0.22	0.10
tenure	0.02	1.00	0.25	0.82
MonthlyCharges	0.22	0.25	1.00	0.65
TotalCharges	0.10	0.82	0.65	1.00

In [45]:
```
plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    fmt=".2f",
    square=True
)
```

```
plt.title("Correlogram of Numerical Variables")
plt.tight_layout()
plt.show()
```



In [46]:

```
sample_data = data.sample(min(100, len(data)), random_state=42)

plt.figure(figsize=(9, 6))

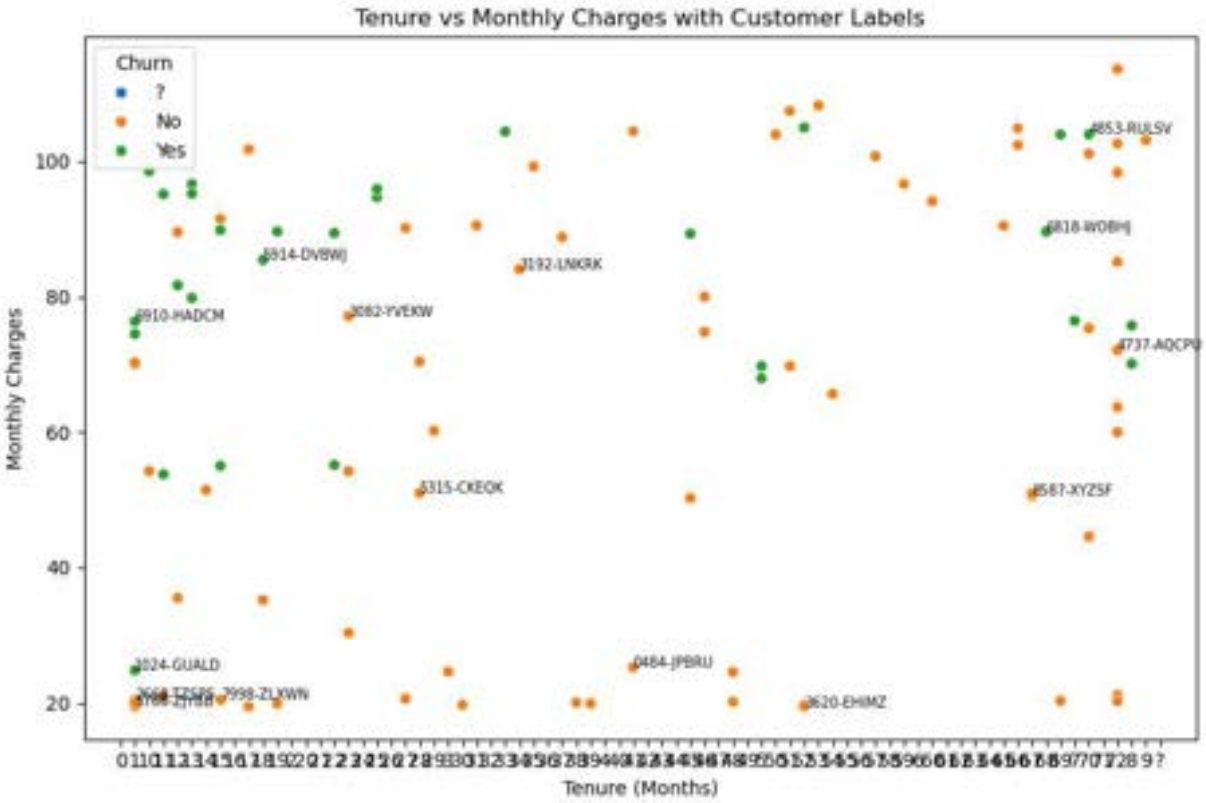
sns.scatterplot(
    data=sample_data,
    x="tenure",
    y="MonthlyCharges",
    hue="Churn"
)

plt.title("Tenure vs Monthly Charges with Customer Labels")
plt.xlabel("Tenure (Months)")
plt.ylabel("Monthly Charges")

for _, row in sample_data.head(15).iterrows():
    plt.annotate(
        str(row["customerID"]),
        (row["tenure"], row["MonthlyCharges"]),
        fontsize=7
```

```
    )

plt.tight_layout()
plt.show()
```



Assessment 5: Multivariate Analysis

Name: Dharsha R

Register Number: 23BDS0274

Objective

To perform multivariate analysis using scatter plots, bubble plots, faceting, pair plots, and parallel coordinates on the Telco Customer Churn dataset.

In [47]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import parallel_coordinates

sns.set_theme(style="whitegrid")

data.columns = data.columns.str.strip()

for col in ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen"]:
```

```
    data[col] = pd.to_numeric(data[col], errors="coerce")

numeric_cols = ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen"]

plot_data = data.dropna(subset=numeric_cols + ["Churn"]).copy()

print("Dataset Shape:", data.shape)
print("Numeric columns:", numeric_cols)
display(plot_data.head())
```

```
Dataset Shape: (7043, 21)
Numeric columns: ['tenure', 'MonthlyCharges', 'TotalCharges', 'SeniorCitizen']
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	female	0.0	Yes	No	1.0	No	No ph se
1	5575-GNVDE	male	0.0	No	No	34.0	Yes	
2	3668-QPYBK	male	0.0	No	No	2.0	Yes	
3	7795-CFOCW	male	0.0	No	No	45.0	No	No ph se
4	9237-HQITU	female	0.0	No	No	2.0	Yes	

5 rows × 21 columns

In [48]:
```
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=plot_data.sample(min(1000, len(plot_data)), random_state=42),
    x="tenure",
    y="MonthlyCharges",
    hue="Churn",
    alpha=0.7
)

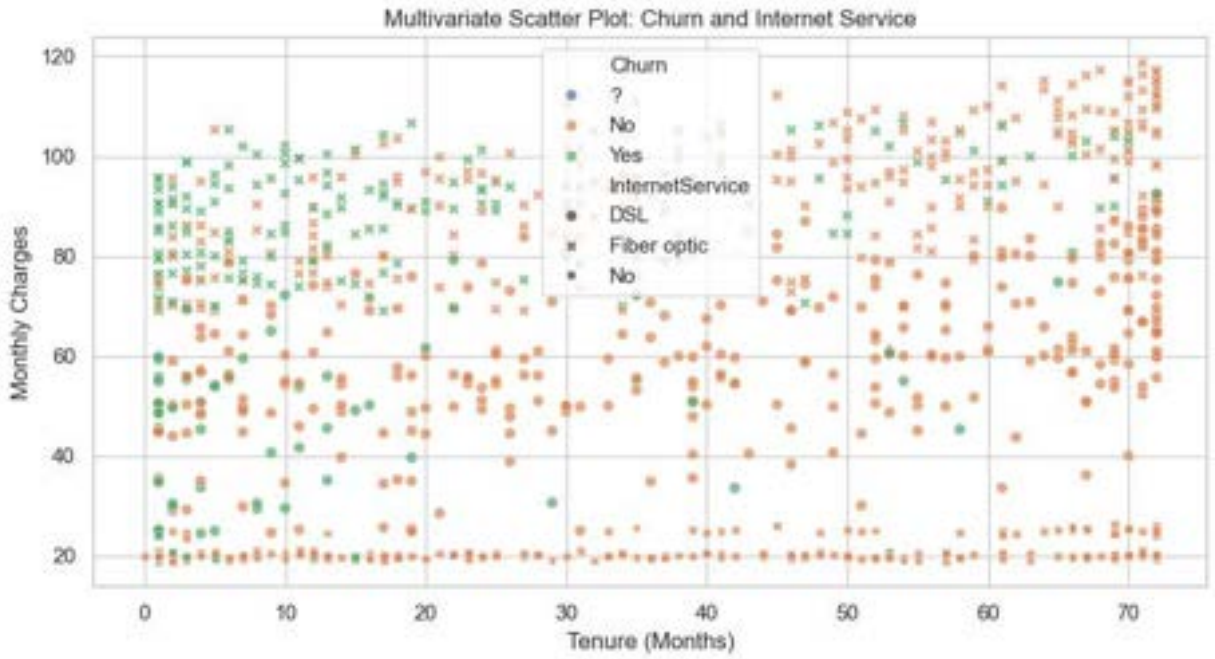
plt.title("Tenure vs Monthly Charges by Churn")
plt.xlabel("Tenure (Months)")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```



In [49]:
```
plt.figure(figsize=(9, 5))

sns.scatterplot(
    data=plot_data.sample(min(1000, len(plot_data)), random_state=42),
    x="tenure",
    y="MonthlyCharges",
    hue="Churn",
    style="InternetService",
    alpha=0.7
)

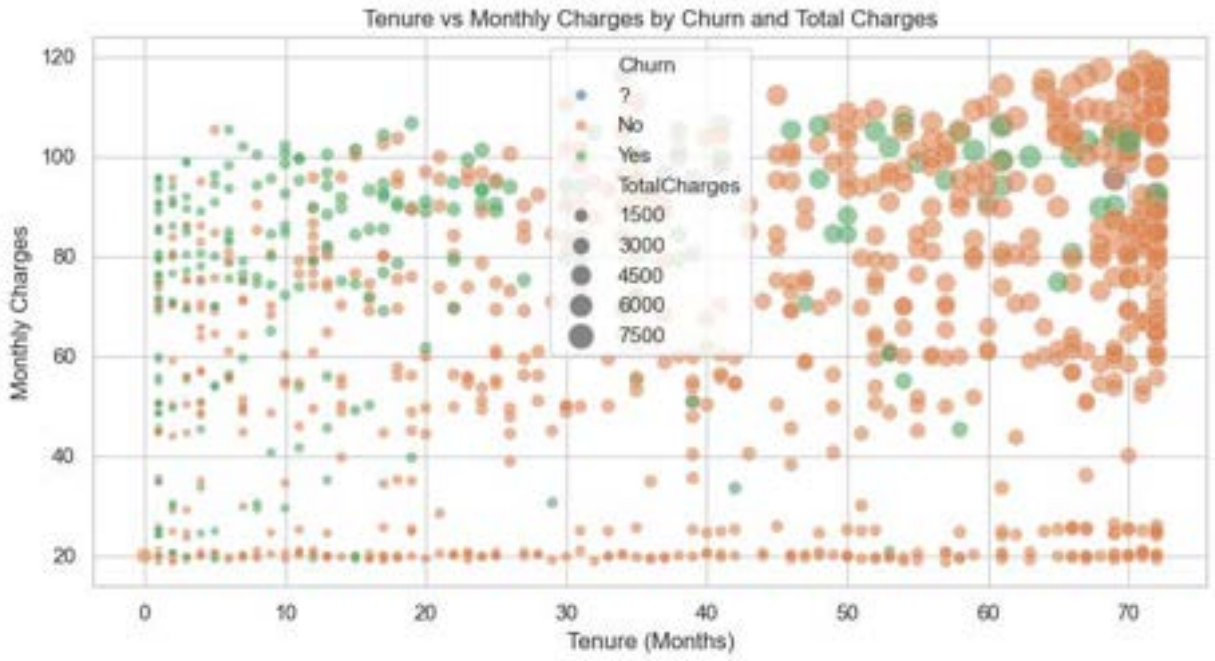
plt.title("Multivariate Scatter Plot: Churn and Internet Service")
plt.xlabel("Tenure (Months)")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```



In [50]:
```
plt.figure(figsize=(9, 5))

sns.scatterplot(
    data=plot_data.sample(min(1000, len(plot_data)), random_state=42),
    x="tenure",
    y="MonthlyCharges",
    hue="Churn",
    size="TotalCharges",
    sizes=(20, 200),
    alpha=0.6,
    legend="brief"
)

plt.title("Tenure vs Monthly Charges by Churn and Total Charges")
plt.xlabel("Tenure (Months)")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```



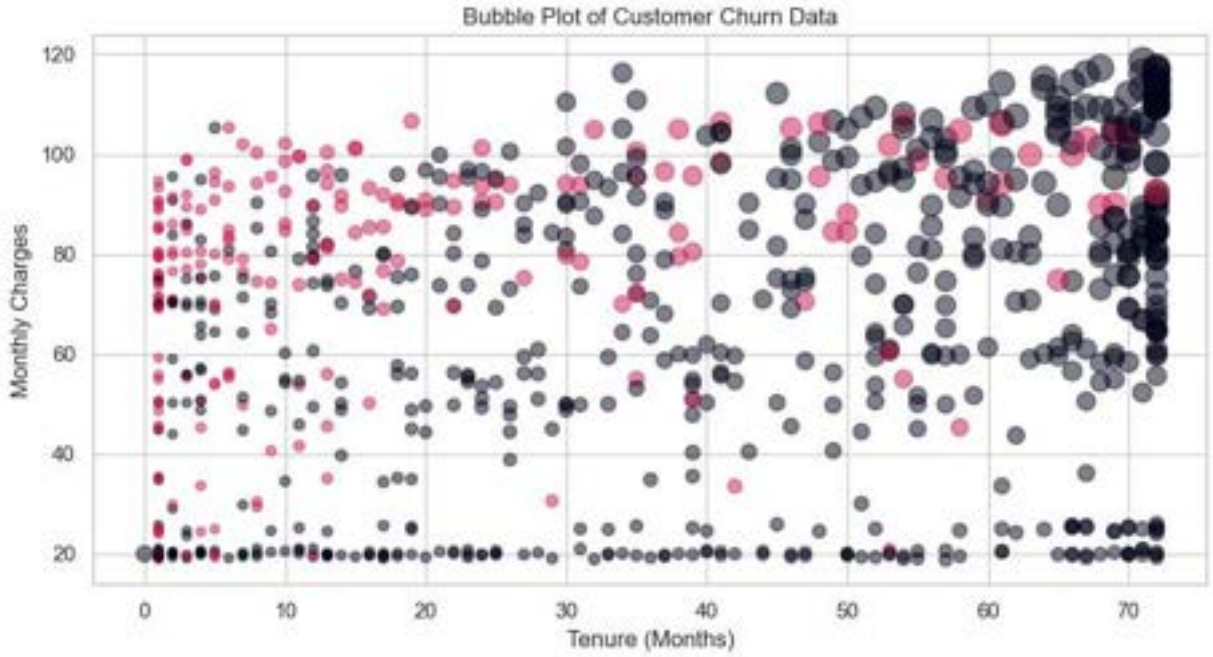
In [51]:

```
bubble_data = plot_data.sample(min(800, len(plot_data)), random_state=42)

plt.figure(figsize=(9, 5))

plt.scatter(
    bubble_data["tenure"],
    bubble_data["MonthlyCharges"],
    s=20 + 180 * (
        bubble_data["TotalCharges"] /
        max(plot_data["TotalCharges"].max(), 1)
    ),
    c=pd.factorize(bubble_data["Churn"])[0],
    alpha=0.5
)

plt.title("Bubble Plot of Customer Churn Data")
plt.xlabel("Tenure (Months)")
plt.ylabel("Monthly Charges")
plt.tight_layout()
plt.show()
```



In [52]:
```
facet_data = plot_data.sample(min(2000, len(plot_data)), random_state=42)

g = sns.relplot(
    data=facet_data,
    x="tenure",
    y="MonthlyCharges",
    col="Churn",
    hue="InternetService",
    kind="scatter",
    height=4,
    aspect=1.1,
    alpha=0.7
)

g.set_axis_labels("Tenure (Months)", "Monthly Charges")
g.set_titles("Churn = {col_name}")
g.fig.suptitle("Facet Wrap: Tenure and Monthly Charges", y=1.05)
plt.show()
```

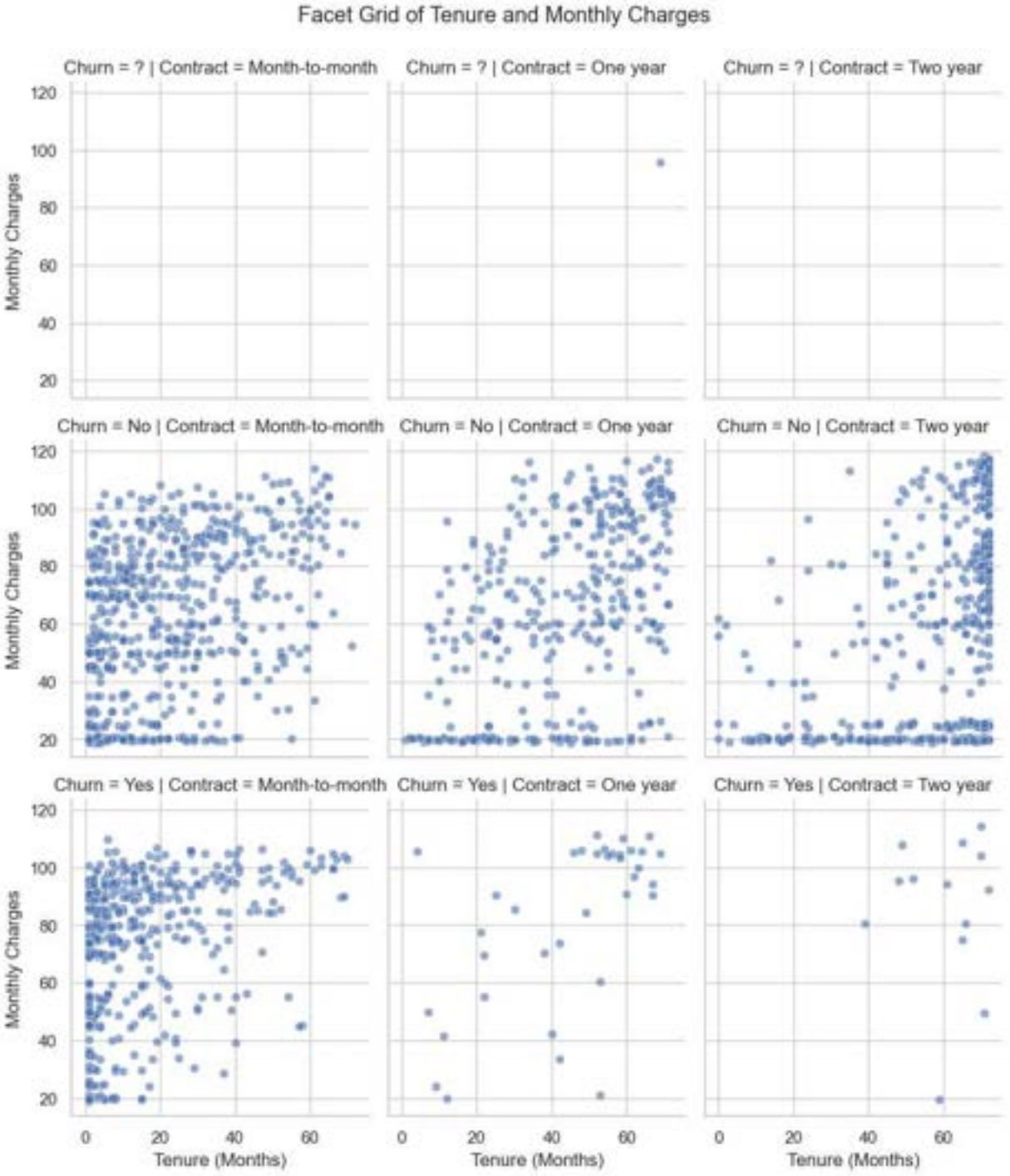


In [53]:
```
facet_data = plot_data.sample(min(2000, len(plot_data)), random_state=42)

g = sns.relplot(
    data=facet_data,
    x="tenure",
```

```
    y="MonthlyCharges",
    row="Churn",
    col="Contract",
    kind="scatter",
    height=3.5,
    aspect=0.9,
    alpha=0.6
)

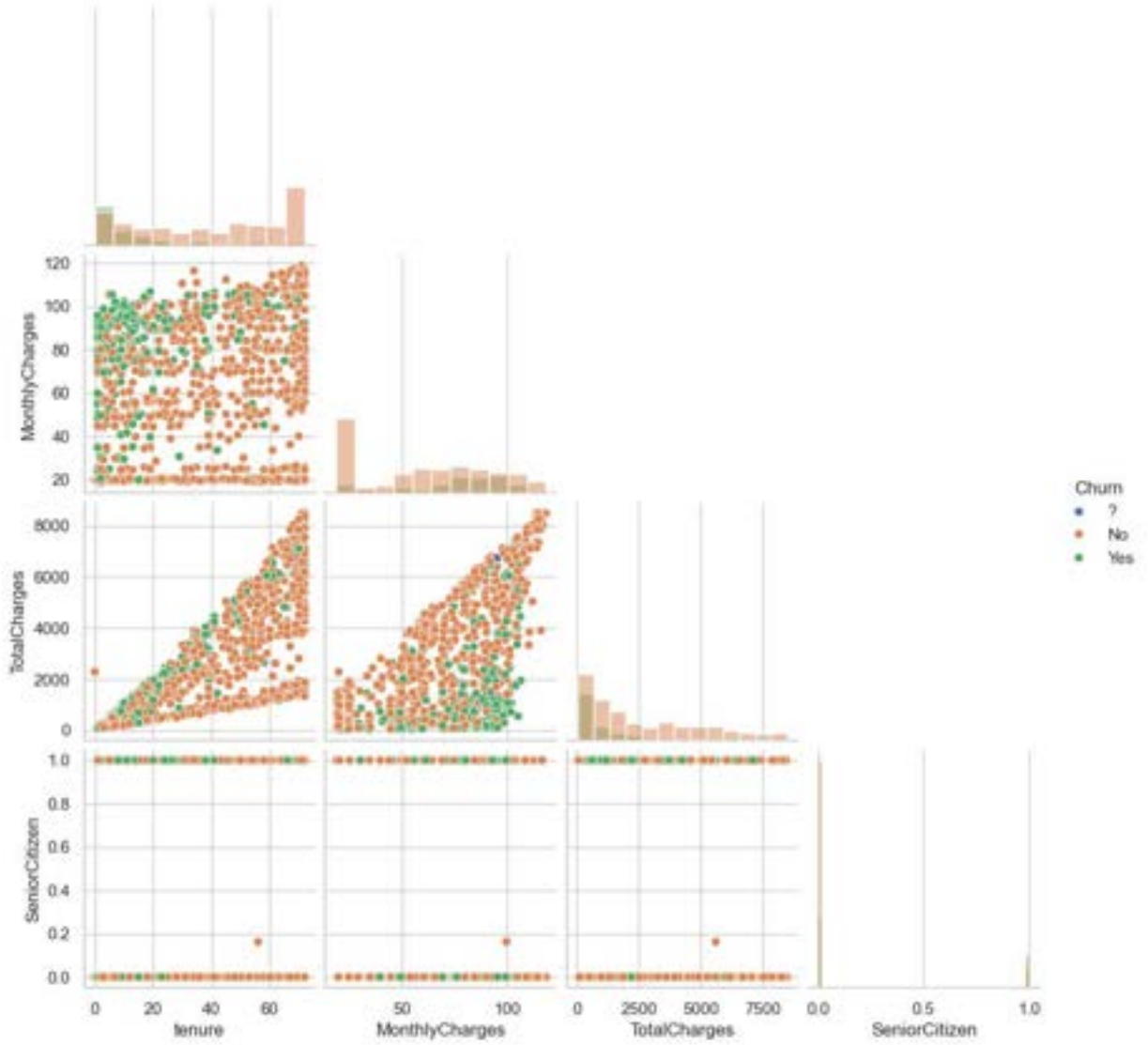
g.set_axis_labels("Tenure (Months)", "Monthly Charges")
g.set_titles("Churn = {row_name} | Contract = {col_name}")
g.fig.suptitle("Facet Grid of Tenure and Monthly Charges", y=1.03)
plt.show()
```



In [54]:
```
pair_data = plot_data[
    ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen", "Churn"]
].sample(min(1000, len(plot_data)), random_state=42)

sns.pairplot(
    pair_data,
    vars=["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen"],
    hue="Churn",
    diag_kind="hist",
    corner=True
)

plt.show()
```



In [55]:
```
parallel_data = plot_data[
    ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen", "Churn"]
].copy()

features = ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen"]

for col in features:
    minimum = parallel_data[col].min()
```

```
    maximum = parallel_data[col].max()

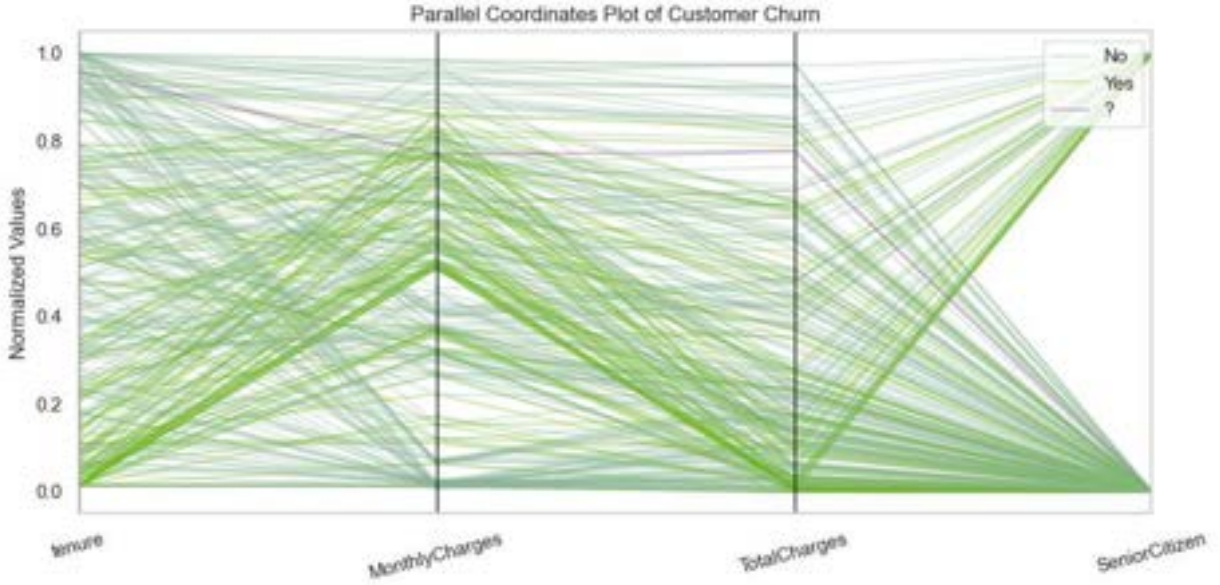
    if maximum != minimum:
        parallel_data[col] = (
            (parallel_data[col] - minimum) / (maximum - minimum)
        )
    else:
        parallel_data[col] = 0

parallel_sample = parallel_data.sample(
    min(300, len(parallel_data)),
    random_state=42
)

plt.figure(figsize=(10, 5))

parallel_coordinates(
    parallel_sample,
    class_column="Churn",
    cols=features,
    alpha=0.3
)

plt.title("Parallel Coordinates Plot of Customer Churn")
plt.ylabel("Normalized Values")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```



Conclusion

Multivariate analysis was performed on the Telco Customer Churn dataset using scatter plots, bubble plots, facet wrap, facet grid, pair plots, and parallel coordinates. These visualizations help compare multiple variables and identify patterns and relationships among customer tenure, monthly charges, total charges, internet service, contract type, and churn.

Assessment 6: Time Series Analysis

Name: Dharsha R

Register Number: 23BDS0274

Objective

To perform time series analysis using the AirPassengers dataset, visualize trends and seasonality, analyze ACF and PACF plots, fit an ARIMA model, and forecast future passenger values.

In [56]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA

passengers = [
    112,118,132,129,121,135,148,148,136,119,104,118,
    115,126,141,135,125,149,170,170,158,133,114,140,
    145,150,178,163,172,178,199,199,184,162,146,166,
    171,180,193,181,183,218,230,242,209,191,172,194,
    196,196,236,235,229,243,264,272,237,211,180,201,
    204,188,235,227,234,264,302,293,259,229,203,229,
    242,233,267,269,270,315,364,347,312,274,237,278,
    284,277,317,313,318,374,413,405,355,306,271,306,
    315,301,356,348,355,422,465,467,404,347,305,336,
    340,318,362,348,363,435,491,505,404,359,310,337,
    360,342,406,396,420,472,548,559,463,407,362,405,
    417,391,419,461,472,535,622,606,508,461,390,432
]

dates = pd.date_range(start="1949-01-01", periods=len(passengers), freq="MS")
air_data = pd.Series(passengers, index=dates, name="Passengers")

print(air_data.head())
print("Dataset size:", len(air_data))
```

```
1949-01-01    112
1949-02-01    118
1949-03-01    132
1949-04-01    129
1949-05-01    121
Freq: MS, Name: Passengers, dtype: int64
Dataset size: 144
```

In [57]:
```
print("First 5 observations:")
display(air_data.head())

print("Last 5 observations:")
display(air_data.tail())
```

```
First 5 observations:
1949-01-01    112
1949-02-01    118
1949-03-01    132
1949-04-01    129
1949-05-01    121
Freq: MS, Name: Passengers, dtype: int64
Last 5 observations:
1960-08-01    606
1960-09-01    508
1960-10-01    461
1960-11-01    390
1960-12-01    432
Freq: MS, Name: Passengers, dtype: int64
```

In [58]:
```
print("Start Date:", air_data.index.min())
print("End Date:", air_data.index.max())
print("Frequency:", air_data.index.freq)
print("Data Type:", air_data.dtype)
print("Number of observations:", len(air_data))
```

```
Start Date: 1949-01-01 00:00:00
End Date: 1960-12-01 00:00:00
Frequency: <MonthBegin>
Data Type: int64
Number of observations: 144
```

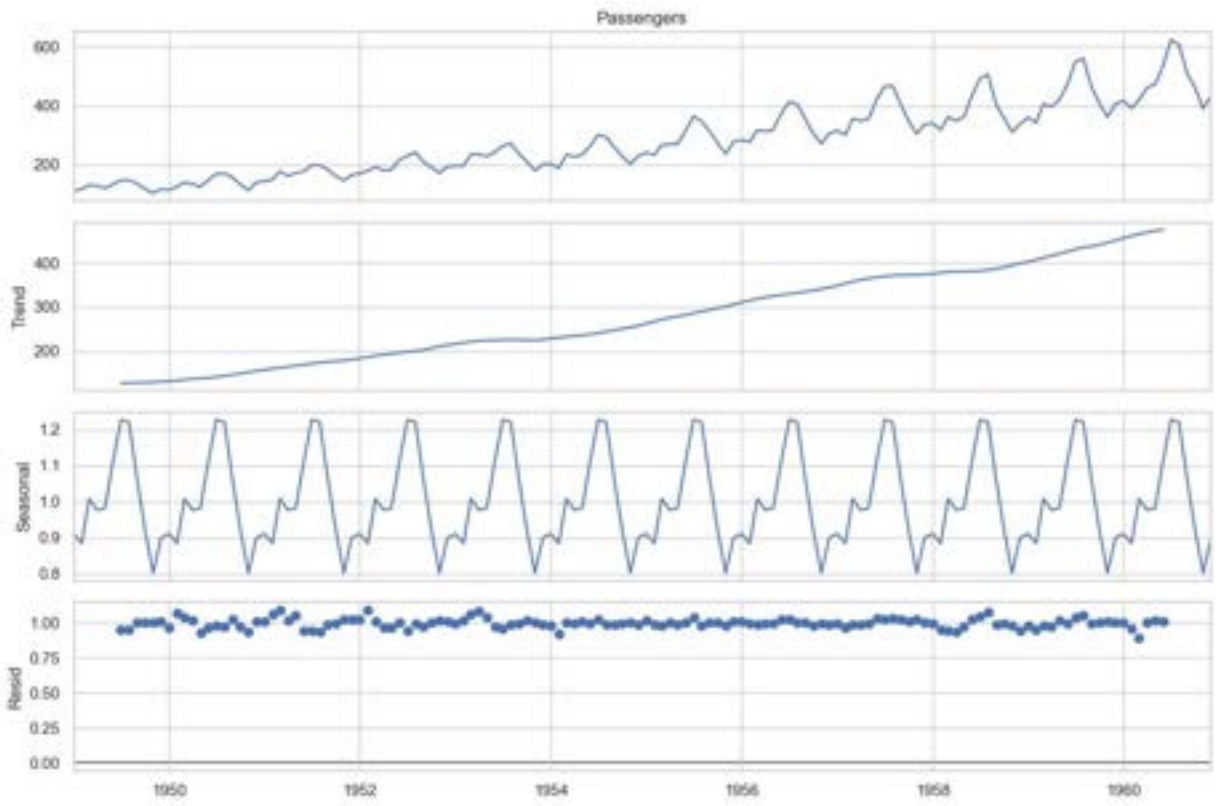
In [59]:
```
plt.figure(figsize=(12, 5))

plt.plot(air_data, marker="o")
plt.title("Monthly International Airline Passengers (1949–1960)")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.show()
```



In [60]:
```
decomposition = seasonal_decompose(
    air_data,
    model="multiplicative",
    period=12
)

decomposition.plot()
plt.gcf().set_size_inches(12, 8)
plt.tight_layout()
plt.show()
```

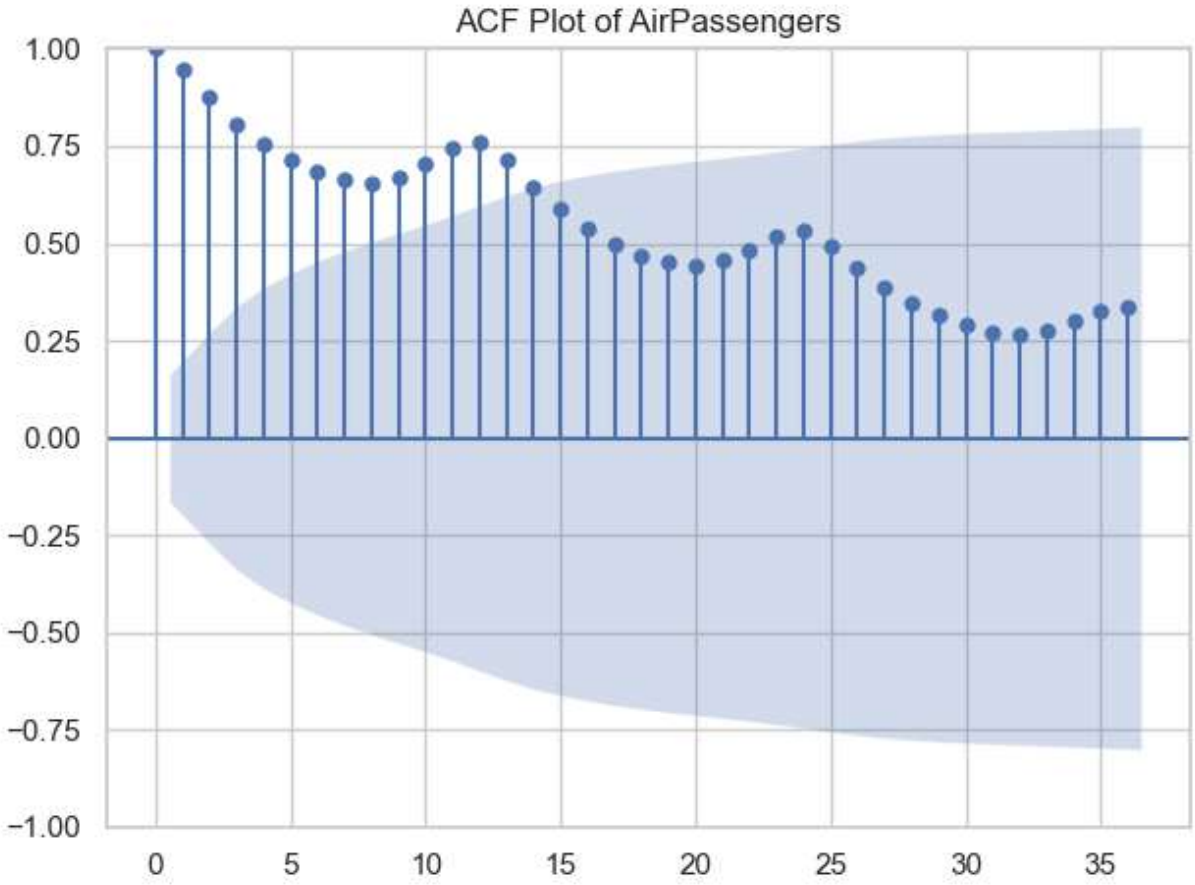


In [61]:
```
plt.figure(figsize=(10, 4))

plot_acf(air_data, lags=36)
plt.title("ACF Plot of AirPassengers")
```

```
plt.tight_layout()
plt.show()
```

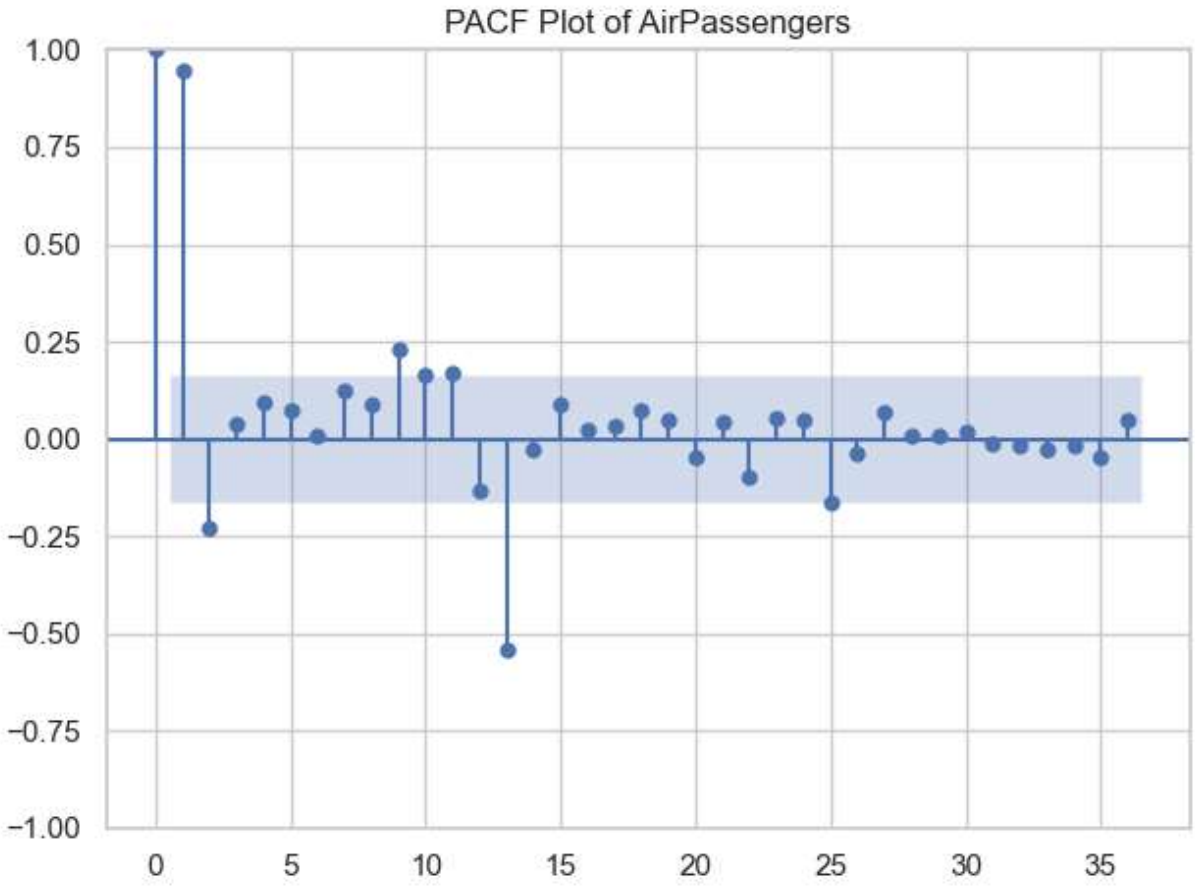
```
<Figure size 1000x400 with 0 Axes>
```



In [62]:
```
plt.figure(figsize=(10, 4))

plot_pacf(air_data, lags=36, method="ywm")
plt.title("PACF Plot of AirPassengers")
plt.tight_layout()
plt.show()
```

```
<Figure size 1000x400 with 0 Axes>
```



In [64]:
```
import warnings
from statsmodels.tools.sm_exceptions import ConvergenceWarning
from statsmodels.tsa.arima.model import ARIMA

warnings.filterwarnings("ignore", category=ConvergenceWarning)

model = ARIMA(
    air_data,
    order=(1, 1, 1),
    enforce_stationarity=False,
    enforce_invertibility=False
)

model_fit = model.fit()

print(model_fit.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  144
Model:                 ARIMA(1, 1, 1)   Log Likelihood                -685.646
Date:                Sun, 18 Oct 2026   AIC                           1377.293
Time:                        13:38:22   BIC                           1386.139
Sample:                    01-01-1949   HQIC                          1380.888
                         - 12-01-1960
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.4631      0.125     -3.704      0.000      -0.708      -0.218
ma.L1          0.8581      0.080     10.729      0.000       0.701       1.015
sigma2       974.6344    110.197      8.845      0.000     758.653    1190.616
===================================================================================
Ljung-Box (L1) (Q):                   0.14   Jarque-Bera (JB):                 1.69
Prob(Q):                              0.71   Prob(JB):                         0.43
Heteroskedasticity (H):               6.90   Skew:                            -0.19
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.37
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-ste
p).
```

In [65]:

```python
forecast_result = model_fit.get_forecast(steps=12)

forecast_values = forecast_result.predicted_mean
confidence_interval = forecast_result.conf_int()

forecast_dates = pd.date_range(
    start=air_data.index[-1] + pd.offsets.MonthBegin(1),
    periods=12,
    freq="MS"
)

forecast_values.index = forecast_dates
confidence_interval.index = forecast_dates

forecast_table = pd.DataFrame({
    "Forecast": forecast_values,
    "Lower Bound": confidence_interval.iloc[:, 0],
    "Upper Bound": confidence_interval.iloc[:, 1]
})

display(forecast_table.round(2))
```

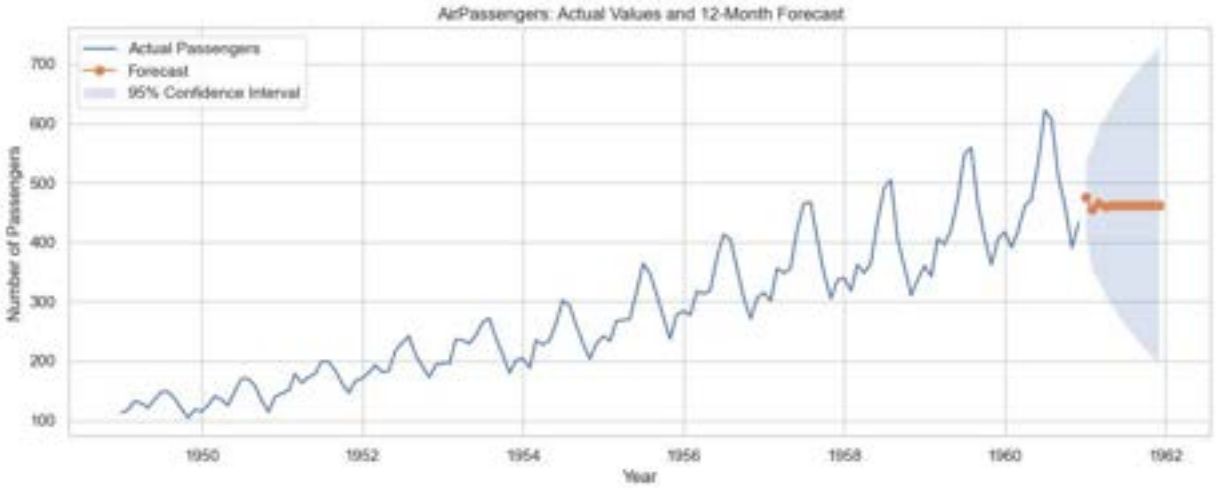
	Forecast	Lower Bound	Upper Bound
1961-01-01	475.98	414.80	537.17
1961-02-01	455.62	350.59	560.64
1961-03-01	465.05	336.48	593.62
1961-04-01	460.68	309.59	611.77
1961-05-01	462.70	293.15	632.26
1961-06-01	461.77	275.11	648.43
1961-07-01	462.20	260.07	664.33
1961-08-01	462.00	245.42	678.58
1961-09-01	462.09	232.01	692.18
1961-10-01	462.05	219.19	704.91
1961-11-01	462.07	207.08	717.06
1961-12-01	462.06	195.49	728.63

In [66]:
```
plt.figure(figsize=(12, 5))

plt.plot(air_data, label="Actual Passengers")
plt.plot(forecast_values, label="Forecast", marker="o")

plt.fill_between(
    forecast_dates,
    confidence_interval.iloc[:, 0],
    confidence_interval.iloc[:, 1],
    alpha=0.2,
    label="95% Confidence Interval"
)

plt.title("AirPassengers: Actual Values and 12-Month Forecast")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.tight_layout()
plt.show()
```



Conclusion

Time series analysis was performed on the AirPassengers dataset. The time series plot showed the trend in monthly passenger counts. Seasonal decomposition separated the trend, seasonal, and residual components. ACF and PACF plots were used to examine autocorrelation. An ARIMA model was fitted, and passenger counts for the next 12 months were forecasted.

Assessment 7: 1D, 2D, and N-D Analysis

Name: Dharsha R

Register Number: 23BDS0274

Objective

To perform one-dimensional, two-dimensional, and multidimensional analysis on the Telco Customer Churn dataset using statistical summaries, frequency tables, visualizations, correlation analysis, and grouped analysis.

In [67]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/telco%20custome

data = pd.read_csv(url)
data.columns = data.columns.str.strip()

for col in ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen"]:
    data[col] = pd.to_numeric(data[col], errors="coerce")
```

```
data["TotalCharges"] = data["TotalCharges"].fillna(data["TotalCharges"].median())

sns.set_theme(style="whitegrid")

print("Dataset Shape:", data.shape)
display(data.head())
```

```
Dataset Shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	Female	0.0	Yes	No	1.0	No	No pl se
1	5575-GNVDE	Male	0.0	No	No	34.0	Yes	
2	3668-QPYBK	Male	0.0	No	No	2.0	Yes	
3	7795-CFOCW	Male	0.0	No	No	45.0	No	No pl se
4	9237-HQITU	Female	0.0	No	No	2.0	Yes	

5 rows × 21 columns

In [68]:
```
numeric_cols = ["tenure", "MonthlyCharges", "TotalCharges"]

for col in numeric_cols:
    x = data[col].dropna()

    print("\nColumn:", col)
    print("Mean:", x.mean())
    print("Median:", x.median())
    print("Minimum:", x.min())
    print("Maximum:", x.max())
    print("Range:", x.max() - x.min())
    print("Variance:", x.var())
    print("Standard Deviation:", x.std())
    print("IQR:", x.quantile(0.75) - x.quantile(0.25))
    print("Skewness:", x.skew())
    print("Kurtosis:", x.kurt())
```

```
Column: tenure
Mean: 32.37324243715381
Median: 29.0
Minimum: 0.0
Maximum: 72.0
Range: 72.0
Variance: 603.2956134765715
Standard Deviation: 24.562076733789663
IQR: 46.0
Skewness: 0.23932614516306638
Kurtosis: -1.387712707096787

Column: MonthlyCharges
Mean: 64.76188920454545
Median: 70.35
Minimum: 18.25
Maximum: 118.75
Range: 100.5
Variance: 905.2506826074163
Standard Deviation: 30.087384110411065
IQR: 54.349999999999994
Skewness: -0.22063130718130275
Kurtosis: -1.2571775588890481

Column: TotalCharges
Mean: 2281.9169281556156
Median: 1397.475
Minimum: 18.8
Maximum: 8684.8
Range: 8666.0
Variance: 5131449.978239644
Standard Deviation: 2265.2703984821865
IQR: 3384.375
Skewness: 0.9637890936414893
Kurtosis: -0.22640031928829618
```

In [69]:

```python
summary = data[numeric_cols].agg(
    ["mean", "median", "min", "max", "var", "std", "skew", "kurt"]
).T

summary["range"] = (
    data[numeric_cols].max() - data[numeric_cols].min()
)

summary["IQR"] = (
    data[numeric_cols].quantile(0.75)
    - data[numeric_cols].quantile(0.25)
)

display(summary.round(3))
```

	mean	median	min	max	var	std	skew	kurt	n
tenure	32.373	29.000	0.00	72.00	603.296	24.562	0.239	-1.388	
MonthlyCharges	64.762	70.350	18.25	118.75	905.251	30.087	-0.221	-1.257	
TotalCharges	2281.917	1397.475	18.80	8684.80	5131449.978	2265.270	0.964	-0.226	8(

In [70]:
```
frequency = data["Contract"].value_counts()
relative_frequency = data["Contract"].value_counts(normalize=True)

frequency_table = pd.DataFrame({
    "Frequency": frequency,
    "Relative Frequency": relative_frequency,
    "Percentage": relative_frequency * 100
})

display(frequency_table.round(2))
```

Contract	Frequency	Relative Frequency	Percentage
Month-to-month	3875	0.55	55.02
Two year	1695	0.24	24.07
One year	1473	0.21	20.91

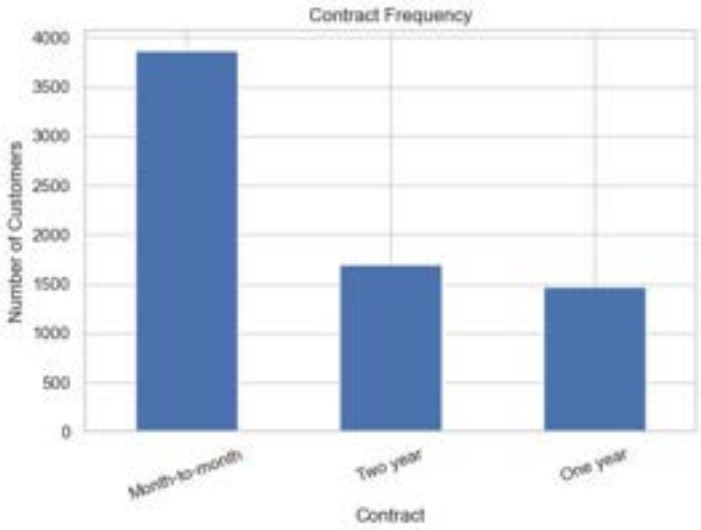
In [71]:
```
contract_counts = data["Contract"].value_counts()

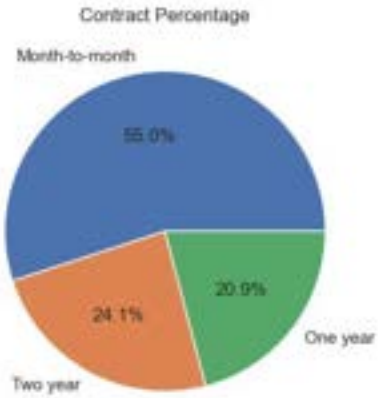
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

contract_counts.plot(kind="bar", ax=axes[0])
axes[0].set_title("Contract Frequency")
axes[0].set_xlabel("Contract")
axes[0].set_ylabel("Number of Customers")
axes[0].tick_params(axis="x", rotation=20)

contract_counts.plot(kind="pie", autopct="%1.1f%%", ax=axes[1])
axes[1].set_title("Contract Percentage")
axes[1].set_ylabel("")

plt.tight_layout()
plt.show()
```





In [72]:
```
data["tenure_group"] = pd.cut(
    data["tenure"],
    bins=[-1, 12, 24, 36, 48, 60, 72],
    labels=["0-12", "13-24", "25-36", "37-48", "49-60", "61-72"]
)

print("Binned Frequency:")
display(data["tenure_group"].value_counts(sort=False).to_frame("Frequency"))
```

```
Binned Frequency:
```

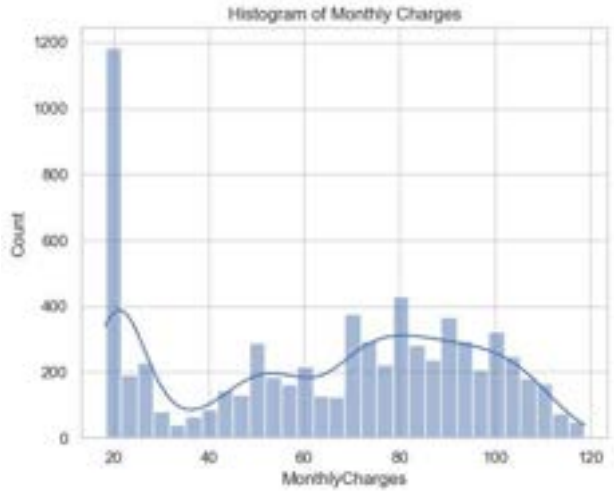
tenure_group	Frequency
0-12	2186
13-24	1023
25-36	831
37-48	762
49-60	832
61-72	1407

In [73]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.histplot(data=data, x="MonthlyCharges", bins=30, kde=True, ax=axes[0])
axes[0].set_title("Histogram of Monthly Charges")

sns.boxplot(data=data, y="MonthlyCharges", ax=axes[1])
axes[1].set_title("Boxplot of Monthly Charges")

plt.tight_layout()
plt.show()
```





In [74]:
```
contingency = pd.crosstab(data["Contract"], data["Churn"])

print("Contingency Table:")
display(contingency)

print("Row Percentages:")
display((pd.crosstab(data["Contract"], data["Churn"], normalize="index") * 100).rou

print("Column Percentages:")
display((pd.crosstab(data["Contract"], data["Churn"], normalize="columns") * 100).r
```

Contingency Table:

Contract	Churn ?	No	Yes
Month-to-month	0	2220	1654
One year	1	1306	166
Two year	1	1646	48

Row Percentages:

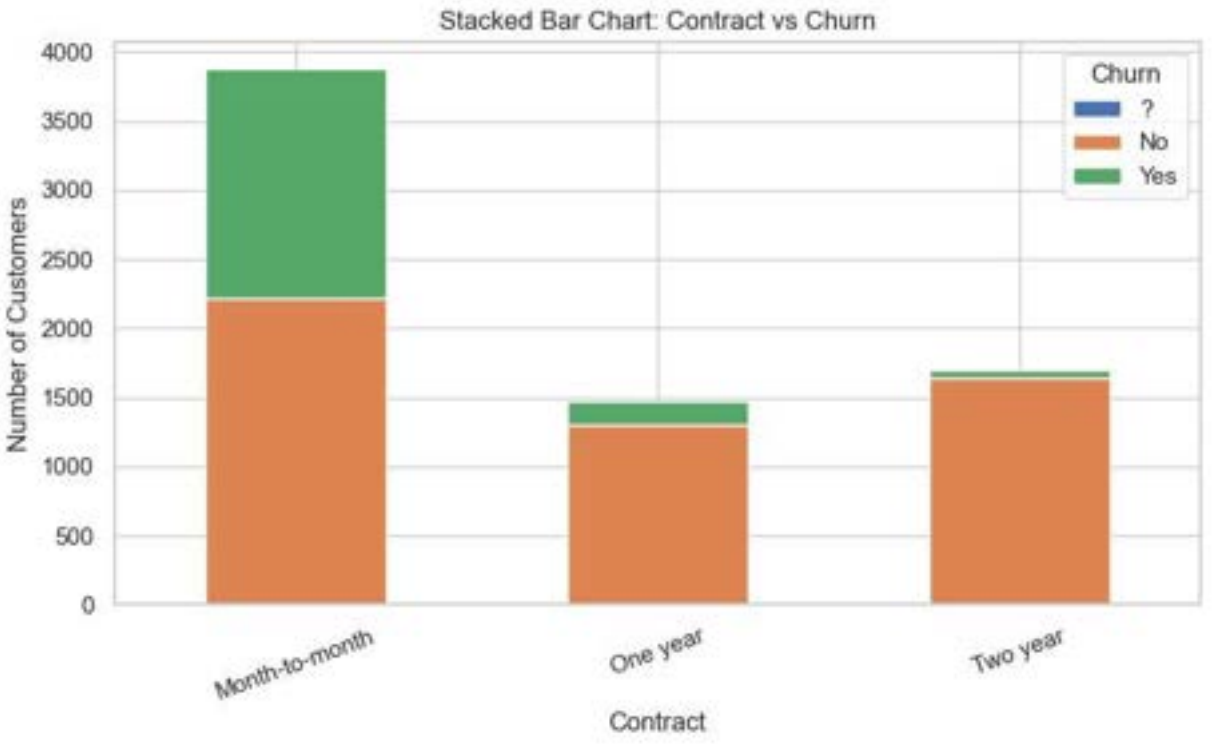
Contract	Churn ?	No	Yes
Month-to-month	0.00	57.31	42.69
One year	0.07	88.66	11.27
Two year	0.06	97.11	2.83

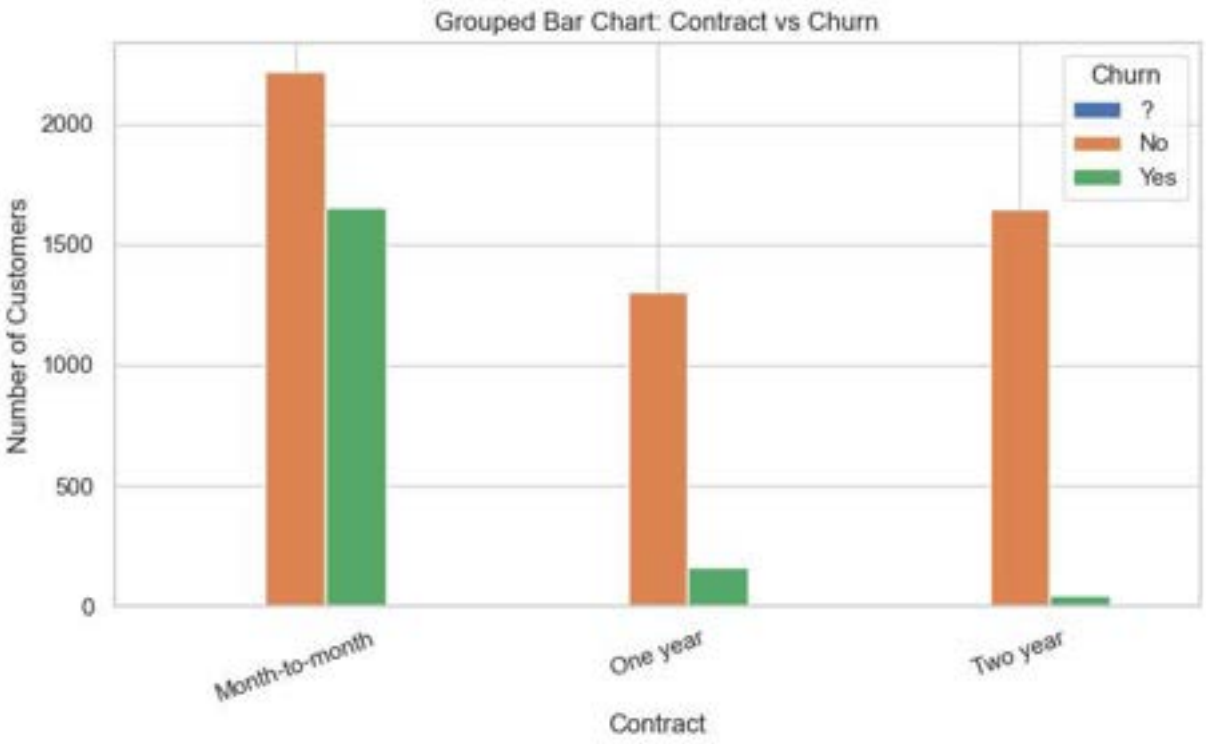
Column Percentages:

Churn	?	No	Yes
Contract			
Month-to-month	0.0	42.92	88.54
One year	50.0	25.25	8.89
Two year	50.0	31.83	2.57

In [75]:
```
contingency.plot(kind="bar", stacked=True, figsize=(8, 5))
plt.title("Stacked Bar Chart: Contract vs Churn")
plt.xlabel("Contract")
plt.ylabel("Number of Customers")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()

contingency.plot(kind="bar", stacked=False, figsize=(8, 5))
plt.title("Grouped Bar Chart: Contract vs Churn")
plt.xlabel("Contract")
plt.ylabel("Number of Customers")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```





In [76]:
```
group_summary = data.groupby("Contract", observed=True)["MonthlyCharges"].agg(
    ["mean", "median", "std"]
)

display(group_summary.round(2))
```

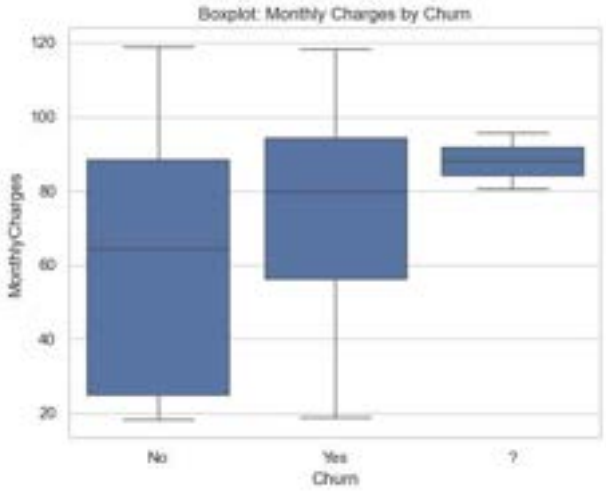
Contract	mean	median	std
Month-to-month	66.40	73.25	26.93
One year	65.02	68.75	31.83
Two year	60.79	64.35	34.67

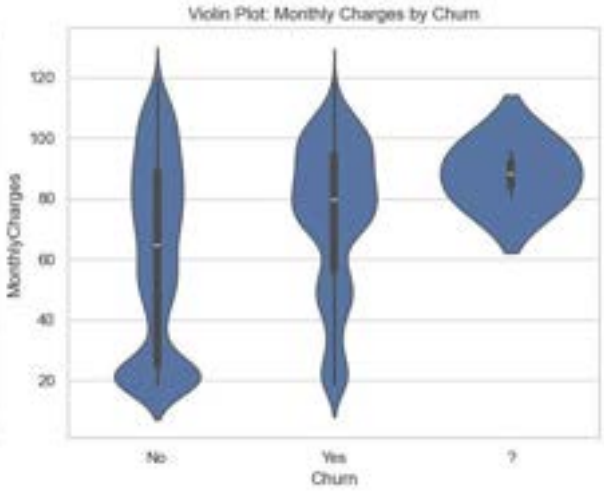
In [77]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.boxplot(data=data, x="Churn", y="MonthlyCharges", ax=axes[0])
axes[0].set_title("Boxplot: Monthly Charges by Churn")

sns.violinplot(data=data, x="Churn", y="MonthlyCharges", ax=axes[1])
axes[1].set_title("Violin Plot: Monthly Charges by Churn")

plt.tight_layout()
plt.show()
```





In [78]:
```
print("Pearson Correlation:")
print(data["tenure"].corr(data["MonthlyCharges"], method="pearson"))

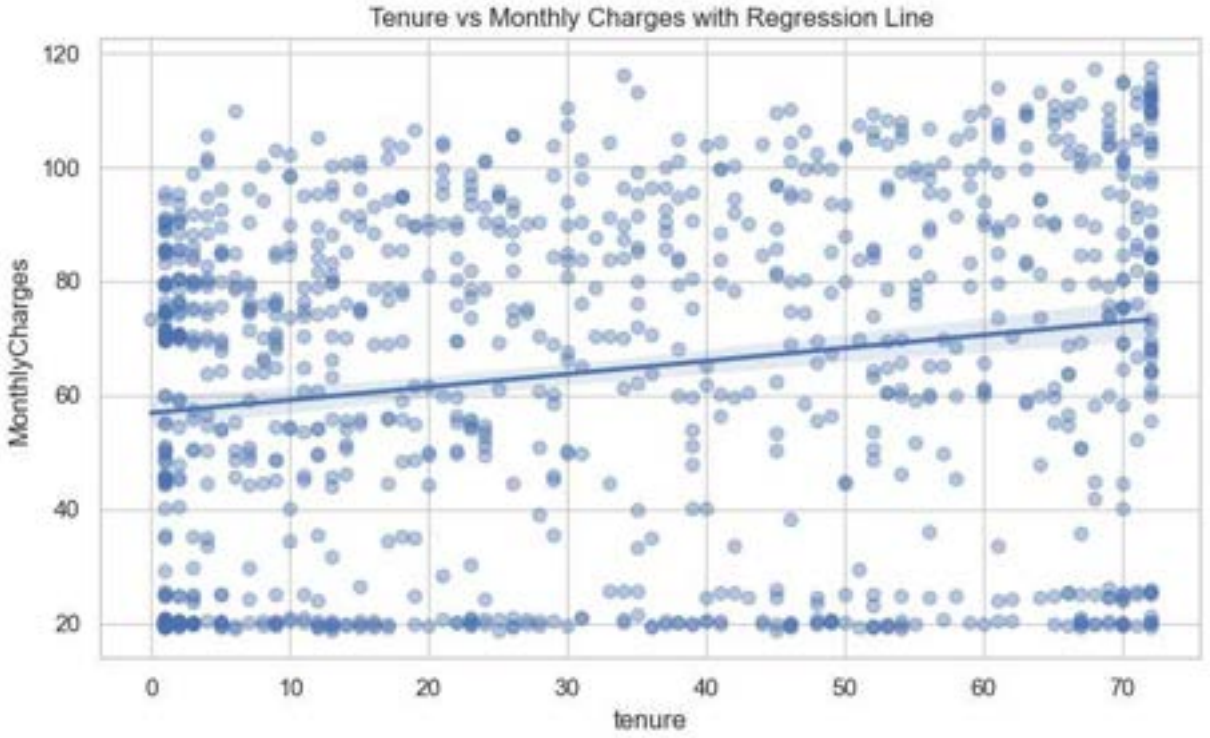
print("Spearman Correlation:")
print(data["tenure"].corr(data["MonthlyCharges"], method="spearman"))

plt.figure(figsize=(8, 5))

sns.regplot(
    data=data.sample(min(1000, len(data)), random_state=42),
    x="tenure",
    y="MonthlyCharges",
    scatter_kws={"alpha": 0.4}
)

plt.title("Tenure vs Monthly Charges with Regression Line")
plt.tight_layout()
plt.show()
```

```
Pearson Correlation:
0.24795895523972553
Spearman Correlation:
0.27653923173719586
```



In [79]:
```
grouped_stats = data.groupby(
    ["Contract", "Churn"], observed=True
)["MonthlyCharges"].agg(["count", "mean", "median", "std"])

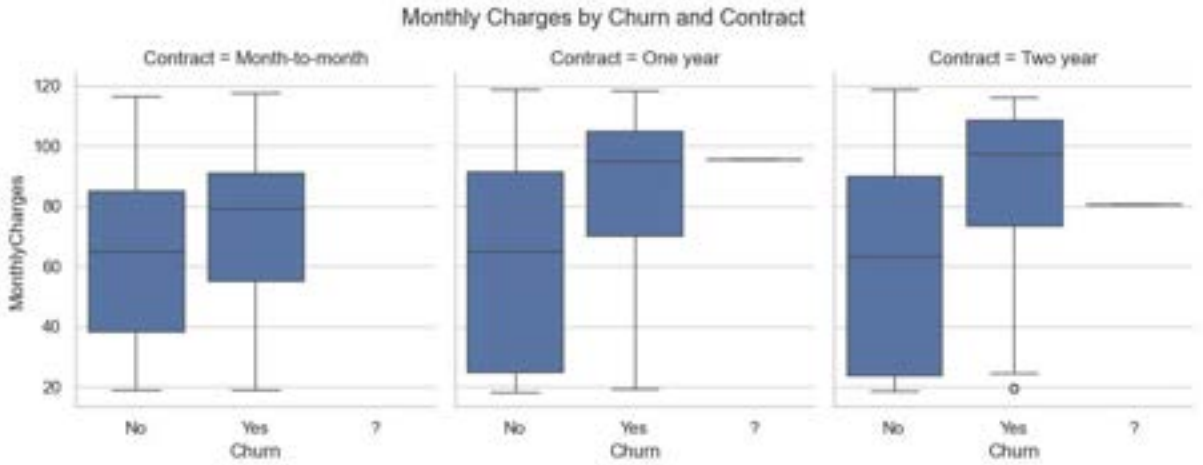
display(grouped_stats.round(2))
```

Contract	Churn	count	mean	median	std
Month-to-month	No	2219	61.46	64.95	27.87
	Yes	1654	73.04	79.05	24.08
One year	?	1	95.65	95.65	NaN
	No	1306	62.48	64.82	31.66
	Yes	165	84.91	94.85	25.66
Two year	?	1	80.60	80.60	NaN
	No	1645	60.02	63.30	34.55
	Yes	48	86.78	97.28	28.92

In [80]:
```
g = sns.catplot(
    data=data,
    x="Churn",
    y="MonthlyCharges",
    col="Contract",
    kind="box",
    height=4,
    aspect=0.9
```

```
)

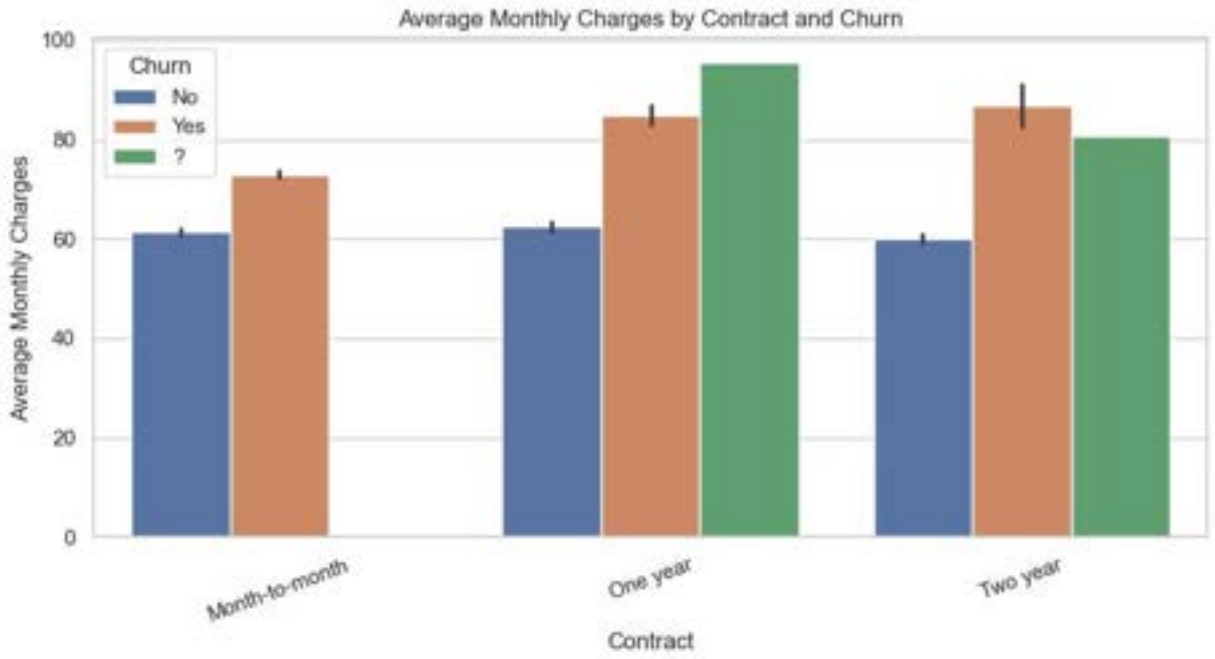
g.fig.suptitle("Monthly Charges by Churn and Contract", y=1.05)
plt.show()
```



In [81]:
```
plt.figure(figsize=(9, 5))

sns.barplot(
    data=data,
    x="Contract",
    y="MonthlyCharges",
    hue="Churn",
    errorbar="se"
)

plt.title("Average Monthly Charges by Contract and Churn")
plt.xlabel("Contract")
plt.ylabel("Average Monthly Charges")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

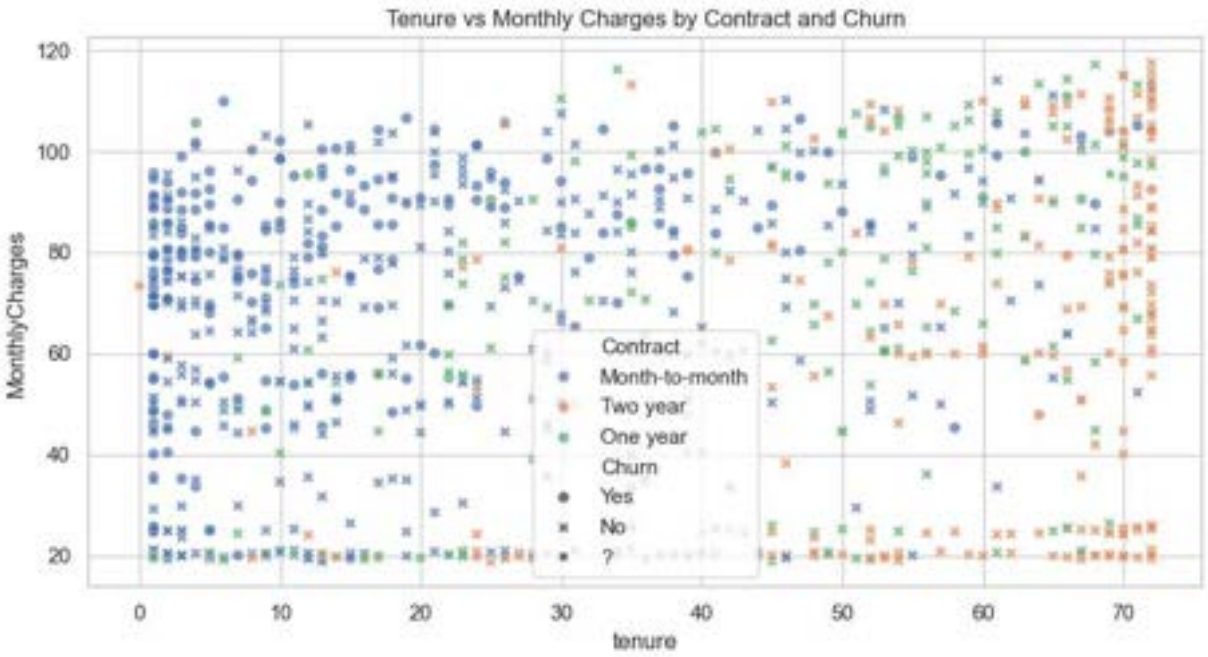


In [82]:
```
sample_data = data.sample(min(1000, len(data)), random_state=42)

plt.figure(figsize=(9, 5))

sns.scatterplot(
    data=sample_data,
    x="tenure",
    y="MonthlyCharges",
    hue="Contract",
    style="Churn",
    alpha=0.7
)

plt.title("Tenure vs Monthly Charges by Contract and Churn")
plt.tight_layout()
plt.show()
```



In [83]:
```
for contract in data["Contract"].dropna().unique():
    subset = data[data["Contract"] == contract]

    correlation = subset["tenure"].corr(subset["MonthlyCharges"])

    print(contract, "- Correlation:", round(correlation, 3))
```

```
Month-to-month - Correlation: 0.363
One year - Correlation: 0.486
Two year - Correlation: 0.41
```

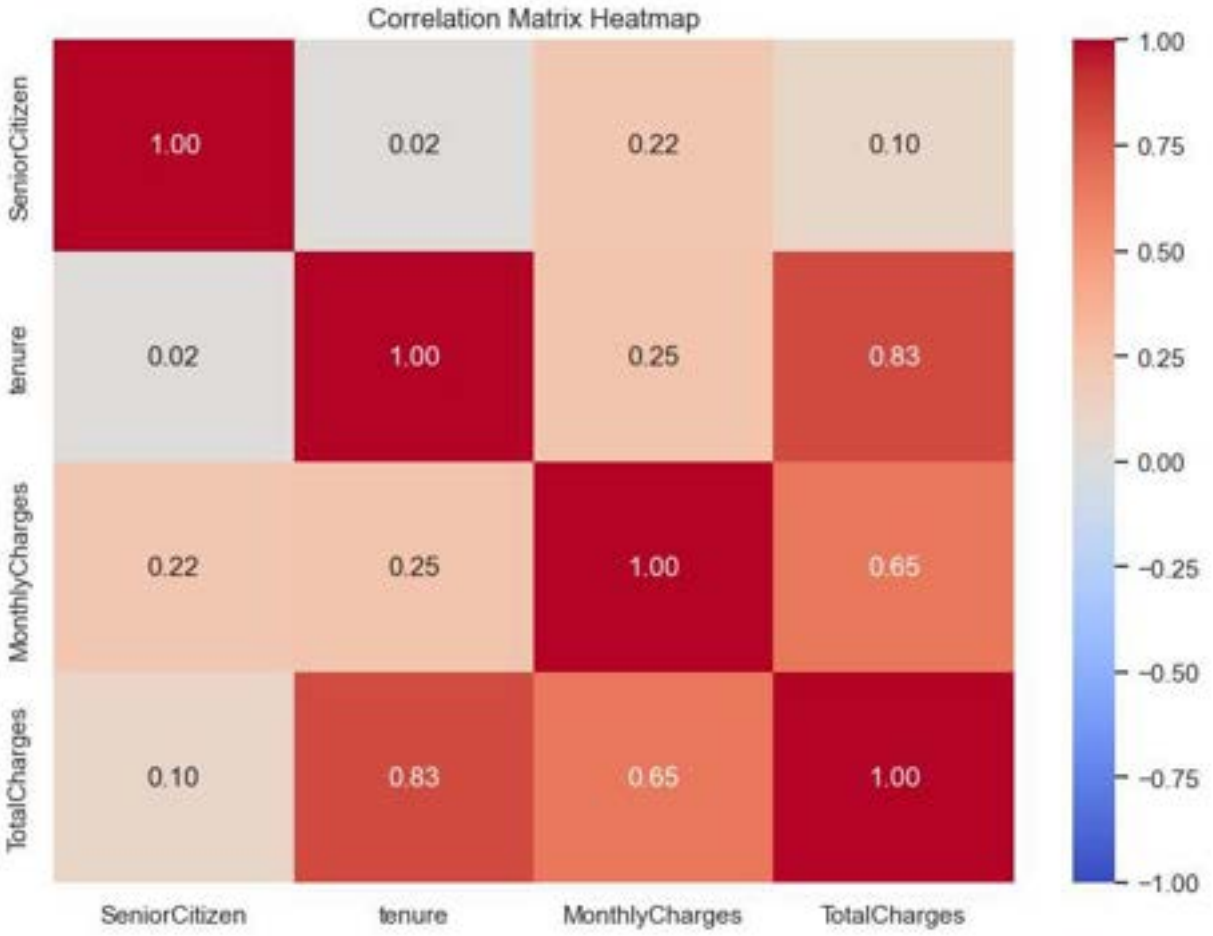
In [84]:
```
numeric_data = data[
    ["SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges"]
]

correlation_matrix = numeric_data.corr()

plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    fmt=".2f"
)

plt.title("Correlation Matrix Heatmap")
plt.tight_layout()
plt.show()
```

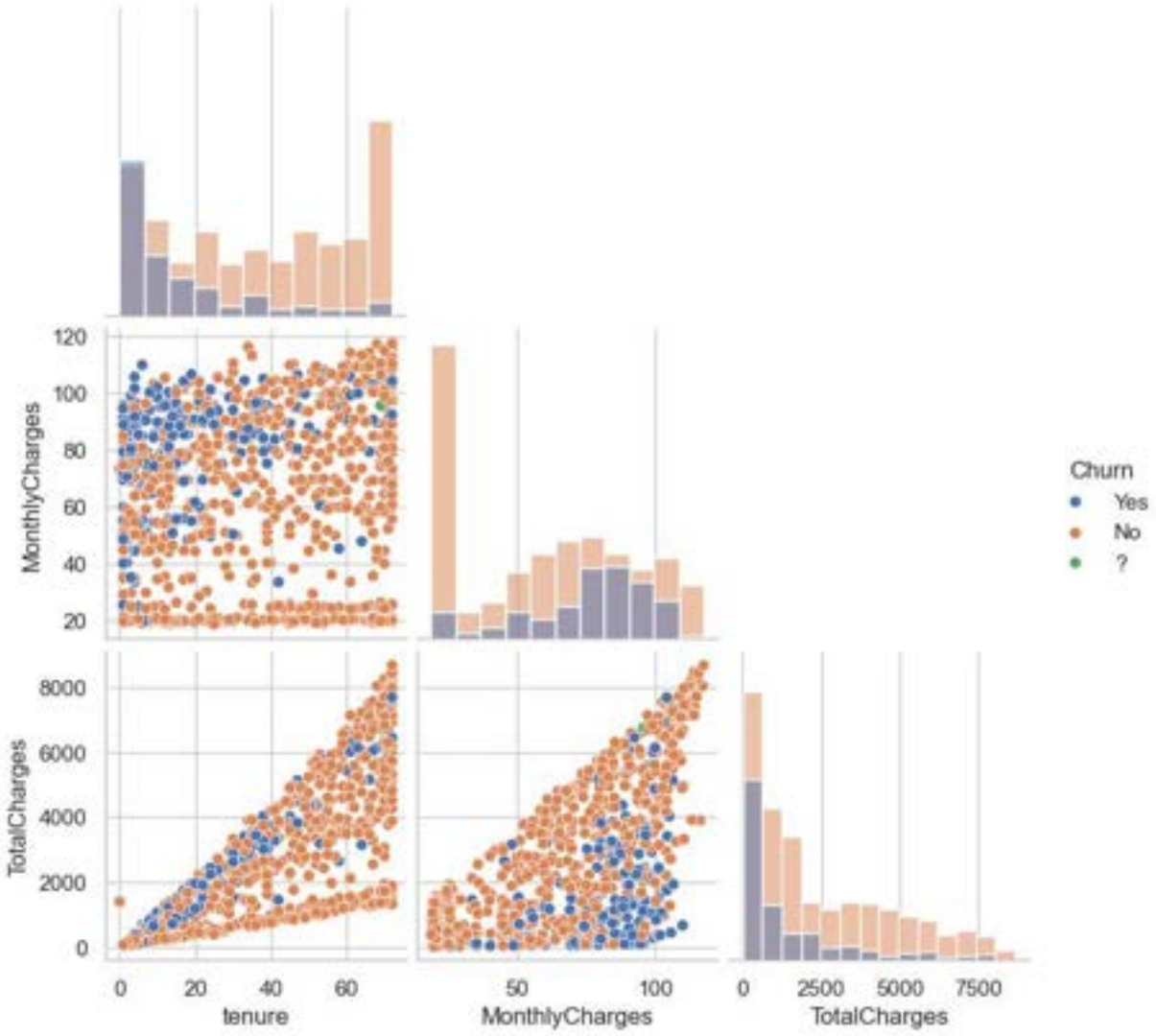


In [85]:

```
pair_data = data[
    ["tenure", "MonthlyCharges", "TotalCharges", "Churn"]
].sample(min(1000, len(data)), random_state=42)

sns.pairplot(
    pair_data,
    vars=["tenure", "MonthlyCharges", "TotalCharges"],
    hue="Churn",
    corner=True,
    diag_kind="hist"
)

plt.show()
```



Conclusion

One-dimensional, two-dimensional, and multidimensional analyses were performed on the Telco Customer Churn dataset. Descriptive statistics, frequency tables, contingency tables, histograms, boxplots, violin plots, scatter plots, grouped analysis, correlation matrices, and pair plots were used to understand the data and identify relationships between customer characteristics and churn.

Assessment 8: K-Means and Hierarchical Clustering

Name: Dharsha R

Register Number: 23BDS0274

Objective

To perform K-Means and hierarchical clustering on the Telco Customer Churn dataset, determine a suitable number of clusters using the elbow method, visualize clusters, and analyze the resulting groups.

In [86]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from scipy.spatial.distance import pdist

url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/telco%20custome

data = pd.read_csv(url)
data.columns = data.columns.str.strip()

for col in ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen"]:
    data[col] = pd.to_numeric(data[col], errors="coerce")

print("Dataset Shape:", data.shape)
display(data.head())
```

```
Dataset Shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	Female	0.0	Yes	No	1.0	No	No pl se
1	5575-GNVDE	Male	0.0	No	No	34.0	Yes	
2	3668-QPYBK	Male	0.0	No	No	2.0	Yes	
3	7795-CFOCW	Male	0.0	No	No	45.0	No	No pl se
4	9237-HQITU	Female	0.0	No	No	2.0	Yes	

5 rows × 21 columns

In [87]:
```
print("First 5 rows:")
display(data.head())

print("Dataset Information:")
```

```
data.info()

print("\nMissing values:")
print(data.isnull().sum())

print("\nTotal missing values:", data.isnull().sum().sum())

print("\nChurn distribution:")
print(data["Churn"].value_counts())
```

```
First 5 rows:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	Female	0.0	Yes	No	1.0	No	No pl se
1	5575-GNVDE	Male	0.0	No	No	34.0	Yes	
2	3668-QPYBK	Male	0.0	No	No	2.0	Yes	
3	7795-CFOCW	Male	0.0	No	No	45.0	No	No pl se
4	9237-HQITU	Female	0.0	No	No	2.0	Yes	

5 rows × 21 columns



```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   object 
 1   gender            7043 non-null   object 
 2   SeniorCitizen     7035 non-null   float64
 3   Partner           7038 non-null   object 
 4   Dependents        7043 non-null   object 
 5   tenure            7041 non-null   float64
 6   PhoneService      7043 non-null   object 
 7   MultipleLines     7043 non-null   object 
 8   InternetService   7043 non-null   object 
 9   OnlineSecurity    7043 non-null   object 
 10  OnlineBackup      7043 non-null   object 
 11  DeviceProtection  7043 non-null   object 
 12  TechSupport       7043 non-null   object 
 13  StreamingTV       7043 non-null   object 
 14  StreamingMovies   7043 non-null   object 
 15  Contract          7043 non-null   object 
 16  PaperlessBilling  7043 non-null   object 
 17  PaymentMethod     7043 non-null   object 
 18  MonthlyCharges    7040 non-null   float64
 19  TotalCharges      7032 non-null   float64
 20  Churn             7042 non-null   object 
dtypes: float64(4), object(17)
memory usage: 1.1+ MB

Missing values:
customerID           0
gender               0
SeniorCitizen        8
Partner              5
Dependents           0
tenure               2
PhoneService         0
MultipleLines        0
InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       3
TotalCharges        11
Churn                1
dtype: int64

Total missing values: 30
```

```
Churn distribution:
Churn
No     5172
Yes    1868
?         2
Name: count, dtype: int64
```

In [88]:
```
features = ["tenure", "MonthlyCharges", "TotalCharges", "SeniorCitizen"]

cluster_data = data[features].copy()

cluster_data = cluster_data.replace([np.inf, -np.inf], np.nan)

cluster_data = cluster_data.fillna(cluster_data.median())

print("Selected Features:")
display(cluster_data.head())

print("Missing Values After Cleaning:")
print(cluster_data.isnull().sum())
```

```
Selected Features:
```

	tenure	MonthlyCharges	TotalCharges	SeniorCitizen
0	1.0	29.85	29.85	0.0
1	34.0	56.95	1889.50	0.0
2	2.0	53.85	108.15	0.0
3	45.0	42.30	1840.75	0.0
4	2.0	70.70	151.65	0.0

```
Missing Values After Cleaning:
tenure            0
MonthlyCharges    0
TotalCharges      0
SeniorCitizen     0
dtype: int64
```

In [89]:
```
scaler = StandardScaler()

scaled_data = scaler.fit_transform(cluster_data)

scaled_df = pd.DataFrame(
    scaled_data,
    columns=features,
    index=cluster_data.index
)

display(scaled_df.head())

print("Standardized Dataset Shape:", scaled_df.shape)
```

	tenure	MonthlyCharges	TotalCharges	SeniorCitizen
0	-1.277534	-1.160750	-0.994242	-0.439916
1	0.066283	-0.259791	-0.173244	-0.439916
2	-1.236812	-0.362853	-0.959674	-0.439916
3	0.514222	-0.746841	-0.194766	-0.439916
4	-1.236812	0.197338	-0.940470	-0.439916

```
Standardized Dataset Shape: (7043, 4)
```

In [90]:
```
kmeans = KMeans(
    n_clusters=2,
    random_state=42,
    n_init=10
)

kmeans_labels = kmeans.fit_predict(scaled_data)

data["KMeans_Cluster"] = kmeans_labels

print("Cluster Centers:")
display(pd.DataFrame(kmeans.cluster_centers_, columns=features))

print("\nCluster Counts:")
print(data["KMeans_Cluster"].value_counts().sort_index())
```

```
Cluster Centers:
```

	tenure	MonthlyCharges	TotalCharges	SeniorCitizen
0	-0.505153	-0.426398	-0.628056	-0.110634
1	0.979735	0.826992	1.218104	0.214573

```
Cluster Counts:
KMeans_Cluster
0    4645
1    2398
Name: count, dtype: int64
```

In [91]:
```
wcss = []

for k in range(1, 11):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    model.fit(scaled_data)
    wcss.append(model.inertia_)

wcss_table = pd.DataFrame({
    "Number of Clusters (K)": range(1, 11),
```

```
    "WCSS": wcss
})

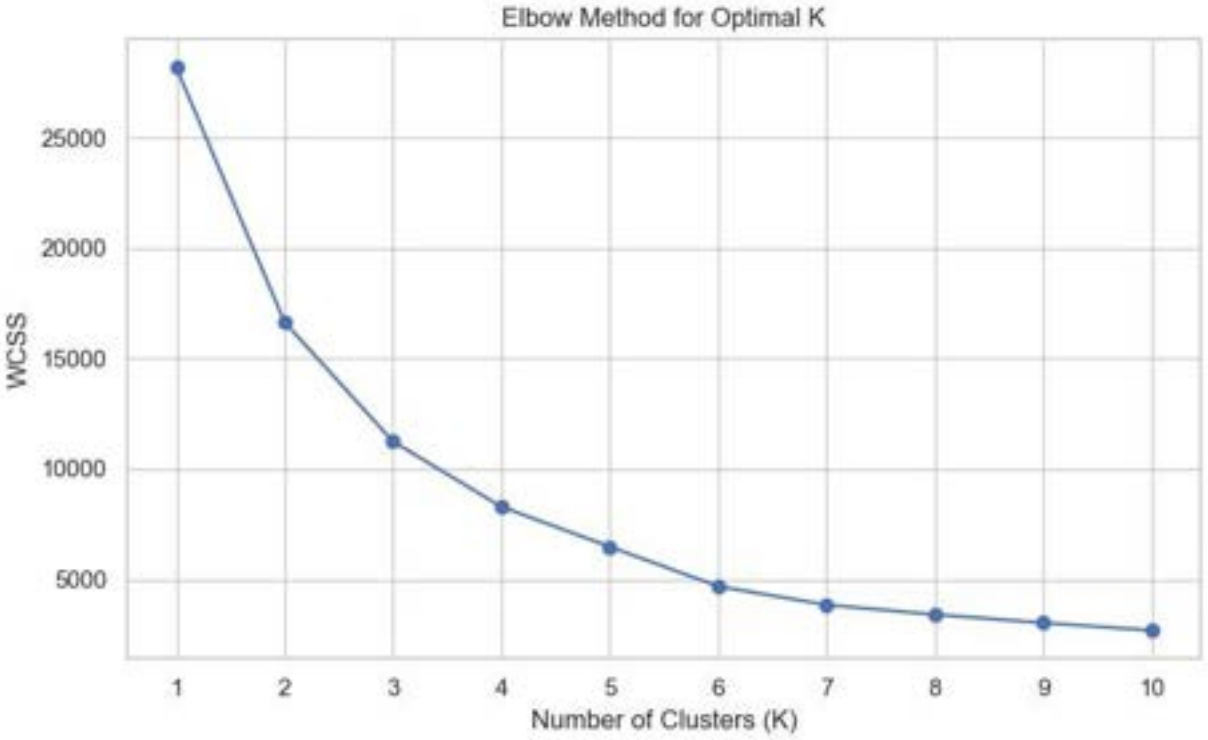
display(wcss_table)
```

	Number of Clusters (K)	WCSS
0	1	28172.000000
1	2	16647.364954
2	3	11244.211752
3	4	8288.927792
4	5	6480.127623
5	6	4684.741406
6	7	3847.276405
7	8	3406.959958
8	9	3033.942509
9	10	2689.581571

In [92]:
```
plt.figure(figsize=(8, 5))

plt.plot(range(1, 11), wcss, marker="o")

plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.xticks(range(1, 11))
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [93]:
```
optimal_k = 3

kmeans_final = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)

final_labels = kmeans_final.fit_predict(scaled_data)

data["KMeans_Cluster_3"] = final_labels

print("Cluster Counts:")
print(data["KMeans_Cluster_3"].value_counts().sort_index())

print("\nCluster Centers:")
display(pd.DataFrame(
    scaler.inverse_transform(kmeans_final.cluster_centers_),
    columns=features
))
```

```
Cluster Counts:
KMeans_Cluster_3
0    1142
1    4026
2    1875
Name: count, dtype: int64

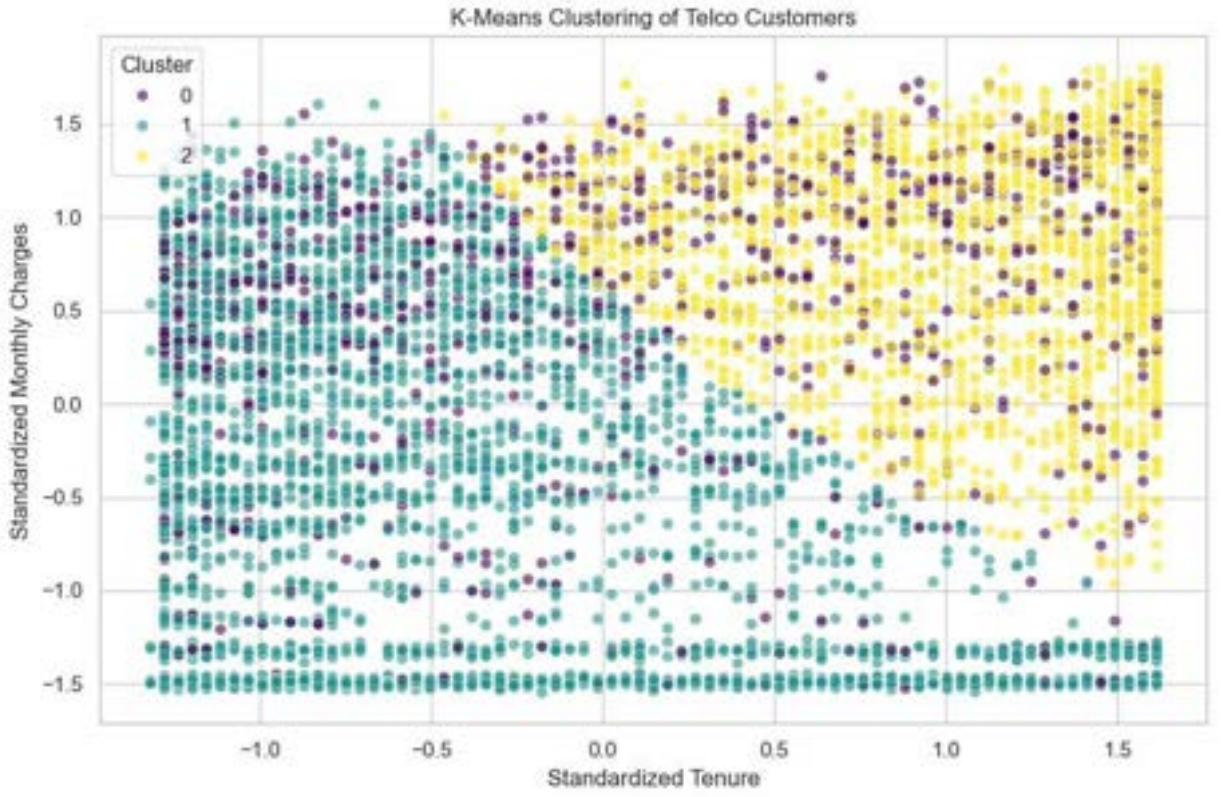
Cluster Centers:
```

	tenure	MonthlyCharges	TotalCharges	SeniorCitizen
0	33.295972	79.820359	2810.465193	1.000000e+00
1	20.522863	49.369148	839.011649	6.661338e-16
2	57.213639	88.608657	5053.706340	-6.106227e-16

In [94]:
```
plt.figure(figsize=(9, 6))

sns.scatterplot(
    x=scaled_data[:, 0],
    y=scaled_data[:, 1],
    hue=final_labels,
    palette="viridis",
    alpha=0.6
)

plt.title("K-Means Clustering of Telco Customers")
plt.xlabel("Standardized Tenure")
plt.ylabel("Standardized Monthly Charges")
plt.legend(title="Cluster")
plt.tight_layout()
plt.show()
```



In [95]:
```
sample_size = min(500, len(scaled_data))

rng = np.random.RandomState(42)
sample_indices = rng.choice(
    len(scaled_data),
    size=sample_size,
```

```
    replace=False
)

hierarchical_sample = scaled_data[sample_indices]

distance_matrix = pdist(
    hierarchical_sample,
    metric="euclidean"
)

linkage_matrix = linkage(
    distance_matrix,
    method="ward"
)

print("Hierarchical clustering completed.")
print("Sample size:", sample_size)
print("Linkage matrix shape:", linkage_matrix.shape)
```

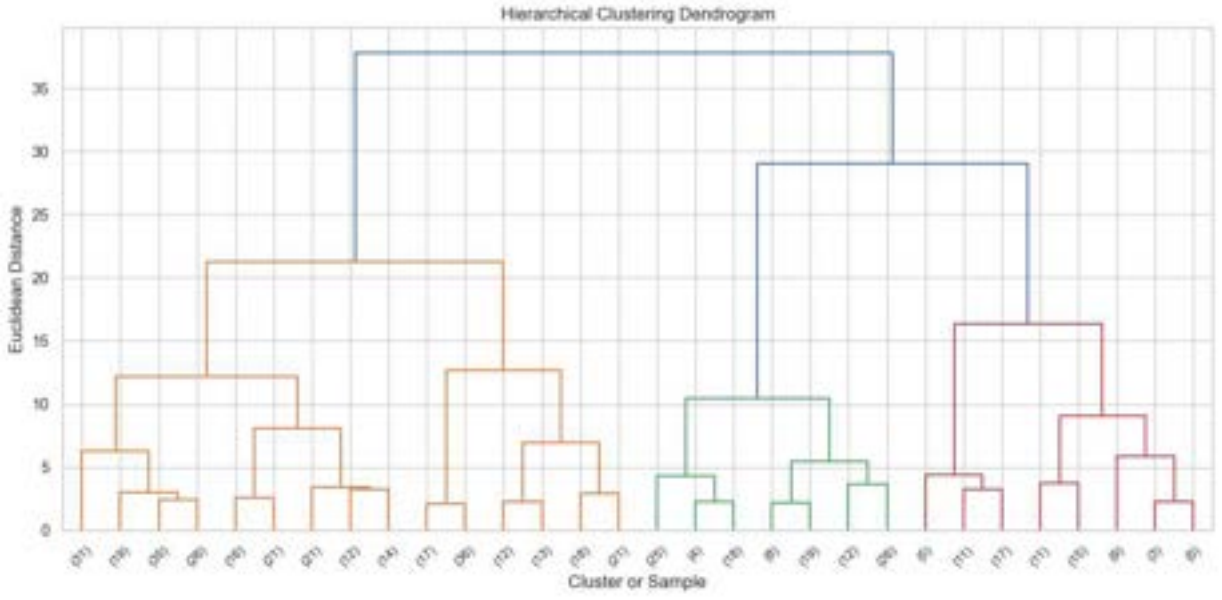
```
Hierarchical clustering completed.
Sample size: 500
Linkage matrix shape: (499, 4)
```

In [96]:

```
plt.figure(figsize=(12, 6))

dendrogram(
    linkage_matrix,
    truncate_mode="lastp",
    p=30,
    leaf_rotation=45,
    leaf_font_size=9
)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Cluster or Sample")
plt.ylabel("Euclidean Distance")
plt.tight_layout()
plt.show()
```



In [97]:
```
hierarchical_labels = fcluster(
    linkage_matrix,
    t=3,
    criterion="maxclust"
)

hierarchical_result = pd.DataFrame({
    "Sample_Index": sample_indices,
    "Hierarchical_Cluster": hierarchical_labels
})

print("Hierarchical Cluster Counts:")
print(hierarchical_result["Hierarchical_Cluster"].value_counts().sort_index())

display(hierarchical_result.head(10))
```

```
Hierarchical Cluster Counts:
Hierarchical_Cluster
1    312
2    112
3     76
Name: count, dtype: int64
```

	Sample_Index	Hierarchical_Cluster
0	185	1
1	2715	1
2	3825	1
3	1807	1
4	132	2
5	1263	3
6	3732	1
7	1672	2
8	811	2
9	2526	1

In [98]:
```
cluster_summary = data.groupby(
    "KMeans_Cluster_3"
)[features].mean()

print("Average Feature Values for Each K-Means Cluster:")
display(cluster_summary.round(2))

print("\nChurn Distribution Within Each Cluster:")
display(pd.crosstab(
    data["KMeans_Cluster_3"],
    data["Churn"],
```

```
    normalize="index"
).round(3))
```

```
Average Feature Values for Each K-Means Cluster:
```

KMeans_Cluster_3	tenure	MonthlyCharges	TotalCharges	SeniorCitizen
0	33.30	79.82	2810.47	1.0
1	20.52	49.38	838.36	0.0
2	57.24	88.62	5056.32	0.0

```
Churn Distribution Within Each Cluster:
```

Churn	?	No	Yes
KMeans_Cluster_3			
0	0.000	0.583	0.417
1	0.000	0.721	0.279
2	0.001	0.854	0.145

Conclusion

K-Means and hierarchical clustering were applied to the Telco Customer Churn dataset. Numerical features were cleaned and standardized before clustering. The elbow method was used to examine WCSS for different values of K. K-Means clusters were visualized using a scatter plot, while hierarchical clustering was performed using Euclidean distance and Ward linkage. A dendrogram and cluster counts were used to study the resulting groups.

Assessment 9: Principal Component Analysis (PCA)

Name: Dharsha R

Register Number: 23BDS0274

Objective

To apply Principal Component Analysis to the Telco Customer Churn dataset, standardize numerical features, analyze explained variance, visualize principal components, and display PCA scores.

In [99]:
```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/telco%20custome

data = pd.read_csv(url)
data.columns = data.columns.str.strip()

print("Dataset Shape:", data.shape)
display(data.head())
```

```
Dataset Shape: (7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleL
0	7590-VHVEG	Female	0.0	Yes	No	1	No	No pl se
1	5575-GNVDE	Male	0.0	No	No	34	Yes	
2	3668-QPYBK	Male	0.0	No	No	2	Yes	
3	7795-CFOCW	Male	0.0	No	No	45	No	No pl se
4	9237-HQITU	Female	0.0	No	No	2	Yes	

5 rows × 21 columns

In [100…

```
numeric_data = data[
    ["SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges"]
].copy()

for col in numeric_data.columns:
    numeric_data[col] = pd.to_numeric(
        numeric_data[col], errors="coerce"
    )

numeric_data = numeric_data.replace([np.inf, -np.inf], np.nan)
numeric_data = numeric_data.fillna(numeric_data.median())

print("Missing Values:")
print(numeric_data.isnull().sum())

display(numeric_data.head())
```

```
Missing Values:
SeniorCitizen     0
tenure            0
MonthlyCharges    0
TotalCharges      0
dtype: int64
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
0	0.0	1.0	29.85	29.85
1	0.0	34.0	56.95	1889.50
2	0.0	2.0	53.85	108.15
3	0.0	45.0	42.30	1840.75
4	0.0	2.0	70.70	151.65

In [101…

```python
scaler = StandardScaler()

scaled_data = scaler.fit_transform(numeric_data)

scaled_df = pd.DataFrame(
    scaled_data,
    columns=numeric_data.columns
)

print("Standardized Data:")
display(scaled_df.head())
```

```
Standardized Data:
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
0	-0.439916	-1.277534	-1.160750	-0.994242
1	-0.439916	0.066283	-0.259791	-0.173244
2	-0.439916	-1.236812	-0.362853	-0.959674
3	-0.439916	0.514222	-0.746841	-0.194766
4	-0.439916	-1.236812	0.197338	-0.940470

In [102…

```python
pca = PCA()

pca_result = pca.fit_transform(scaled_data)

pca_columns = [
    "PC" + str(i + 1) for i in range(pca_result.shape[1])
]

pca_scores = pd.DataFrame(
    pca_result,
    columns=pca_columns
)
```

```
print("PCA completed successfully.")
print("Number of Principal Components:", pca.n_components_)
```

```
PCA completed successfully.
Number of Principal Components: 4
```

In [103…

```
explained_variance = pca.explained_variance_ratio_ * 100
cumulative_variance = np.cumsum(explained_variance)

pca_summary = pd.DataFrame({
    "Principal Component": pca_columns,
    "Explained Variance (%)": explained_variance,
    "Cumulative Variance (%)": cumulative_variance
})

display(pca_summary.round(2))
```

	Principal Component	Explained Variance (%)	Cumulative Variance (%)
0	PC1	55.21	55.21
1	PC2	26.37	81.58
2	PC3	16.92	98.49
3	PC4	1.51	100.00

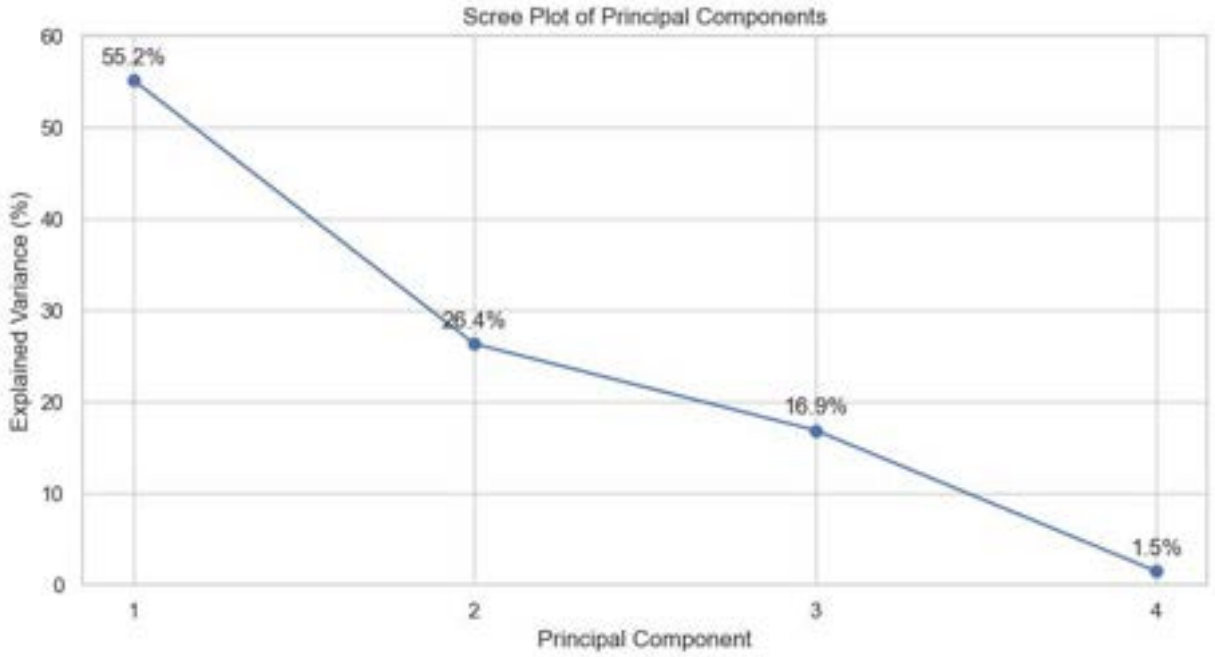
In [104…

```
plt.figure(figsize=(9, 5))

plt.plot(
    range(1, len(explained_variance) + 1),
    explained_variance,
    marker="o"
)

for i, value in enumerate(explained_variance):
    plt.annotate(
        f"{value:.1f}%",
        (i + 1, value),
        textcoords="offset points",
        xytext=(0, 8),
        ha="center"
    )

plt.title("Scree Plot of Principal Components")
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance (%)")
plt.xticks(range(1, len(explained_variance) + 1))
plt.ylim(0, 60)
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [105…

```
plt.figure(figsize=(10, 7))

sample_size = min(300, len(pca_result))
rng = np.random.RandomState(42)

sample_indices = rng.choice(
    len(pca_result),
    size=sample_size,
    replace=False
)

plt.scatter(
    pca_result[sample_indices, 0],
    pca_result[sample_indices, 1],
    alpha=0.35,
    s=20,
    label="Customers"
)

loadings = pca.components_.T

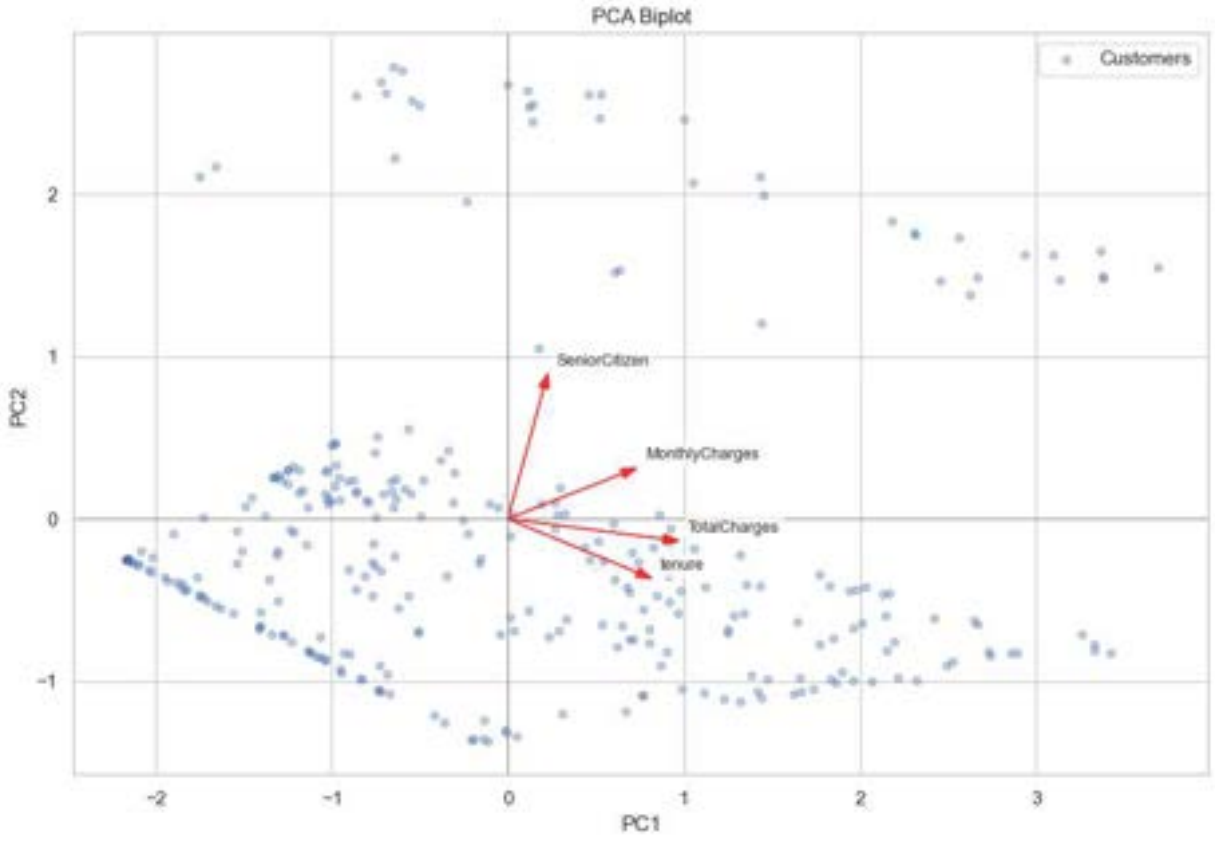
scale_x = np.std(pca_result[:, 0])
scale_y = np.std(pca_result[:, 1])

for i, feature in enumerate(numeric_data.columns):
    x = loadings[i, 0] * scale_x
    y = loadings[i, 1] * scale_y

    plt.arrow(
        0, 0, x, y,
        color="red",
        alpha=0.8,
        head_width=0.06,
        length_includes_head=True
    )
```

```
    plt.annotate(
        feature,
        (x, y),
        xytext=(5, 5),
        textcoords="offset points",
        fontsize=9,
        bbox=dict(boxstyle="round,pad=0.2", fc="white", alpha=0.7)
    )

plt.axhline(0, color="gray", linewidth=0.7)
plt.axvline(0, color="gray", linewidth=0.7)
plt.title("PCA Biplot")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
plt.tight_layout()
plt.show()
```



In [106…

```
print("First Six PCA Scores:")
display(pca_scores.head(6).round(4))
```

First Six PCA Scores:

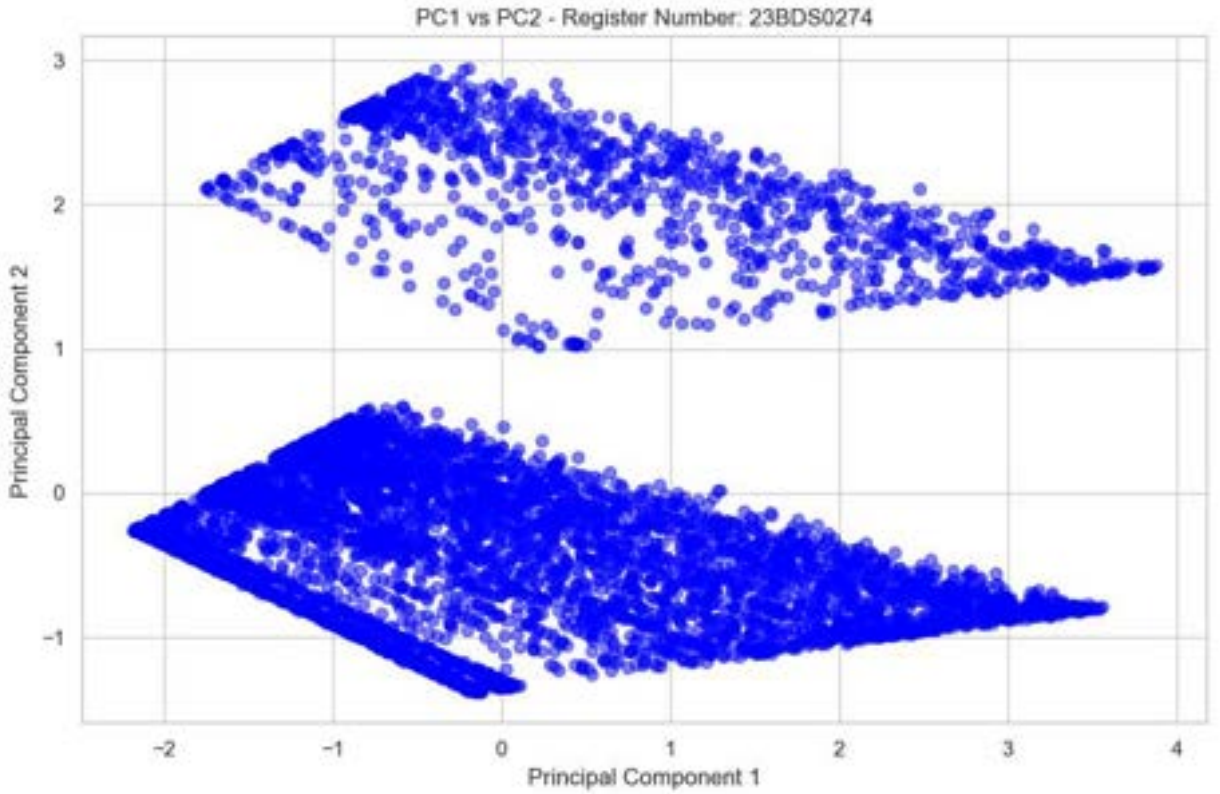
	PC1	PC2	PC3	PC4
0	-1.9957	-0.1486	0.0233	0.3950
1	-0.2726	-0.4638	-0.0162	-0.0759
2	-1.5567	0.0760	0.5801	0.1013
3	-0.2804	-0.7702	-0.5970	-0.1595
4	-1.2676	0.2447	0.9856	-0.0928
5	-0.4643	0.4121	1.5556	-0.3670

In [107…

```
plt.figure(figsize=(9, 6))

plt.scatter(
    pca_scores["PC1"],
    pca_scores["PC2"],
    color="blue",
    marker="o",
    alpha=0.5
)

plt.title("PC1 vs PC2 - Register Number: 23BDS0274")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [108...
```
print(
    "Total variance explained by the first two components:",
    round(cumulative_variance[1], 2),
    "%"
)

print(
    "Total variance explained by all components:",
    round(cumulative_variance[-1], 2),
    "%"
)
```

```
Total variance explained by the first two components: 81.58 %
Total variance explained by all components: 100.0 %
```

Conclusion

Principal Component Analysis was performed on the Telco Customer Churn dataset. Numerical features were cleaned and standardized before PCA was applied. The explained variance and cumulative variance were examined using a summary table and scree plot. A biplot was created to visualize observations and feature loadings, the first six PCA scores were displayed, and PC1 was plotted against PC2.

Additional Experiment: Linear Regression and Model Evaluation

Name: Dharsha R

Register Number: 23BDS0274

Objective

To build a Linear Regression model, predict monthly charges based on customer tenure, and evaluate model performance using MAE, MSE, RMSE, and R-squared.

In [109...
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

url = "https://raw.githubusercontent.com/salemprakash/EDA/main/Data/telco%20custome

data = pd.read_csv(url)
data.columns = data.columns.str.strip()
```

```
data["tenure"] = pd.to_numeric(data["tenure"], errors="coerce")
data["MonthlyCharges"] = pd.to_numeric(data["MonthlyCharges"], errors="coerce")

data = data.dropna(subset=["tenure", "MonthlyCharges"])

X = data[["tenure"]]
y = data["MonthlyCharges"]

print("Dataset Shape:", data.shape)
display(data[["tenure", "MonthlyCharges"]].head())
```

```
Dataset Shape: (7038, 21)
```

	tenure	MonthlyCharges
0	1.0	29.85
1	34.0	56.95
2	2.0	53.85
3	45.0	42.30
4	2.0	70.70

In [110…

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
Training Samples: 5630
Testing Samples: 1408
```

In [111…

```
model = LinearRegression()

model.fit(X_train, y_train)

print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_[0])
```

```
Intercept: 55.01120688840053
Coefficient: 0.31066835986199975
```

In [112…

```
y_pred = model.predict(X_test)

results = pd.DataFrame({
    "Actual Monthly Charges": y_test.values,
    "Predicted Monthly Charges": y_pred
})

display(results.head(10).round(2))
```

	Actual Monthly Charges	Predicted Monthly Charges
0	19.80	66.51
1	91.50	65.88
2	84.25	62.16
3	70.50	70.54
4	26.80	62.78
5	19.85	66.51
6	75.35	56.25
7	98.70	62.16
8	20.35	63.40
9	114.60	77.07

In [113…

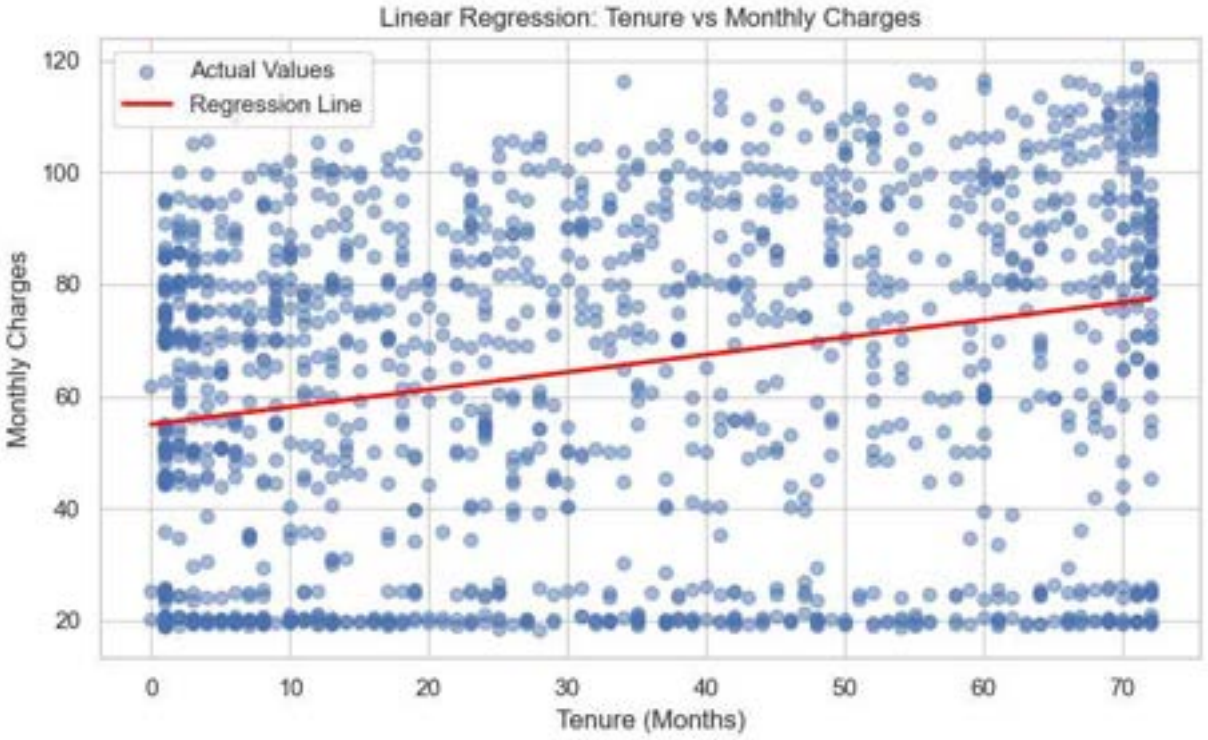
```
plt.figure(figsize=(8, 5))

plt.scatter(
    X_test["tenure"],
    y_test,
    alpha=0.5,
    label="Actual Values"
)

sorted_indices = np.argsort(X_test["tenure"].values)

plt.plot(
    X_test["tenure"].values[sorted_indices],
    y_pred[sorted_indices],
    color="red",
    linewidth=2,
    label="Regression Line"
)

plt.title("Linear Regression: Tenure vs Monthly Charges")
plt.xlabel("Tenure (Months)")
plt.ylabel("Monthly Charges")
plt.legend()
plt.tight_layout()
plt.show()
```



In [114…

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Model Evaluation Metrics")
print("------------------------")
print("Mean Absolute Error (MAE):", round(mae, 3))
print("Mean Squared Error (MSE):", round(mse, 3))
print("Root Mean Squared Error (RMSE):", round(rmse, 3))
print("R-squared (R2):", round(r2, 3))
```

```
Model Evaluation Metrics
------------------------
Mean Absolute Error (MAE): 25.237
Mean Squared Error (MSE): 854.193
Root Mean Squared Error (RMSE): 29.227
R-squared (R2): 0.048
```

In [115…

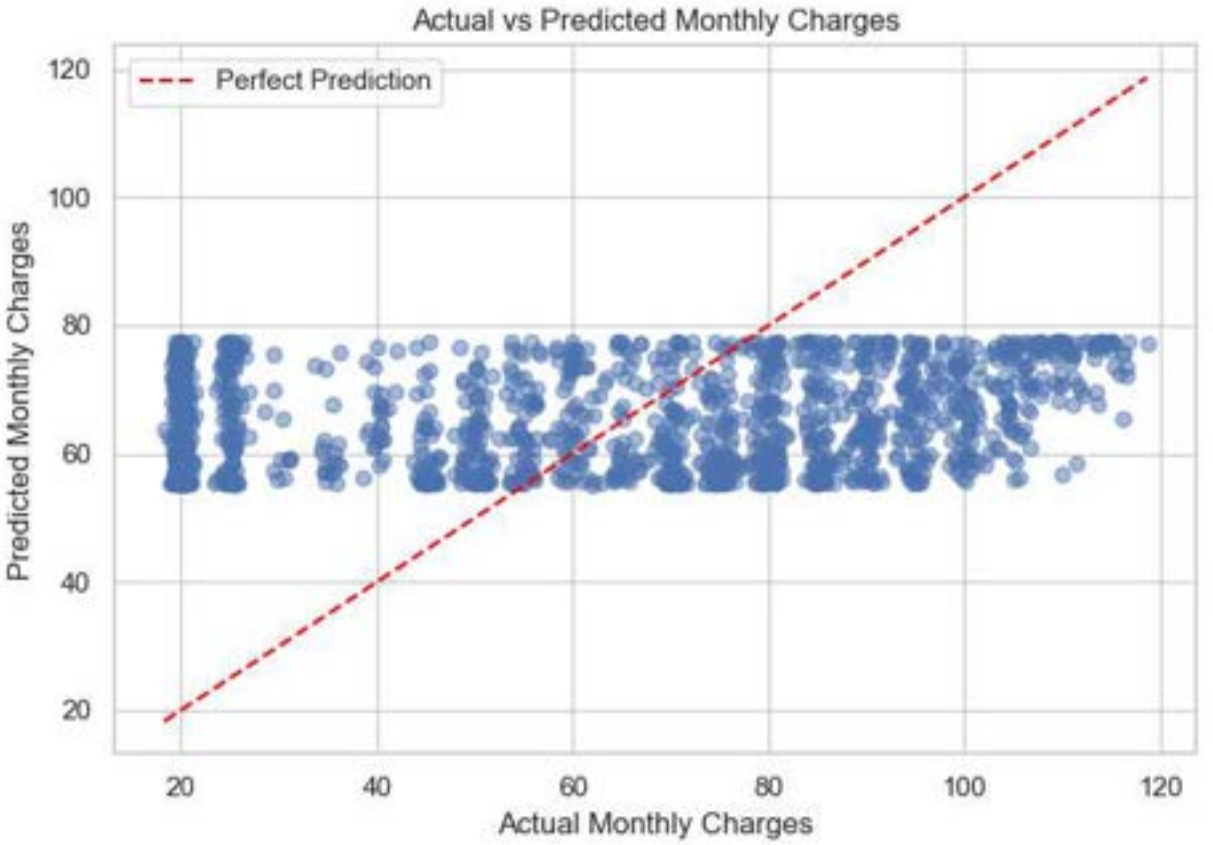
```
plt.figure(figsize=(7, 5))

plt.scatter(y_test, y_pred, alpha=0.5)

minimum = min(y_test.min(), y_pred.min())
maximum = max(y_test.max(), y_pred.max())

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    color="red",
    linestyle="--",
    label="Perfect Prediction"
)
```

```
plt.title("Actual vs Predicted Monthly Charges")
plt.xlabel("Actual Monthly Charges")
plt.ylabel("Predicted Monthly Charges")
plt.legend()
plt.tight_layout()
plt.show()
```



Conclusion

A Linear Regression model was developed to predict monthly charges using customer tenure. The dataset was divided into training and testing sets. The model was trained and used to predict monthly charges. Performance was evaluated using MAE, MSE, RMSE, and R-squared. Actual and predicted values were visualized to understand model performance.

Video Presentation link :

https://drive.google.com/file/d/1kvjjf0XaX0ULUA1vTQUypF9x1r4oUNCJ/view?usp=sharing